BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

HCM-UTE

BÁO CÁO CUỐI KỲ

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VÀ KỸ THUẬT PHẦN MỀM

THIẾT KẾ HỆ THỐNG GIẢ LẬP CHIP-8 BẰNG C++/SDL2

GVHD: **Th.S Trần Thị Quỳnh Như**

NHÓM: **9**

SVTH:

Nguyễn Trần Khánh Nguyên	23161301
Nguyễn Quang Quân	23161316
Lê Văn Hậu	23161257
Phạm Đắc Vinh	23161363
Phạm Văn Nhiệm	23161305

Tp. Hồ Chí Minh, tháng 6 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ

----------□□□----------



BÁO CÁO CUỐI KỲ
LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VÀ KỸ THUẬT PHẦN MỀM
CHỦ ĐỀ: THIẾT KẾ HỆ THỐNG GIẢ LẬP CHIP-8 BẰNG C++/SDL2

GVHD: **Th.S Trần Thị Quỳnh Như**

NHÓM: **9**

SVTH:	Nguyễn Trần Khánh Nguyên	23161301
	Nguyễn Quang Quân	23161316
	Lê Văn Hậu	23161257
	Phạm Đắc Vinh	23161363
	Phạm Văn Nhiệm	23161305

Tp. Hồ Chí Minh, tháng 6 năm 2026

MỤC LỤC

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI 1
1.1. Lý do chọn đề tài 1
1.2. Mục tiêu đề tài 2
1.3. Giới hạn của đề tài 3
1.4. Đối tượng và phạm vi nghiên cứu 5
1.5. Cấu trúc báo cáo 7
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT 10
2.1. Tổng quan về giả lập 10
2.1.1. Khái niệm giả lập 11
2.1.2. Nguyên lý hoạt động của trình giả lập 12
2.2. Giới thiệu CHIP-8 14
2.2.1. Đặc điểm của CHIP-8 16
2.2.2. ROM trong CHIP-8 18
2.3. Kiến trúc CHIP-8 20
2.3.1. Bộ nhớ 22
2.3.2. Thanh ghi 24
2.3.3. Stack 25
2.3.4. Bộ đếm chương trình 27
2.3.5. Bộ định thời 29
2.3.6. Màn hình và bàn phím 30
2.4. Tập lệnh CHIP-8 32
2.4.1. Chu trình Fetch - Decode - Execute 33
2.4.2. Nhóm lệnh điều khiển 35
2.4.3. Nhóm lệnh xử lý dữ liệu 36
2.4.4. Nhóm lệnh đồ họa 38
2.4.5. Nhóm lệnh bàn phím 40
2.5. Lập trình hướng đối tượng trong C++ 42

2.5.1. Lớp và đối tượng....43
2.5.2. Đóng gói dữ liệu....44
2.5.3. Áp dụng OOP trong chương trình....46
2.6. Thư viện SDL2....48
2.6.1. Tạo cửa sổ hiển thị....49
2.6.2. Ánh xạ và xử lý bàn phím....51
2.6.3. Cập nhật hình ảnh lên màn hình....53
CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG....55
3.1. Phân tích yêu cầu....55
3.2. Thiết kế kiến trúc hệ thống....57
3.2.1. Thành phần xử lý CHIP-8....58
3.2.2. Thành phần hiển thị và nhập liệu....59
3.2.3. Thành phần điều khiển chương trình chính....59
3.3. Thiết kế lớp Chip8....60
3.3.1. Thuộc tính của lớp....61
3.3.2. Phương thức tải ROM....62
3.3.3. Phương thức xử lý chu kỳ lệnh....63
3.3.4. Các hàm xử lý opcode....65
3.4. Thiết kế lớp Platform....66
3.4.1. Thuộc tính của lớp....67
3.4.2. Phương thức cập nhật màn hình....67
3.4.3. Phương thức xử lý bàn phím....68
3.5. Thiết kế luồng hoạt động....70
3.5.1. Luồng khởi động chương trình....71
3.5.2. Luồng tải ROM....72
3.5.3. Luồng xử lý vòng lặp game....73
3.5.4. Luồng thoát chương trình....74
CHƯƠNG 4 CÀI ĐẶT CHƯƠNG TRÌNH....76
VÀ KẾT QUẢ KIỂM THỬ....76

4.1. Môi trường cài đặt 76
4.1.1. Hệ điều hành 77
4.1.2. Trình biên dịch 78
4.2. Cấu trúc thư mục dự án 80
4.2.1. Thư mục src 81
4.2.2. Thư mục roms 83
4.2.3. File cấu hình CMake 85
4.3. Cài đặt lớp Chip8 87
4.3.1. Khởi tạo bộ nhớ và thanh ghi 87
4.3.2. Nạp fontset vào bộ nhớ 88
4.3.3. Tải ROM vào bộ nhớ 89
4.3.4. Xử lý chu kỳ lệnh 90
4.4. Cài đặt xử lý opcode 91
4.4.1. Nhóm lệnh nhảy và gọi chương trình con 91
4.4.2. Nhóm lệnh gán và tính toán 92
4.4.3. Nhóm lệnh vẽ sprite 93
4.4.4. Nhóm lệnh bàn phím 94
4.4.5. Nhóm lệnh timer và bộ nhớ 94
4.5. Cài đặt lớp Platform 95
4.5.1. Khởi tạo SDL2 95
4.5.2. Tạo cửa sổ hiển thị 95
4.5.3. Render màn hình CHIP-8 95
4.5.4. Ánh xạ bàn phím 96
4.6. Cài đặt chương trình chính 97
4.6.1. Xử lý tham số dòng lệnh 97
4.6.2. Tìm kiếm ROM 98
4.6.3. Khởi tạo emulator 98
4.6.4. Xử lý vòng lặp chính 99
4.7. Hướng dẫn build và chạy chương trình 99

4.8. Kết quả kiểm thử 5 game 100
CHƯƠNG 5 KẾT LUẬN.....107
5.1. Kết luận.....107
5.2. Hướng phát triển.....107
TÀI LIỆU THAM KHẢO 109

DANH MỤC HÌNH ẢNH

Hình 1 Nguyên lý hoạt động của trình giả lập 13
Hình 2 Sơ đồ kiến trúc thành phần và chiều trao đổi dữ liệu của hệ thống 57
Hình 3 Sơ đồ lớp Chip8 với giao diện công khai, trạng thái nội bộ và cơ chế ... 60
Hình 4 Lưu đồ fetch - decode - execute và cập nhật timer của Chip8::Cycle() . 64
Hình 5 Sơ đồ tuần tự từ sự kiện người dùng đến khung hình SDL2 68
Hình 6 Lưu đồ tiếp nhận và ánh xạ sự kiện bàn phím của Platform::ProcessInput() 70
Hình 7 Lưu đồ khởi động, thực thi và kết thúc của chương trình chính 71
Hình 8 Cây thư mục tổng quan của dự án 80
Hình 9 Các tệp mã nguồn trong thư mục src 82
Hình 10 Một phần danh sách ROM trong thư mục roms 84
Hình 11 Sơ đồ phân nhóm xử lý opcode trong lớp Chip8 91
Hình 12 Nguyên lý vẽ sprite bằng phép XOR 93
Hình 13 Luồng hiển thị và nhập liệu qua SDL2 96
Hình 14 Sơ đồ ánh xạ bàn phím CHIP-8 sang bàn phím máy tính 96
Hình 15 Luồng tìm kiếm và chuẩn hóa đường dẫn ROM 98
Hình 16 Vòng lặp chính của emulator 99
Hình 17 Quy trình build và chạy kiểm thử chương trình 100
Hình 18 Kết quả chạy ROM Dodge 102
Hình 19 Kết quả chạy ROM Brix 103
Hình 20 Kết quả chạy ROM Tetris 103
Hình 21 Kết quả chạy ROM Submarine 104
Hình 22 Kết quả chạy ROM Tic-Tac-Toe 104

DANH MỤC BẢNG

Bảng 1 Các yêu cầu chức năng của hệ thống.....55
Bảng 2 Các yêu cầu chất lượng và ràng buộc kỹ thuật.....56
Bảng 3 Vai trò và giao diện giữa các thành phần chính.....58
Bảng 4 Thuộc tính và ý nghĩa mô phỏng trong lớp Chip8 61
Bảng 5 Các quyết định trong quy trình tải ROM.....63
Bảng 6 Thuật toán xử lý một chu kỳ lệnh CHIP-8 65
Bảng 7 Nhóm opcode đã tổ chức trong bảng giải mã.....66
Bảng 8 Thuộc tính và vòng đời tài nguyên của Platform 67
Bảng 9 Ánh xạ bàn phím hiện có trong Platform::ProcessInput() 69
Bảng 10 Các nhánh kết quả của quá trình khởi động và tải ROM 73
Bảng 11 Thuật toán vòng lặp chính 74
Bảng 12 Tổng hợp môi trường cài đặt và công cụ sử dụng.....77
Bảng 13 Vai trò của các công cụ trong quy trình build.....79
Bảng 14 Vai trò các thành phần chính trong thư mục dự án.....81
Bảng 15 Chức năng các tệp trong thư mục src 83
Bảng 16 Vai trò dữ liệu trong thư mục roms.....85
Bảng 17 Nội dung chính của cấu hình CMake 86
Bảng 18 Nhóm opcode điều khiển luồng.....92
Bảng 19 Một số opcode nhóm gán và tính toán 93
Bảng 20 Nhóm opcode xử lý bàn phím 94
Bảng 21 Nhóm opcode timer và bộ nhớ 94
Bảng 22 Tài nguyên SDL2 trong lớp Platform.....95
Bảng 23 Bảng ánh xạ bàn phím trong chương trình.....97
Bảng 24 Ý nghĩa các nhóm tham số dòng lệnh 98
Bảng 25 Quy trình build và chạy chương trình 100
Bảng 26 Kiểm thử các ROM.....101
Bảng 27 Tổng hợp kết quả kiểm thử các ROM 105

DANH SÁCH NHÓM THAM GIA LÀM BÁO CÁO

HỌC KÌ II, NĂM HỌC: 2025-2026

Tên đề tài: Thiết kế hệ thống giả lập CHIP-8 bằng C++/ SDL2

STT	HỌ VÀ TÊN	PHÂN CÔNG NHIỆM VỤ	MỨC ĐỘ HOÀN THÀNH
1	Nguyễn Trần Khánh Nguyên	Chương 2 (2.4 – 2.6), chương 3 (3.1 – 3.2)	100%
2	Nguyễn Quang Quân	Chương 1, chương 2 (2.1 – 2.3)	100%
3	Lê Văn Hậu	Chương 4 (4.1 – 4.4)	100%
4	Phạm Đắc Vinh	Chương 3 (3.3 – 3.5)	100%
5	Phạm Văn Nhiệm	Chương 4 (4.4 – 4.8), chương 5	100%

Nhận xét của giảng viên:

Hình thức:

.....

.....

Nội dung:.....

Tổng điểm:.....

Ngày 27 tháng 6 năm 2026

Giảng viên

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong quá trình học tập môn Lập trình hướng đối tượng và Kỹ thuật phần mềm, nhóm nhận thấy rằng việc chỉ học lý thuyết là chưa đủ để hiểu rõ cách xây dựng một phần mềm hoàn chỉnh. Một dự án thực tế cần phải kết hợp nhiều kiến thức như phân tích yêu cầu, thiết kế chương trình, lập trình hướng đối tượng, tổ chức mã nguồn, kiểm thử và xây dựng giao diện. Vì vậy, nhóm lựa chọn đề tài xây dựng trình giả lập CHIP-8 (CHIP-8 Emulator) nhằm vận dụng tổng hợp các kiến thức đã học vào một sản phẩm cụ thể.

CHIP-8 là một hệ thống máy ảo đơn giản nhưng vẫn bao gồm đầy đủ các thành phần cơ bản của một máy tính như bộ nhớ, thanh ghi, bộ đếm chương trình (Program Counter), ngăn xếp (Stack), bộ định thời (Timer), màn hình hiển thị và bàn phím. Nhờ cấu trúc không quá phức tạp, CHIP-8 là nền tảng phù hợp để nghiên cứu nguyên lý hoạt động của bộ xử lý và quá trình thực thi lệnh, đồng thời giúp người học dễ dàng tiếp cận với kỹ thuật xây dựng một trình giả lập.

Bên cạnh đó, đề tài còn giúp nhóm áp dụng các nguyên lý của lập trình hướng đối tượng trong quá trình phát triển phần mềm. Chương trình được thiết kế theo mô hình các lớp như Chip8 và Platform, trong đó mỗi lớp đảm nhận một nhiệm vụ riêng. Lớp Chip8 chịu trách nhiệm mô phỏng hệ thống CHIP-8 và xử lý các opcode, trong khi lớp Platform đảm nhiệm việc giao tiếp với thư viện SDL2 để hiển thị hình ảnh và xử lý bàn phím. Việc phân chia chức năng theo từng lớp giúp chương trình có cấu trúc rõ ràng, dễ bảo trì và thuận tiện cho việc mở rộng sau này.

Ngoài ra, nhóm lựa chọn thư viện SDL2 để xây dựng giao diện hiển thị vì đây là thư viện phổ biến trong phát triển ứng dụng đa phương tiện bằng C++. SDL2 hỗ trợ tạo cửa sổ, hiển thị đồ họa và xử lý sự kiện bàn phím, đáp ứng đầy đủ các yêu cầu của một trình giả lập CHIP-8.

Một lý do quan trọng khác là sau khi hoàn thành đề tài, chương trình không chỉ mô phỏng được kiến trúc của CHIP-8 mà còn có khả năng nạp và thực thi nhiều ROM trò chơi khác nhau như Tetris, Space Invaders, Blitz, Submarine và Dodge. Điều này giúp nhóm đánh giá được tính đúng đắn của chương trình thông qua quá trình kiểm thử thực tế, đồng thời tạo ra một sản phẩm có tính trực quan và giá trị ứng dụng cao.

Từ những lý do trên, nhóm quyết định lựa chọn đề tài "Xây dựng trình giả lập CHIP-8 bằng ngôn ngữ C++ và thư viện SDL2". Thông qua đề tài này, nhóm có cơ hội củng cố kiến thức về lập trình hướng đối tượng, quy trình phát triển phần mềm và nâng cao kỹ năng phân tích, thiết kế cũng như triển khai một dự án phần mềm hoàn chỉnh.

1.2. Mục tiêu đề tài

Đề tài được thực hiện với mục tiêu xây dựng một trình giả lập CHIP-8 bằng ngôn ngữ lập trình C++ kết hợp với thư viện SDL2. Chương trình có khả năng mô phỏng các thành phần cơ bản của hệ thống CHIP-8, thực hiện việc nạp ROM, đọc và giải mã các opcode, đồng thời thực thi chương trình theo đúng nguyên lý hoạt động của kiến trúc CHIP-8. Thông qua đó, người dùng có thể chạy được các trò chơi CHIP-8 trên máy tính hiện đại mà không cần sử dụng phần cứng gốc.

Một mục tiêu quan trọng của đề tài là nghiên cứu kiến trúc của hệ thống CHIP-8 để hiểu rõ cách một chương trình được thực thi từ khi nạp vào bộ nhớ cho đến khi hiển thị kết quả lên màn hình. Trong quá trình thực hiện, nhóm tập trung xây dựng và mô phỏng các thành phần chính của CHIP-8 như bộ nhớ 4096 byte, hệ thống thanh ghi, Program Counter, Stack, Timer, màn hình hiển thị 64×32 pixel và bàn phím 16 phím. Đồng thời, nhóm xây dựng cơ chế xử lý tập lệnh (Opcode) theo chu trình Fetch – Decode – Execute nhằm đảm bảo trình giả lập có thể thực thi chính xác các chương trình được viết cho CHIP-8.

Bên cạnh việc xây dựng một trình giả lập hoạt động ổn định, đề tài còn hướng đến việc vận dụng các kiến thức của môn Lập trình hướng đối tượng vào quá trình phát triển phần mềm. Chương trình được tổ chức theo mô hình hướng đối tượng với các lớp chính như Chip8 và Platform, trong đó mỗi lớp đảm nhận một chức năng riêng biệt. Lớp Chip8 chịu trách nhiệm mô phỏng toàn bộ hệ thống CHIP-8 và xử lý các opcode, còn lớp Platform đảm nhiệm việc tạo cửa sổ hiển thị, cập nhật màn hình và xử lý sự kiện bàn phím thông qua thư viện SDL2. Việc phân chia chương trình theo từng lớp giúp mã nguồn rõ ràng, dễ quản lý, thuận tiện cho việc kiểm thử, bảo trì và mở rộng trong tương lai.

Đề tài cũng tạo điều kiện để nhóm áp dụng các nội dung của môn Kỹ thuật phần mềm vào quá trình xây dựng sản phẩm. Toàn bộ chương trình được triển khai theo các giai đoạn từ nghiên cứu yêu cầu, phân tích hệ thống, thiết kế kiến trúc, cài đặt chương

trình cho đến kiểm thử và đánh giá kết quả. Quá trình này giúp nhóm rèn luyện tư duy phát triển phần mềm theo quy trình, thay vì chỉ tập trung vào việc lập trình.

Sau khi hoàn thành, chương trình được kiểm thử bằng nhiều ROM CHIP-8 phổ biến như Tetris, Space Invaders, Blitz, Submarine và Dodge nhằm đánh giá khả năng thực thi opcode, xử lý đồ họa và nhận dữ liệu từ bàn phím. Việc kiểm thử với nhiều trò chơi khác nhau giúp xác minh tính đúng đắn của trình giả lập, đồng thời đánh giá mức độ ổn định của chương trình trong nhiều tình huống hoạt động.

Thông qua việc thực hiện đề tài, nhóm không chỉ xây dựng được một trình giả lập CHIP-8 có khả năng hoạt động ổn định mà còn nâng cao kiến thức về kiến trúc hệ thống, lập trình hướng đối tượng, tổ chức chương trình theo module và quy trình phát triển phần mềm. Đây là những kiến thức và kỹ năng quan trọng, tạo nền tảng để tiếp tục nghiên cứu và phát triển các dự án phần mềm có quy mô lớn hơn trong tương lai.

1.3. Giới hạn của đề tài

Trong quá trình thực hiện đề tài, nhóm xác định phạm vi nghiên cứu và xây dựng chương trình dựa trên mục tiêu của học phần Lập trình hướng đối tượng và Kỹ thuật phần mềm. Vì vậy, đề tài tập trung vào việc xây dựng một trình giả lập CHIP-8 có khả năng mô phỏng các thành phần cơ bản của hệ thống và thực thi các chương trình được viết cho CHIP-8, thay vì hướng đến việc phát triển một trình giả lập hoàn chỉnh với đầy đủ các tính năng như các phần mềm thương mại. Việc xác định rõ giới hạn của đề tài giúp nhóm tập trung vào những nội dung cốt lõi, đảm bảo chương trình đáp ứng được yêu cầu của học phần và có thể hoàn thành trong thời gian quy định.

Về mặt chức năng, chương trình được xây dựng nhằm mô phỏng đầy đủ các thành phần cơ bản của hệ thống CHIP-8, bao gồm bộ nhớ, thanh ghi, Program Counter, Stack, Timer, màn hình hiển thị, bàn phím và cơ chế xử lý tập lệnh. Chương trình có khả năng đọc dữ liệu từ các file ROM, nạp ROM vào bộ nhớ, giải mã từng opcode và thực hiện chu trình Fetch – Decode – Execute để điều khiển hoạt động của toàn bộ hệ thống. Tuy nhiên, đề tài chỉ tập trung vào việc mô phỏng kiến trúc CHIP-8 truyền thống và chưa nghiên cứu các phiên bản mở rộng của CHIP-8 như Super CHIP (SCHIP) hoặc XO-CHIP. Đây là những hệ thống có thêm nhiều tập lệnh, hỗ trợ độ phân giải cao hơn và bổ sung các cơ chế xử lý mới, đòi hỏi quá trình nghiên cứu và cài đặt phức tạp hơn so với phạm vi của đề tài.

Trong quá trình xây dựng chương trình, nhóm tập trung vào việc đảm bảo tính chính xác của quá trình thực thi opcode, tổ chức mã nguồn theo hướng đối tượng và xây dựng một chương trình có cấu trúc rõ ràng. Các chức năng được ưu tiên triển khai bao gồm khởi tạo hệ thống, nạp ROM, xử lý bộ nhớ, xử lý thanh ghi, thực hiện các nhóm opcode, cập nhật màn hình và nhận dữ liệu từ bàn phím thông qua thư viện SDL2. Những chức năng này là nền tảng để một trình giả lập CHIP-8 có thể hoạt động đúng theo đặc tả của hệ thống.

Bên cạnh các chức năng đã được xây dựng, chương trình hiện chưa hỗ trợ một số tính năng thường xuất hiện trong các trình giả lập hiện đại. Chẳng hạn, chương trình chưa có giao diện quản lý ROM, chưa hỗ trợ lưu và khôi phục trạng thái trò chơi (Save/Load State), chưa tích hợp công cụ gỡ lỗi (Debugger), chưa hỗ trợ điều chỉnh tốc độ thực thi hoặc hiển thị thông tin chi tiết về trạng thái hoạt động của CPU trong quá trình chạy. Đây đều là những chức năng có thể được bổ sung trong các phiên bản phát triển tiếp theo nhưng không nằm trong mục tiêu chính của đề tài.

Về mặt công nghệ, chương trình được xây dựng bằng ngôn ngữ lập trình C++17, sử dụng thư viện SDL2 để xử lý việc hiển thị đồ họa và nhận tín hiệu từ bàn phím, đồng thời sử dụng CMake để quản lý quá trình biên dịch. Quá trình phát triển và kiểm thử được thực hiện trên hệ điều hành Windows. Mặc dù C++ và SDL2 đều hỗ trợ đa nền tảng, nhóm chưa tiến hành kiểm thử trên các hệ điều hành khác như Linux hoặc macOS, do đó khả năng tương thích trên các nền tảng này chưa được đánh giá một cách đầy đủ.

Phạm vi kiểm thử của đề tài cũng được giới hạn ở một số ROM CHIP-8 phổ biến như Tetris, Space Invaders, Blitz, Submarine và Dodge. Đây là các ROM có mức độ phổ biến cao và sử dụng nhiều nhóm opcode khác nhau, giúp nhóm đánh giá khả năng xử lý của trình giả lập trong nhiều tình huống. Tuy nhiên, số lượng ROM được kiểm thử vẫn còn hạn chế so với toàn bộ các ROM CHIP-8 hiện có. Vì vậy, mặc dù chương trình hoạt động ổn định với các ROM đã thử nghiệm, nhóm chưa thể khẳng định trình giả lập có thể tương thích hoàn toàn với tất cả các chương trình được phát triển cho CHIP-8.

Ngoài ra, đề tài chủ yếu tập trung vào việc xây dựng và mô phỏng hoạt động của hệ thống CHIP-8 dưới góc độ phát triển phần mềm. Những nội dung chuyên sâu liên quan đến tối ưu hiệu năng, kiến trúc phần cứng của các hệ thống giả lập hiện đại hoặc

các kỹ thuật tối ưu hóa trình biên dịch không nằm trong phạm vi nghiên cứu của đề tài. Thay vào đó, nhóm ưu tiên xây dựng một chương trình có cấu trúc rõ ràng, dễ hiểu, dễ bảo trì và thể hiện được việc vận dụng các nguyên lý của lập trình hướng đối tượng cũng như quy trình phát triển phần mềm.

Mặc dù còn tồn tại một số giới hạn nhất định, đề tài đã hoàn thành mục tiêu xây dựng một trình giả lập CHIP-8 có khả năng hoạt động ổn định, thực hiện đúng chu trình xử lý lệnh và chạy thành công nhiều ROM phổ biến. Những giới hạn được trình bày ở trên không làm ảnh hưởng đến mục tiêu chính của đề tài mà ngược lại còn giúp xác định rõ phạm vi nghiên cứu, tạo nền tảng để chương trình có thể tiếp tục được mở rộng trong tương lai. Trong các phiên bản tiếp theo, chương trình có thể được bổ sung thêm các tập lệnh mở rộng, tăng khả năng tương thích với nhiều ROM hơn, hoàn thiện giao diện người dùng và tích hợp các công cụ hỗ trợ kiểm thử, gỡ lỗi nhằm nâng cao chất lượng và khả năng ứng dụng của trình giả lập.

1.4. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu chính của đề tài là hệ thống máy ảo CHIP-8 và quá trình xây dựng một trình giả lập CHIP-8 bằng ngôn ngữ lập trình C++. Trong đó, nhóm tập trung tìm hiểu cách một hệ thống CHIP-8 hoạt động, cách chương trình được nạp vào bộ nhớ, cách các opcode được đọc và thực thi, cũng như cách dữ liệu được hiển thị lên màn hình và tương tác với bàn phím. Đây là những nội dung cốt lõi giúp nhóm hiểu được nguyên lý vận hành của một trình giả lập ở mức cơ bản.

Về mặt kiến trúc, đề tài nghiên cứu các thành phần quan trọng của CHIP-8 như bộ nhớ 4096 byte, hệ thống 16 thanh ghi đa dụng, thanh ghi địa chỉ, Program Counter, Stack, bộ định thời, màn hình hiển thị 64×32 pixel và bàn phím 16 phím. Các thành phần này tuy đơn giản hơn nhiều so với một máy tính hiện đại nhưng vẫn thể hiện đầy đủ tư duy tổ chức của một hệ thống xử lý chương trình. Việc nghiên cứu các thành phần trên giúp nhóm có cơ sở để mô phỏng lại chúng bằng phần mềm thông qua các biến, mảng và phương thức trong chương trình.

Bên cạnh kiến trúc CHIP-8, đề tài còn nghiên cứu tập lệnh opcode của hệ thống. Mỗi opcode đại diện cho một thao tác cụ thể như xóa màn hình, nhảy đến địa chỉ, gán dữ liệu cho thanh ghi, cộng giá trị, gọi chương trình con, xử lý bàn phím hoặc vẽ sprite lên màn hình. Việc giải mã và thực thi chính xác các opcode là yếu tố quan trọng quyết định khả năng chạy ROM của trình giả lập. Vì vậy, nhóm tập trung cài đặt các nhóm opcode cần thiết để chương trình có thể thực thi ổn định nhiều ROM CHIP-8 phổ biến.

Về mặt lập trình, đối tượng nghiên cứu của đề tài còn bao gồm phương pháp tổ chức chương trình theo hướng đối tượng. Project được xây dựng với các lớp chính như Chip8 và Platform, trong đó lớp Chip8 đảm nhiệm phần lõi của trình giả lập, còn lớp Platform đảm nhiệm phần hiển thị và xử lý bàn phím thông qua SDL2. Cách tổ chức này giúp tách biệt rõ ràng giữa logic giả lập và phần giao tiếp với hệ thống bên ngoài. Đây cũng là nội dung phù hợp với yêu cầu của môn Lập trình hướng đối tượng, vì chương trình thể hiện được việc phân chia trách nhiệm giữa các lớp, đóng gói dữ liệu và xây dựng các phương thức xử lý tương ứng.

Về phạm vi nghiên cứu, đề tài tập trung xây dựng một trình giả lập CHIP-8 chạy trên môi trường máy tính cá nhân, sử dụng ngôn ngữ C++17, thư viện SDL2 và hệ thống build CMake. Chương trình có khả năng nạp các file ROM có phần mở rộng .ch8, đưa dữ liệu ROM vào bộ nhớ của CHIP-8, thực hiện chu trình đọc lệnh, giải mã lệnh, thực thi lệnh và cập nhật màn hình. Phần hiển thị được giới hạn ở màn hình chuẩn 64×32 pixel của CHIP-8, sau đó được phóng đại lên cửa sổ SDL2 để người dùng dễ quan sát trong quá trình chạy chương trình.

Trong phạm vi của đồ án, nhóm chỉ tập trung vào các chức năng cơ bản và cần thiết của một trình giả lập CHIP-8. Chương trình không hướng đến việc xây dựng một giao diện đồ họa phức tạp, không phát triển hệ thống quản lý ROM nâng cao và không hỗ trợ các tính năng mở rộng như lưu trạng thái trò chơi, gỡ lỗi opcode, tua nhanh tốc độ chạy hoặc mô phỏng âm thanh chi tiết. Những chức

năng này tuy có giá trị trong các emulator hoàn chỉnh nhưng vượt quá phạm vi thực hiện của đề tài trong học phần.

Ngoài ra, phạm vi kiểm thử của đề tài được thực hiện thông qua một số ROM CHIP-8 phổ biến và dễ thao tác như Tetris, Space Invaders, Blitz, Submarine và Dodge. Việc kiểm thử trên nhiều ROM khác nhau giúp nhóm đánh giá khả năng thực thi opcode, khả năng hiển thị đồ họa, xử lý bàn phím và độ ổn định của chương trình. Tuy nhiên, do sự khác nhau giữa các ROM và các biến thể CHIP-8, chương trình không đảm bảo tương thích tuyệt đối với tất cả các ROM hiện có, đặc biệt là những ROM yêu cầu các tập lệnh mở rộng ngoài chuẩn CHIP-8 cơ bản.

Nhìn chung, đối tượng và phạm vi nghiên cứu của đề tài được xác định rõ ràng trong giới hạn của một đồ án học phần. Nhóm tập trung vào việc hiểu và mô phỏng đúng nguyên lý hoạt động của CHIP-8, đồng thời vận dụng kiến thức lập trình hướng đối tượng và kỹ thuật phần mềm để xây dựng một chương trình có cấu trúc rõ ràng, dễ hiểu, dễ kiểm thử và có khả năng phát triển thêm trong tương lai.

1.5. Cấu trúc báo cáo

Để trình bày đầy đủ quá trình nghiên cứu, phân tích, thiết kế, xây dựng và đánh giá chương trình, báo cáo được tổ chức thành năm chương với nội dung được sắp xếp theo trình tự từ cơ sở lý thuyết đến quá trình triển khai thực tế. Việc bố cục báo cáo theo từng chương giúp người đọc dễ dàng theo dõi các bước thực hiện đề tài, đồng thời thể hiện rõ cách nhóm vận dụng kiến thức của môn Lập trình hướng đối tượng và Kỹ thuật phần mềm trong quá trình xây dựng trình giả lập CHIP-8.

Chương 1: Tổng quan đề tài trình bày những nội dung mở đầu của báo cáo, bao gồm lý do lựa chọn đề tài, mục tiêu nghiên cứu, giới hạn của đề tài, đối tượng và phạm vi nghiên cứu. Ngoài ra, chương này còn giới thiệu cấu trúc của toàn bộ báo cáo nhằm giúp người đọc có cái nhìn tổng quan về các nội dung sẽ được trình bày trong những chương tiếp theo.

Chương 2: Cơ sở lý thuyết tập trung giới thiệu các kiến thức nền tảng phục vụ cho việc xây dựng trình giả lập CHIP-8. Nội dung chương bao gồm khái niệm và nguyên

lý hoạt động của trình giả lập, giới thiệu hệ thống CHIP-8, kiến trúc của CHIP-8 như bộ nhớ, thanh ghi, Stack, Program Counter, Timer, màn hình và bàn phím. Bên cạnh đó, chương còn trình bày chu trình thực thi lệnh Fetch – Decode – Execute, các nhóm opcode cơ bản của CHIP-8, các nguyên lý lập trình hướng đối tượng trong C++ cũng như vai trò của thư viện SDL2 trong việc xây dựng giao diện hiển thị và xử lý sự kiện bàn phím. Đây là cơ sở lý thuyết quan trọng để nhóm triển khai việc thiết kế và xây dựng chương trình.

Chương 3: Phân tích và thiết kế hệ thống trình bày quá trình phân tích yêu cầu của chương trình, xác định các chức năng cần xây dựng và thiết kế kiến trúc tổng thể của hệ thống. Chương này mô tả cách chương trình được chia thành các thành phần chính như lớp Chip8, lớp Platform và chương trình điều khiển chính. Đồng thời, chương cũng trình bày thiết kế các thuộc tính, phương thức của từng lớp và mô tả luồng hoạt động của chương trình từ khi khởi động, tải ROM, thực hiện chu kỳ xử lý lệnh cho đến khi kết thúc chương trình. Nội dung của chương thể hiện rõ việc áp dụng phương pháp lập trình hướng đối tượng trong quá trình thiết kế phần mềm.

Chương 4: Cài đặt chương trình và kết quả kiểm thử tập trung trình bày quá trình triển khai chương trình trên môi trường phát triển thực tế. Nội dung chương giới thiệu cấu trúc thư mục dự án, quá trình cài đặt các lớp chính, xây dựng cơ chế nạp ROM, xử lý opcode, cập nhật màn hình, xử lý bàn phím và xây dựng vòng lặp chính của chương trình. Ngoài ra, chương còn trình bày quy trình biên dịch, hướng dẫn chạy chương trình và kết quả kiểm thử thông qua nhiều ROM CHIP-8 khác nhau như Tetris, Space Invaders, Blitz, Submarine và Dodge nhằm đánh giá tính chính xác và độ ổn định của trình giả lập.

Chương 5: Kết luận tổng hợp những kết quả đạt được trong quá trình thực hiện đề tài, đánh giá những ưu điểm cũng như các hạn chế còn tồn tại của chương trình. Trên cơ sở đó, nhóm đề xuất một số hướng phát triển như mở rộng tập lệnh, hỗ trợ các biến thể của CHIP-8, bổ sung các công cụ hỗ trợ gỡ lỗi hoặc phát triển giao diện thân thiện hơn nhằm nâng cao khả năng ứng dụng của chương trình trong tương lai.

Nhìn chung, toàn bộ báo cáo được xây dựng theo quy trình phát triển phần mềm từ nghiên cứu cơ sở lý thuyết, phân tích yêu cầu, thiết kế hệ thống, cài đặt chương trình cho đến kiểm thử và đánh giá kết quả. Cách bố cục này không chỉ giúp người đọc dễ

dàng theo dõi quá trình triển khai đề tài mà còn thể hiện rõ việc vận dụng các kiến thức của môn Lập trình hướng đối tượng và Kỹ thuật phần mềm trong quá trình xây dựng một trình giả lập CHIP-8 hoàn chỉnh.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về giả lập

Cùng với sự phát triển của công nghệ thông tin, phần cứng máy tính liên tục được cải tiến, kéo theo sự thay đổi của các hệ điều hành, kiến trúc bộ xử lý và môi trường thực thi chương trình. Điều này giúp nâng cao hiệu năng của các hệ thống hiện đại nhưng đồng thời cũng làm cho nhiều phần mềm và trò chơi được phát triển trên các nền tảng cũ không còn khả năng hoạt động trực tiếp trên các thiết bị mới. Để giải quyết vấn đề này, công nghệ giả lập (Emulation) đã được nghiên cứu và phát triển nhằm tái tạo môi trường hoạt động của các hệ thống cũ trên phần cứng hiện đại.

Hiện nay, trình giả lập được sử dụng trong nhiều lĩnh vực khác nhau như bảo tồn các hệ thống máy tính đời cũ, nghiên cứu kiến trúc máy tính, phát triển và kiểm thử phần mềm nhúng, cũng như hỗ trợ chạy các trò chơi điện tử cổ điển trên máy tính và các thiết bị hiện đại. Thay vì phải sử dụng đúng phần cứng ban đầu, người dùng chỉ cần một chương trình giả lập để tái hiện môi trường hoạt động của hệ thống cần mô phỏng. Điều này giúp giảm chi phí, tăng tính linh hoạt và tạo điều kiện thuận lợi cho việc nghiên cứu cũng như phát triển phần mềm.

Đối với lĩnh vực phát triển trò chơi, các trình giả lập đóng vai trò quan trọng trong việc bảo tồn và duy trì giá trị của những hệ thống đã ngừng sản xuất. Nhiều trò chơi được phát triển từ hàng chục năm trước vẫn có thể hoạt động trên các máy tính hiện đại thông qua các trình giả lập mà không cần thay đổi mã nguồn ban đầu. Chính vì vậy, công nghệ giả lập không chỉ có ý nghĩa về mặt kỹ thuật mà còn góp phần lưu giữ các giá trị lịch sử của ngành công nghiệp phần mềm.

Trong phạm vi của đề tài này, nhóm tập trung nghiên cứu và xây dựng một trình giả lập cho hệ thống CHIP-8. Đây là một trong những hệ thống máy ảo đơn giản, có kiến trúc gọn nhẹ nhưng vẫn bao gồm đầy đủ các thành phần cơ bản của một máy tính như bộ nhớ, thanh ghi, ngăn xếp, bộ đếm chương trình, bộ định thời, màn hình hiển thị và bàn phím. Nhờ kiến trúc đơn giản, CHIP-8 thường được lựa chọn làm nền tảng để nghiên cứu nguyên lý hoạt động của CPU, cách thực thi tập lệnh và phương pháp xây dựng trình giả lập.

Việc lựa chọn CHIP-8 làm đối tượng nghiên cứu không chỉ phù hợp với yêu cầu của học phần Lập trình hướng đối tượng và Kỹ thuật phần mềm mà còn tạo điều kiện

để nhóm vận dụng các kiến thức về thiết kế chương trình, tổ chức mã nguồn theo hướng đối tượng và xây dựng một sản phẩm phần mềm hoàn chỉnh. Trên cơ sở đó, các nội dung tiếp theo sẽ trình bày chi tiết về khái niệm giả lập, nguyên lý hoạt động của trình giả lập cũng như những đặc điểm của hệ thống CHIP-8 được sử dụng trong đề tài.

2.1.1. Khái niệm giả lập

Giả lập (Emulation) là kỹ thuật sử dụng phần mềm để mô phỏng hoạt động của một hệ thống hoặc thiết bị khác, cho phép các chương trình được thiết kế cho hệ thống đó có thể hoạt động trên một nền tảng phần cứng khác. Chương trình thực hiện nhiệm vụ này được gọi là trình giả lập (Emulator). Thay vì yêu cầu người dùng phải sở hữu đúng thiết bị gốc, trình giả lập sẽ tái tạo lại môi trường hoạt động của hệ thống bằng phần mềm, từ đó giúp các chương trình hoặc trò chơi cũ có thể tiếp tục được sử dụng trên các máy tính hiện đại.

Về bản chất, một trình giả lập phải mô phỏng đầy đủ những thành phần quan trọng của hệ thống cần giả lập. Đối với nhiều hệ thống máy tính, các thành phần này bao gồm bộ xử lý (CPU), bộ nhớ (Memory), các thanh ghi (Registers), thiết bị nhập xuất (Input/Output), màn hình hiển thị và bàn phím. Khi chương trình được nạp vào trình giả lập, Emulator sẽ thực hiện việc đọc từng lệnh của chương trình, giải mã ý nghĩa của lệnh và mô phỏng đúng thao tác mà phần cứng thật sẽ thực hiện. Nhờ đó, chương trình có thể hoạt động gần giống như khi chạy trên hệ thống gốc.

Hiện nay, công nghệ giả lập được ứng dụng trong nhiều lĩnh vực khác nhau. Trong lĩnh vực phát triển phần mềm, giả lập giúp lập trình viên kiểm thử chương trình trên nhiều nền tảng mà không cần sử dụng phần cứng thật. Trong lĩnh vực giáo dục, các trình giả lập được sử dụng để hỗ trợ giảng dạy về kiến trúc máy tính, hệ điều hành và nguyên lý hoạt động của bộ xử lý. Đối với ngành công nghiệp trò chơi điện tử, trình giả lập đóng vai trò quan trọng trong việc bảo tồn các hệ máy và trò chơi cổ điển, cho phép người dùng tiếp tục trải nghiệm các trò chơi đã được phát triển từ nhiều năm trước trên các thiết bị hiện đại.

Trong đề tài này, nhóm xây dựng một trình giả lập dành cho hệ thống CHIP-8. Trình giả lập có nhiệm vụ mô phỏng các thành phần cơ bản của CHIP-8 như bộ nhớ, thanh ghi, Program Counter, Stack, Timer, màn hình và bàn phím. Khi người dùng lựa chọn một ROM trò chơi, chương trình sẽ nạp dữ liệu của ROM vào bộ nhớ, tiến hành

đọc và giải mã các opcode, sau đó thực hiện các lệnh tương ứng để cập nhật trạng thái của hệ thống và hiển thị kết quả lên màn hình thông qua thư viện SDL2. Toàn bộ quá trình này được thực hiện hoàn toàn bằng phần mềm nhưng vẫn tuân theo nguyên lý hoạt động của hệ thống CHIP-8.

Có thể thấy, trình giả lập không chỉ đơn thuần là một chương trình dùng để chạy trò chơi mà còn là sự mô phỏng lại toàn bộ môi trường hoạt động của một hệ thống máy tính. Vì vậy, để xây dựng một Emulator hoạt động chính xác, người phát triển cần hiểu rõ kiến trúc của hệ thống cần giả lập cũng như nguyên lý thực thi chương trình của hệ thống đó. Đây cũng chính là nền tảng để nhóm tiếp tục nghiên cứu nguyên lý hoạt động của trình giả lập CHIP-8 trong các mục tiếp theo.

2.1.2. Nguyên lý hoạt động của trình giả lập

Mặc dù mỗi trình giả lập được xây dựng cho một hệ thống khác nhau, nhưng về cơ bản chúng đều hoạt động dựa trên nguyên lý mô phỏng lại môi trường thực thi của hệ thống gốc bằng phần mềm. Thay vì sử dụng phần cứng thật, trình giả lập sẽ xây dựng các thành phần tương ứng như bộ nhớ, bộ xử lý, thanh ghi, màn hình và các thiết bị nhập xuất ngay trong chương trình. Khi đó, các chương trình được phát triển cho hệ thống gốc có thể hoạt động trên máy tính hiện đại thông qua môi trường mà trình giả lập tạo ra.

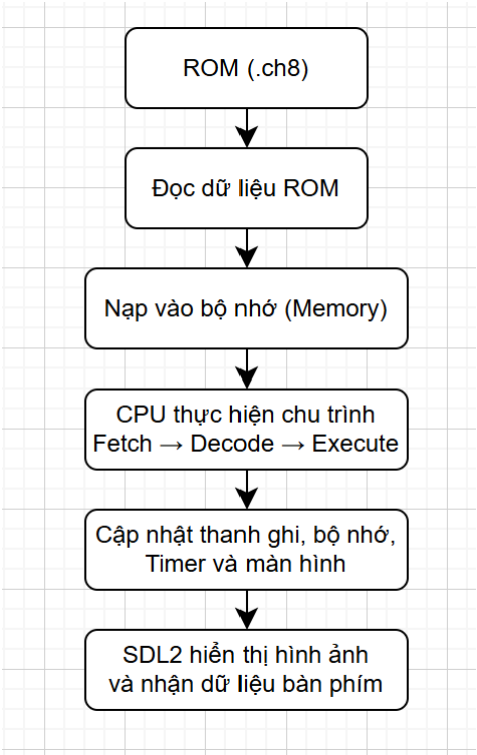
Quá trình hoạt động của một trình giả lập thường bắt đầu từ việc nạp chương trình vào bộ nhớ. Đối với đề tài này, chương trình cần thực thi được lưu trong các tệp ROM của CHIP-8. Khi người dùng lựa chọn một ROM, trình giả lập sẽ đọc toàn bộ dữ liệu trong tệp và nạp vào vùng nhớ theo đúng quy định của kiến trúc CHIP-8. Sau khi hoàn tất quá trình nạp ROM, bộ đếm chương trình (Program Counter) sẽ được đặt tại vị trí bắt đầu của chương trình để chuẩn bị thực hiện lệnh đầu tiên.

Sau khi chương trình được nạp vào bộ nhớ, trình giả lập sẽ thực hiện liên tục chu trình xử lý lệnh của CPU. Trong mỗi chu kỳ, Emulator đọc một opcode từ bộ nhớ, tiến hành giải mã để xác định loại lệnh cần thực hiện, sau đó mô phỏng đúng chức năng của lệnh đó. Tùy theo từng opcode, trình giả lập có thể thực hiện nhiều thao tác khác nhau như thay đổi giá trị của thanh ghi, truy cập bộ nhớ, gọi chương trình con, cập nhật bộ định thời, kiểm tra trạng thái bàn phím hoặc vẽ dữ liệu lên màn hình. Quá trình này

được lặp lại liên tục cho đến khi chương trình kết thúc hoặc người dùng đóng trình giả lập.

Trong quá trình thực thi, trạng thái của toàn bộ hệ thống luôn được cập nhật sau mỗi chu kỳ xử lý lệnh. Nếu opcode tác động đến bộ nhớ hoặc thanh ghi, các giá trị tương ứng sẽ được thay đổi. Nếu opcode điều khiển đồ họa, dữ liệu hiển thị trên màn hình sẽ được cập nhật. Tương tự, khi người dùng nhấn phím trên bàn phím máy tính, trình giả lập sẽ chuyển đổi tín hiệu đó thành trạng thái của bàn phím CHIP-8 để chương trình đang chạy có thể xử lý như trên hệ thống gốc. Nhờ cơ chế này, ROM có thể hoạt động chính xác mặc dù toàn bộ phần cứng đã được thay thế bằng phần mềm.

Đối với đề tài xây dựng trình giả lập CHIP-8, nguyên lý hoạt động trên được hiện thực thông qua các thành phần chính của chương trình. Lớp Chip8 chịu trách nhiệm mô phỏng toàn bộ hệ thống CHIP-8, bao gồm quản lý bộ nhớ, thanh ghi, Stack, Program Counter, Timer và xử lý các opcode. Trong khi đó, lớp Platform đảm nhiệm việc giao tiếp với hệ điều hành thông qua thư viện SDL2 để tạo cửa sổ hiển thị, cập nhật hình ảnh và xử lý sự kiện bàn phím. Chương trình chính có nhiệm vụ khởi tạo các đối tượng cần thiết, nạp ROM và điều khiển vòng lặp thực thi của trình giả lập.



Hình 1 Nguyên lý hoạt động của trình giả lập

Có thể thấy, nguyên lý hoạt động của trình giả lập không chỉ đơn thuần là đọc và chạy một chương trình, mà còn là quá trình mô phỏng toàn bộ môi trường hoạt động của hệ thống gốc. Độ chính xác của trình giả lập phụ thuộc vào khả năng mô phỏng đúng từng thành phần của hệ thống cũng như việc thực thi chính xác tập lệnh của bộ xử lý. Đây cũng chính là cơ sở để xây dựng trình giả lập CHIP-8 trong đề tài và là nền tảng cho các nội dung sẽ được trình bày ở các mục tiếp theo về kiến trúc CHIP-8 và tập lệnh của hệ thống.

2.2. Giới thiệu CHIP-8

CHIP-8 là một hệ thống máy ảo đơn giản được thiết kế nhằm hỗ trợ việc xây dựng và thực thi các chương trình, đặc biệt là các trò chơi điện tử cổ điển. Khác với một hệ điều hành hay một hệ thống máy tính vật lý hoàn chỉnh, CHIP-8 không đại diện cho một phần cứng cụ thể mà đóng vai trò như một môi trường thực thi với kiến trúc, bộ nhớ, tập lệnh, màn hình và bàn phím được quy định sẵn. Các chương trình được viết cho CHIP-8 sẽ không chạy trực tiếp trên các máy tính hiện đại nếu không có một trình giả lập phù hợp. Vì vậy, để có thể thực thi các chương trình này, cần xây dựng một Emulator có khả năng mô phỏng lại toàn bộ môi trường hoạt động của CHIP-8.

Về bản chất, CHIP-8 có thể được xem như một hệ thống trung gian giữa phần mềm trò chơi và phần cứng thực tế. Khi một chương trình CHIP-8 được nạp vào bộ nhớ, hệ thống sẽ đọc từng lệnh trong chương trình, giải mã ý nghĩa của lệnh và thực hiện thao tác tương ứng. Những thao tác này có thể là thay đổi dữ liệu trong thanh ghi, truy cập bộ nhớ, điều khiển luồng chương trình, xử lý bàn phím hoặc cập nhật màn hình hiển thị. Chính nhờ cơ chế này, CHIP-8 có thể chạy được nhiều chương trình khác nhau miễn là các chương trình đó tuân theo tập lệnh của hệ thống.

Mặc dù có kiến trúc đơn giản, CHIP-8 vẫn thể hiện đầy đủ các thành phần cơ bản thường gặp trong một hệ thống xử lý chương trình. Hệ thống này có bộ nhớ để lưu chương trình và dữ liệu, các thanh ghi để lưu trữ tạm thời giá trị trong quá trình tính toán, Program Counter để xác định vị trí lệnh đang được thực thi, Stack để hỗ trợ gọi và quay lại từ chương trình con, Timer để phục vụ các tác vụ liên quan đến thời gian, cùng với màn hình và bàn phím để giao tiếp với người dùng. Đây là những thành phần quan trọng giúp người học hiểu được nguyên lý hoạt động cơ bản của một hệ thống máy tính và trình giả lập.

Trong đề tài này, CHIP-8 được lựa chọn làm đối tượng nghiên cứu vì kiến trúc của nó không quá phức tạp nhưng vẫn đủ để thể hiện rõ tư duy thiết kế và cài đặt một trình giả lập. Nếu lựa chọn một hệ thống lớn hơn, số lượng tập lệnh và thành phần phần cứng cần mô phỏng sẽ rất nhiều, gây khó khăn trong phạm vi của một đồ án học phần. Ngược lại, CHIP-8 có tập lệnh tương đối gọn, màn hình đơn sắc với độ phân giải 64×32 pixel và bàn phím gồm 16 phím, nhờ đó phù hợp với mục tiêu nghiên cứu, thiết kế và cài đặt trong môn Lập trình hướng đối tượng và Kỹ thuật phần mềm.

Việc xây dựng trình giả lập CHIP-8 không chỉ giúp nhóm hiểu về cơ chế thực thi chương trình mà còn tạo điều kiện để áp dụng phương pháp lập trình hướng đối tượng vào một sản phẩm cụ thể. Trong project, hệ thống CHIP-8 được mô phỏng thông qua lớp Chip8. Lớp này đảm nhiệm việc quản lý bộ nhớ, thanh ghi, Stack, Program Counter, Timer, dữ liệu hiển thị và quá trình xử lý opcode. Phần giao tiếp với người dùng như hiển thị màn hình và xử lý bàn phím được tách riêng sang lớp Platform, sử dụng thư viện SDL2. Cách tổ chức này giúp chương trình có cấu trúc rõ ràng, tách biệt giữa phần lõi giả lập và phần giao tiếp với hệ thống bên ngoài.

Đối với người dùng, CHIP-8 Emulator cho phép chạy các tệp ROM có phần mở rộng .ch8. Những ROM này chứa chương trình hoặc trò chơi được viết theo tập lệnh của CHIP-8. Khi Emulator hoạt động, ROM sẽ được nạp vào bộ nhớ, sau đó từng opcode sẽ được đọc, giải mã và thực thi. Kết quả của quá trình thực thi được phản ánh qua sự thay đổi của màn hình, trạng thái bàn phím, thanh ghi và bộ nhớ. Đây chính là nguyên lý giúp các trò chơi CHIP-8 như Tetris, Space Invaders, Blitz, Submarine và Dodge có thể chạy được trên máy tính hiện đại thông qua chương trình giả lập do nhóm xây dựng.

Có thể thấy, CHIP-8 là một nền tảng phù hợp để nghiên cứu nguyên lý hoạt động của trình giả lập ở mức cơ bản. Hệ thống này không quá lớn để gây khó khăn trong quá trình cài đặt, nhưng vẫn đủ thành phần để thể hiện các vấn đề quan trọng như quản lý bộ nhớ, xử lý tập lệnh, thiết kế lớp, tổ chức chương trình theo module và kiểm thử phần mềm. Vì vậy, việc tìm hiểu CHIP-8 là cơ sở quan trọng trước khi đi vào phân tích kiến trúc, tập lệnh và quá trình cài đặt trình giả lập trong các phần tiếp theo của báo cáo.

2.2.1. Đặc điểm của CHIP-8

CHIP-8 là một hệ thống máy ảo có kiến trúc đơn giản, được thiết kế với mục đích hỗ trợ việc lập trình và thực thi các chương trình nhỏ, đặc biệt là các trò chơi điện tử cổ điển. So với các hệ thống máy tính hiện đại, CHIP-8 có quy mô nhỏ hơn rất nhiều, cả về dung lượng bộ nhớ, khả năng hiển thị, số lượng phím nhập liệu và tập lệnh xử lý. Tuy nhiên, chính sự đơn giản này lại giúp CHIP-8 trở thành một nền tảng phù hợp để nghiên cứu nguyên lý hoạt động của trình giả lập.

Một trong những đặc điểm quan trọng của CHIP-8 là hệ thống này có bộ nhớ dung lượng 4096 byte. Đây là vùng nhớ dùng để lưu trữ chương trình, dữ liệu, fontset và các thông tin cần thiết trong quá trình thực thi. Các chương trình CHIP-8 thường được nạp vào bộ nhớ bắt đầu từ địa chỉ 0x200, trong khi những vùng nhớ thấp hơn thường được dành cho interpreter hoặc dữ liệu hệ thống. Trong project của nhóm, đặc điểm này được thể hiện thông qua việc sử dụng mảng bộ nhớ để mô phỏng toàn bộ không gian nhớ của CHIP-8 và nạp ROM vào đúng vị trí bắt đầu theo quy ước của hệ thống.

CHIP-8 cũng sử dụng hệ thống thanh ghi đơn giản gồm 16 thanh ghi đa dụng, thường được ký hiệu từ V0 đến VF. Các thanh ghi này dùng để lưu trữ dữ liệu tạm thời trong quá trình thực thi opcode. Trong đó, thanh ghi VF có vai trò đặc biệt vì thường được sử dụng làm cờ trạng thái, chẳng hạn như cờ nhớ trong phép toán hoặc cờ va chạm khi vẽ sprite lên màn hình. Cách tổ chức thanh ghi này tuy đơn giản nhưng vẫn thể hiện được nguyên lý xử lý dữ liệu trong một hệ thống thực thi lệnh.

Một đặc điểm khác của CHIP-8 là việc điều khiển chương trình được thực hiện thông qua bộ đếm chương trình, hay còn gọi là Program Counter. Program Counter lưu địa chỉ của opcode tiếp theo cần được đọc từ bộ nhớ. Do mỗi opcode của CHIP-8 có độ dài 2 byte, nên sau khi một opcode được đọc, Program Counter thường được tăng thêm 2 để trỏ đến lệnh tiếp theo. Tuy nhiên, trong một số trường hợp như lệnh nhảy, lệnh gọi chương trình con hoặc lệnh kiểm tra điều kiện, giá trị của Program Counter có thể thay đổi theo logic của opcode. Đây là cơ chế giúp chương trình có thể thực hiện các cấu trúc điều khiển như rẽ nhánh, lặp hoặc gọi hàm.

CHIP-8 còn có Stack dùng để hỗ trợ việc gọi và quay lại từ chương trình con. Khi chương trình thực hiện lệnh gọi chương trình con, địa chỉ hiện tại sẽ được lưu vào

Stack để sau khi chương trình con thực hiện xong, hệ thống có thể quay lại đúng vị trí trước đó. Stack trong CHIP-8 có kích thước nhỏ nhưng đủ để phục vụ các chương trình đơn giản. Trong quá trình xây dựng trình giả lập, việc mô phỏng Stack là cần thiết để các opcode liên quan đến gọi hàm và trả về hoạt động chính xác.

Về hiển thị, CHIP-8 sử dụng màn hình đơn sắc có độ phân giải 64×32 pixel. Mỗi điểm ảnh chỉ có hai trạng thái là bật hoặc tắt. Hình ảnh trong CHIP-8 thường được tạo ra bằng cách vẽ các sprite lên màn hình thông qua các opcode đồ họa. Do độ phân giải nhỏ, khi chạy trên máy tính hiện đại, màn hình CHIP-8 thường được phóng đại theo một tỷ lệ nhất định để người dùng dễ quan sát hơn. Trong project của nhóm, dữ liệu màn hình được lưu trong bộ đệm hiển thị, sau đó được lớp Platform sử dụng SDL2 để cập nhật lên cửa sổ chương trình.

Bàn phím của CHIP-8 gồm 16 phím, được biểu diễn bằng các giá trị hệ thập lục phân từ 0 đến F. Cách bố trí này khác với bàn phím máy tính hiện đại, vì vậy khi xây dựng trình giả lập cần có cơ chế ánh xạ các phím trên máy tính sang bàn phím CHIP-8. Trong project, phần xử lý bàn phím được cài đặt thông qua SDL2, trong đó các sự kiện nhấn và thả phím được chuyển thành trạng thái tương ứng của các phím CHIP-8. Điều này cho phép các ROM trò chơi nhận input từ người dùng như khi chạy trên hệ thống gốc.

CHIP-8 sử dụng hai bộ định thời là Delay Timer và Sound Timer. Hai bộ định thời này có vai trò hỗ trợ các thao tác liên quan đến thời gian trong chương trình. Delay Timer thường được dùng để tạo độ trễ hoặc kiểm soát nhịp xử lý, trong khi Sound Timer liên quan đến việc phát âm thanh. Trong một trình giả lập CHIP-8, các timer cần được cập nhật liên tục trong quá trình thực thi để đảm bảo chương trình hoạt động đúng về mặt thời gian. Mặc dù project của nhóm chủ yếu tập trung vào hiển thị và xử lý gameplay, việc mô phỏng timer vẫn là một phần cần thiết của hệ thống.

Tập lệnh của CHIP-8 cũng là một đặc điểm quan trọng. So với các kiến trúc bộ xử lý hiện đại, số lượng opcode của CHIP-8 tương đối ít và dễ phân loại. Các opcode có thể được chia thành nhiều nhóm như nhóm điều khiển chương trình, nhóm xử lý dữ liệu, nhóm đồ họa, nhóm bàn phím, nhóm timer và nhóm truy cập bộ nhớ. Nhờ tập lệnh gọn, việc xây dựng trình giả lập CHIP-8 phù hợp với phạm vi của một đồ án học phần,

đồng thời vẫn giúp sinh viên hiểu được nguyên lý đọc lệnh, giải mã lệnh và thực thi lệnh của một hệ thống xử lý.

Từ các đặc điểm trên có thể thấy, CHIP-8 tuy đơn giản nhưng vẫn bao gồm đầy đủ các thành phần quan trọng của một hệ thống thực thi chương trình. Điều này giúp đề tài xây dựng trình giả lập CHIP-8 vừa có tính khả thi trong phạm vi môn học, vừa đủ nội dung để vận dụng các kiến thức về lập trình hướng đối tượng, thiết kế hệ thống, tổ chức mã nguồn và kiểm thử phần mềm. Đây cũng là lý do CHIP-8 thường được lựa chọn làm đề tài phù hợp cho những người bắt đầu tìm hiểu về Emulator và kiến trúc hệ thống.

2.2.2. ROM trong CHIP-8

Trong hệ thống CHIP-8, ROM là tệp chứa chương trình hoặc trò chơi được viết theo tập lệnh của CHIP-8. Các tệp ROM này thường có phần mở rộng .ch8 và bên trong chứa dữ liệu nhị phân đại diện cho các opcode mà hệ thống sẽ thực thi. Khi người dùng muốn chạy một trò chơi CHIP-8, trình giả lập sẽ không tự tạo ra nội dung trò chơi mà sẽ đọc dữ liệu từ ROM, nạp dữ liệu đó vào bộ nhớ và thực hiện từng lệnh theo đúng thứ tự được quy định trong chương trình.

ROM có thể được xem là đầu vào chính của trình giả lập. Nếu không có ROM, Emulator chỉ là một môi trường mô phỏng các thành phần của hệ thống CHIP-8 nhưng chưa có chương trình cụ thể để thực thi. Khi một ROM được nạp vào, trình giả lập bắt đầu có dữ liệu để xử lý. Dữ liệu trong ROM không phải là mã nguồn ở dạng văn bản mà là các byte nhị phân đã được biên dịch hoặc viết sẵn theo định dạng của CHIP-8. Vì vậy, trình giả lập cần đọc chính xác từng byte trong file ROM để ghép thành các opcode và thực hiện đúng chức năng tương ứng.

Theo quy ước của CHIP-8, chương trình thường được nạp vào bộ nhớ bắt đầu từ địa chỉ 0x200. Vùng nhớ trước địa chỉ này thường được dành cho interpreter hoặc các dữ liệu hệ thống, trong đó có thể bao gồm fontset dùng để hiển thị các ký tự hệ thập lục phân. Vì vậy, khi xây dựng trình giả lập CHIP-8, việc nạp ROM vào đúng vị trí trong bộ nhớ là yêu cầu quan trọng. Nếu ROM được nạp sai địa chỉ, Program Counter sẽ đọc sai dữ liệu, dẫn đến việc giải mã opcode không chính xác và chương trình có thể không hoạt động.

Trong project của nhóm, quá trình nạp ROM được thực hiện thông qua lớp Chip8. Khi chương trình bắt đầu chạy, người dùng có thể truyền tên ROM cần thực thi thông qua tham số dòng lệnh. Sau đó, chương trình chính sẽ xác định đường dẫn ROM và gọi phương thức nạp ROM để đưa dữ liệu vào bộ nhớ của Emulator. Dữ liệu của ROM được đọc dưới dạng nhị phân, sau đó được sao chép vào vùng nhớ bắt đầu từ địa chỉ 0x200. Sau khi quá trình nạp ROM hoàn tất, trình giả lập có thể bắt đầu thực hiện chu trình đọc, giải mã và thực thi opcode.

Mỗi ROM CHIP-8 thường được xây dựng để điều khiển hoạt động của một trò chơi hoặc một chương trình cụ thể. Ví dụ, một ROM trò chơi có thể chứa các opcode dùng để vẽ nhân vật lên màn hình, kiểm tra phím nhấn của người chơi, xử lý va chạm, cập nhật điểm số hoặc thay đổi trạng thái trò chơi. Khi Emulator thực thi ROM, các opcode này sẽ lần lượt tác động đến thanh ghi, bộ nhớ, màn hình, timer và bàn phím của hệ thống CHIP-8. Nhờ đó, người dùng có thể tương tác với trò chơi thông qua bàn phím máy tính và quan sát kết quả trên cửa sổ hiển thị.

Một điểm cần lưu ý là các ROM CHIP-8 khác nhau có thể sử dụng các nhóm opcode khác nhau. Có những ROM chủ yếu sử dụng các lệnh vẽ sprite và kiểm tra bàn phím, trong khi một số ROM khác lại sử dụng nhiều lệnh nhảy, gọi chương trình con, thao tác bộ nhớ hoặc xử lý timer. Vì vậy, để trình giả lập hoạt động ổn định, các opcode cần được cài đặt đầy đủ và chính xác. Nếu một nhóm opcode chưa được hỗ trợ hoặc được xử lý sai, một số ROM có thể chạy không đúng, hiển thị sai hình ảnh hoặc không phản hồi đúng với thao tác của người dùng.

Trong quá trình kiểm thử đề tài, nhóm sử dụng nhiều ROM phổ biến như Tetris, Space Invaders, Blitz, Submarine và Dodge. Việc lựa chọn nhiều ROM khác nhau giúp kiểm tra khả năng hoạt động của trình giả lập trong nhiều tình huống, bao gồm xử lý đồ họa, nhận phím, cập nhật trạng thái trò chơi và điều khiển luồng chương trình. Đây là bước quan trọng để đánh giá xem Emulator có thực thi đúng tập lệnh CHIP-8 hay không.

Cần phân biệt rõ giữa ROM và Emulator. ROM là chương trình hoặc trò chơi được viết cho hệ thống CHIP-8, còn Emulator là phần mềm mô phỏng môi trường CHIP-8 để thực thi ROM đó. Nói cách khác, ROM đóng vai trò là dữ liệu đầu vào, còn Emulator là hệ thống xử lý dữ liệu đầu vào đó. Trong đề tài này, nhóm không xây dựng

một trò chơi CHIP-8 mới, mà xây dựng một trình giả lập có khả năng nạp và chạy các ROM CHIP-8 có sẵn.

Như vậy, ROM giữ vai trò rất quan trọng trong hoạt động của trình giả lập CHIP-8. Toàn bộ quá trình xử lý của Emulator đều bắt đầu từ việc nạp ROM vào bộ nhớ, sau đó đọc và thực thi các opcode có trong ROM. Việc hiểu rõ cấu trúc và vai trò của ROM giúp nhóm xây dựng đúng cơ chế nạp chương trình, tổ chức bộ nhớ hợp lý và kiểm thử chương trình một cách hiệu quả. Đây cũng là cơ sở để chuyển sang phần tiếp theo về kiến trúc CHIP-8, nơi các thành phần như bộ nhớ, thanh ghi, Stack, Program Counter, Timer, màn hình và bàn phím sẽ được trình bày chi tiết hơn.

2.3. Kiến trúc CHIP-8

Kiến trúc CHIP-8 là nền tảng quan trọng để xây dựng một trình giả lập hoạt động chính xác. Để Emulator có thể nạp và thực thi các chương trình CHIP-8, trước hết cần hiểu rõ hệ thống này được tổ chức như thế nào, bao gồm những thành phần nào và các thành phần đó phối hợp với nhau ra sao trong quá trình xử lý lệnh. Mặc dù CHIP-8 có cấu trúc đơn giản hơn nhiều so với các kiến trúc máy tính hiện đại, hệ thống này vẫn bao gồm đầy đủ các thành phần cơ bản của một môi trường thực thi chương trình như bộ nhớ, thanh ghi, ngăn xếp, bộ đếm chương trình, bộ định thời, màn hình và bàn phím.

Trong CHIP-8, toàn bộ chương trình và dữ liệu được lưu trữ trong bộ nhớ. Khi một ROM được nạp vào hệ thống, dữ liệu của ROM sẽ được đưa vào một vùng nhớ xác định, sau đó bộ xử lý sẽ lần lượt đọc các opcode từ vùng nhớ này để thực thi. Bộ nhớ đóng vai trò trung tâm trong quá trình hoạt động của hệ thống, vì mọi thao tác như đọc lệnh, lưu dữ liệu, truy xuất fontset hoặc thao tác với các vùng dữ liệu đều liên quan đến bộ nhớ. Việc mô phỏng chính xác bộ nhớ là điều kiện đầu tiên để trình giả lập có thể hoạt động đúng.

Bên cạnh bộ nhớ, hệ thống thanh ghi cũng là một thành phần quan trọng của CHIP-8. Các thanh ghi được sử dụng để lưu trữ tạm thời dữ liệu trong quá trình thực thi chương trình. Khi một opcode được xử lý, dữ liệu có thể được đưa vào thanh ghi, lấy ra từ thanh ghi hoặc thực hiện các phép toán giữa các thanh ghi. Ngoài các thanh ghi đa dụng, CHIP-8 còn sử dụng một số thanh ghi đặc biệt để lưu địa chỉ bộ nhớ, trạng thái xử lý hoặc hỗ trợ điều khiển chương trình. Vì vậy, việc hiểu đúng vai trò của từng loại thanh ghi là cần thiết khi xây dựng phần lõi của Emulator.

Stack trong CHIP-8 được sử dụng để hỗ trợ việc gọi và quay lại từ chương trình con. Khi chương trình thực hiện một lệnh gọi chương trình con, địa chỉ hiện tại sẽ được lưu lại để sau khi chương trình con kết thúc, hệ thống có thể quay về đúng vị trí trước đó. Cơ chế này giúp chương trình CHIP-8 có thể tổ chức các đoạn lệnh thành những phần nhỏ hơn, tránh việc phải viết lặp lại cùng một nhóm lệnh nhiều lần. Trong trình giả lập, Stack cần được mô phỏng đúng để các lệnh gọi và trả về hoạt động chính xác.

Bộ đếm chương trình, hay Program Counter, là thành phần dùng để xác định địa chỉ của opcode tiếp theo cần được thực thi. Trong quá trình hoạt động bình thường, Program Counter sẽ tăng dần sau mỗi lần đọc lệnh. Tuy nhiên, đối với các lệnh nhảy, lệnh gọi chương trình con hoặc lệnh rẽ nhánh có điều kiện, giá trị của Program Counter có thể thay đổi sang một địa chỉ khác trong bộ nhớ. Do đó, Program Counter có vai trò điều khiển luồng thực thi của toàn bộ chương trình.

CHIP-8 còn sử dụng hai bộ định thời là Delay Timer và Sound Timer. Hai bộ định thời này được dùng để hỗ trợ các thao tác liên quan đến thời gian trong chương trình. Delay Timer thường được sử dụng để tạo độ trễ hoặc kiểm soát một số hành vi trong trò chơi, trong khi Sound Timer liên quan đến tín hiệu âm thanh. Mặc dù hệ thống timer của CHIP-8 khá đơn giản, nhưng nếu không được cập nhật đúng cách, một số ROM có thể hoạt động sai về tốc độ hoặc trạng thái xử lý.

Ngoài các thành phần xử lý bên trong, CHIP-8 còn có hệ thống hiển thị và bàn phím để giao tiếp với người dùng. Màn hình CHIP-8 có độ phân giải 64×32 pixel và chỉ sử dụng hai trạng thái điểm ảnh là bật hoặc tắt. Các hình ảnh trong chương trình thường được tạo ra bằng cách vẽ sprite lên màn hình. Bàn phím của CHIP-8 gồm 16 phím, được biểu diễn bằng các giá trị từ 0 đến F. Khi xây dựng trình giả lập trên máy tính hiện đại, cần ánh xạ các phím của bàn phím máy tính sang bàn phím CHIP-8 để người dùng có thể điều khiển chương trình.

Trong đề tài này, kiến trúc CHIP-8 được mô phỏng chủ yếu trong lớp Chip8. Các thành phần như bộ nhớ, thanh ghi, Stack, Program Counter, Timer, màn hình và bàn phím được biểu diễn bằng các biến, mảng và cấu trúc dữ liệu trong chương trình. Lớp Chip8 đảm nhiệm việc quản lý trạng thái bên trong của hệ thống và xử lý các opcode, trong khi lớp Platform hỗ trợ hiển thị hình ảnh và nhận dữ liệu bàn phím thông qua thư

viện SDL2. Cách tổ chức này giúp tách biệt phần lõi giả lập với phần giao tiếp bên ngoài, phù hợp với nguyên tắc thiết kế chương trình theo hướng đối tượng.

Có thể thấy, kiến trúc CHIP-8 tuy đơn giản nhưng vẫn thể hiện đầy đủ các thành phần cần thiết của một hệ thống thực thi lệnh. Việc nghiên cứu kiến trúc này giúp nhóm hiểu rõ cách chương trình CHIP-8 được nạp vào bộ nhớ, cách opcode được xử lý, cách dữ liệu được lưu trữ trong thanh ghi, cách điều khiển luồng chương trình và cách kết quả được hiển thị ra màn hình. Đây là cơ sở quan trọng để xây dựng trình giả lập CHIP-8 và cũng là nền tảng cho phần phân tích, thiết kế và cài đặt chương trình ở các chương tiếp theo.

2.3.1. Bộ nhớ

Bộ nhớ là một trong những thành phần quan trọng nhất trong kiến trúc CHIP-8, có nhiệm vụ lưu trữ chương trình, dữ liệu, fontset và các thông tin cần thiết trong quá trình thực thi. Trong CHIP-8, bộ nhớ có dung lượng 4096 byte, tương ứng với không gian địa chỉ từ 0x000 đến 0xFFF. Mặc dù dung lượng này rất nhỏ so với bộ nhớ của các hệ thống máy tính hiện đại, nhưng đối với các chương trình và trò chơi CHIP-8 thì đây là không gian đủ để lưu trữ mã lệnh, dữ liệu hiển thị và các thông tin xử lý cơ bản.

Không gian bộ nhớ của CHIP-8 được tổ chức theo từng vùng với chức năng khác nhau. Vùng nhớ từ 0x000 đến 0x1FF thường được dành cho interpreter hoặc các dữ liệu hệ thống. Trong các trình giả lập hiện đại, vùng nhớ này không dùng để nạp chương trình chính mà thường được sử dụng để lưu fontset, tức là dữ liệu biểu diễn các ký tự hệ thập lục phân từ 0 đến F. Các ký tự này được sử dụng khi chương trình cần hiển thị số hoặc ký hiệu lên màn hình. Việc nạp fontset vào đúng vùng nhớ giúp các opcode liên quan đến hiển thị ký tự hoạt động chính xác.

Vùng nhớ từ địa chỉ 0x200 trở đi là nơi chương trình CHIP-8 thường được nạp vào để thực thi. Khi người dùng chọn một ROM, trình giả lập sẽ đọc nội dung của ROM dưới dạng dữ liệu nhị phân và sao chép vào bộ nhớ bắt đầu từ địa chỉ 0x200. Đây là quy ước quan trọng của CHIP-8, vì Program Counter cũng thường được khởi tạo tại địa chỉ này để bắt đầu đọc opcode đầu tiên của chương trình. Nếu ROM được nạp sai vị trí, trình giả lập có thể đọc nhầm dữ liệu, dẫn đến việc giải mã opcode sai và chương trình không thể hoạt động đúng.

Trong quá trình thực thi, bộ nhớ không chỉ dùng để lưu chương trình mà còn được sử dụng để lưu trữ dữ liệu tạm thời. Một số opcode có nhiệm vụ ghi dữ liệu từ thanh ghi vào bộ nhớ hoặc đọc dữ liệu từ bộ nhớ đưa vào thanh ghi. Ngoài ra, thanh ghi địa chỉ I thường được sử dụng để trỏ đến một vị trí cụ thể trong bộ nhớ, chẳng hạn như vị trí chứa sprite cần vẽ hoặc vùng dữ liệu cần thao tác. Nhờ đó, chương trình CHIP-8 có thể truy cập và xử lý dữ liệu một cách linh hoạt trong phạm vi bộ nhớ cho phép.

Bộ nhớ cũng có vai trò quan trọng trong quá trình xử lý đồ họa. Trong CHIP-8, hình ảnh thường được biểu diễn dưới dạng sprite, tức là các mẫu dữ liệu nhị phân thể hiện trạng thái bật hoặc tắt của từng điểm ảnh. Khi chương trình thực hiện lệnh vẽ, trình giả lập sẽ đọc dữ liệu sprite từ bộ nhớ tại địa chỉ được chỉ định bởi thanh ghi I, sau đó cập nhật các điểm ảnh tương ứng trên màn hình. Vì vậy, nếu dữ liệu trong bộ nhớ không được quản lý đúng, hình ảnh hiển thị có thể bị sai hoặc không xuất hiện.

Trong project của nhóm, bộ nhớ CHIP-8 được mô phỏng bằng một mảng có kích thước cố định. Mảng này đại diện cho toàn bộ không gian nhớ 4096 byte của hệ thống. Khi đối tượng Chip8 được khởi tạo, chương trình thiết lập trạng thái ban đầu cho bộ nhớ và nạp fontset vào vùng nhớ quy định. Khi một ROM được chọn, phương thức nạp ROM sẽ đọc dữ liệu từ file và đưa vào bộ nhớ bắt đầu từ địa chỉ 0x200. Sau đó, trong mỗi chu kỳ xử lý, trình giả lập sẽ lấy opcode từ bộ nhớ dựa trên vị trí hiện tại của Program Counter.

Việc mô phỏng bộ nhớ chính xác là điều kiện cần thiết để trình giả lập CHIP-8 hoạt động ổn định. Mọi thao tác của chương trình đều có liên quan trực tiếp hoặc gián tiếp đến bộ nhớ, từ việc đọc opcode, truy cập sprite, lưu dữ liệu thanh ghi cho đến xử lý các lệnh liên quan đến fontset. Nếu bộ nhớ được tổ chức sai hoặc dữ liệu được nạp sai vị trí, toàn bộ quá trình thực thi chương trình sẽ bị ảnh hưởng. Do đó, trong quá trình xây dựng Emulator, nhóm cần đảm bảo bộ nhớ được khởi tạo đúng kích thước, ROM được nạp đúng địa chỉ và các thao tác đọc ghi bộ nhớ được thực hiện chính xác.

Có thể thấy, mặc dù bộ nhớ của CHIP-8 có dung lượng nhỏ, nhưng đây là thành phần giữ vai trò trung tâm trong toàn bộ hoạt động của hệ thống. Bộ nhớ không chỉ chứa chương trình cần thực thi mà còn là nơi lưu trữ dữ liệu phục vụ cho quá trình xử lý opcode, hiển thị đồ họa và tương tác với các thành phần khác. Vì vậy, việc hiểu rõ tổ

chức bộ nhớ là cơ sở quan trọng để phân tích các thành phần tiếp theo của kiến trúc CHIP-8 như thanh ghi, Stack, Program Counter và tập lệnh của hệ thống.

2.3.2. Thanh ghi

Thanh ghi là một thành phần quan trọng trong kiến trúc CHIP-8, được sử dụng để lưu trữ tạm thời dữ liệu trong quá trình chương trình thực thi. Nếu bộ nhớ là nơi lưu trữ chương trình và dữ liệu trong phạm vi rộng hơn, thì thanh ghi là nơi lưu các giá trị đang được CPU xử lý trực tiếp. Trong mỗi chu kỳ thực thi opcode, dữ liệu có thể được lấy từ bộ nhớ đưa vào thanh ghi, được xử lý giữa các thanh ghi với nhau hoặc được ghi ngược trở lại bộ nhớ tùy theo chức năng của từng lệnh.

CHIP-8 có 16 thanh ghi đa dụng, thường được ký hiệu từ V0 đến VF. Mỗi thanh ghi có kích thước 8 bit, tương ứng với khả năng lưu trữ giá trị từ 0 đến 255. Các thanh ghi này được sử dụng trong hầu hết các thao tác xử lý dữ liệu của chương trình, chẳng hạn như gán giá trị, cộng trừ, so sánh, kiểm tra điều kiện, lưu tọa độ vẽ sprite hoặc lưu trạng thái phím. Khi một opcode được thực thi, nó thường sẽ chỉ định thanh ghi nào được sử dụng thông qua các ký hiệu trong mã lệnh, ví dụ như Vx hoặc Vy.

Trong hệ thống thanh ghi của CHIP-8, thanh ghi VF có vai trò đặc biệt hơn so với các thanh ghi còn lại. Mặc dù VF vẫn là một thanh ghi 8 bit, nó thường được sử dụng như một cờ trạng thái trong một số phép toán và thao tác đồ họa. Ví dụ, trong các phép cộng hoặc trừ, VF có thể được dùng để lưu thông tin về nhớ hoặc mượn. Trong lệnh vẽ sprite, VF thường được dùng để xác định có xảy ra va chạm điểm ảnh hay không. Điều này có nghĩa là khi một điểm ảnh đang bật bị tắt do phép vẽ XOR, thanh ghi VF sẽ được đặt để báo hiệu rằng đã có va chạm xảy ra.

Ngoài 16 thanh ghi đa dụng, CHIP-8 còn sử dụng một thanh ghi địa chỉ đặc biệt thường được ký hiệu là I. Thanh ghi I có nhiệm vụ lưu trữ địa chỉ trong bộ nhớ và thường được dùng khi chương trình cần truy cập dữ liệu tại một vùng nhớ cụ thể. Chẳng hạn, khi cần vẽ sprite lên màn hình, thanh ghi I sẽ trỏ đến vị trí trong bộ nhớ chứa dữ liệu sprite. Sau đó, trình giả lập sẽ đọc dữ liệu từ vị trí này để cập nhật màn hình. Thanh ghi I cũng được sử dụng trong các lệnh liên quan đến fontset, lưu trữ hoặc khôi phục giá trị của các thanh ghi trong bộ nhớ.

Các thanh ghi trong CHIP-8 có quan hệ chặt chẽ với quá trình giải mã và thực thi opcode. Một opcode không chỉ cho biết thao tác cần thực hiện mà còn chỉ ra thanh

ghi nào tham gia vào thao tác đó. Ví dụ, có những opcode dùng để gán một giá trị trực tiếp vào thanh ghi, có opcode dùng để cộng một giá trị vào thanh ghi, có opcode so sánh giá trị của thanh ghi với một hằng số, và cũng có opcode thực hiện phép toán giữa hai thanh ghi. Vì vậy, để trình giả lập hoạt động đúng, chương trình cần xác định chính xác vị trí của các thanh ghi được chỉ định trong opcode và cập nhật giá trị của chúng theo đúng quy tắc.

Trong project của nhóm, hệ thống thanh ghi của CHIP-8 được mô phỏng bằng các biến và mảng trong lớp Chip8. Các thanh ghi đa dụng được lưu trong một mảng có 16 phần tử, tương ứng với các thanh ghi từ V0 đến VF. Thanh ghi địa chỉ I được khai báo riêng để lưu địa chỉ bộ nhớ cần truy cập. Cách tổ chức này giúp việc truy cập và xử lý thanh ghi trở nên thuận tiện trong quá trình cài đặt các hàm xử lý opcode. Khi một opcode được giải mã, chương trình sẽ xác định chỉ số thanh ghi cần thao tác, sau đó thực hiện cập nhật giá trị tương ứng trong mảng thanh ghi.

Việc mô phỏng chính xác hệ thống thanh ghi có ảnh hưởng trực tiếp đến khả năng hoạt động của trình giả lập. Nếu giá trị của thanh ghi bị cập nhật sai, các lệnh tiếp theo có thể xử lý sai dữ liệu, làm cho chương trình hiển thị sai hình ảnh, điều khiển sai luồng thực thi hoặc không phản hồi đúng với thao tác của người dùng. Đặc biệt, các thanh ghi dùng cho tọa độ vẽ, điều kiện rẽ nhánh và cờ trạng thái cần được xử lý chính xác để ROM có thể hoạt động ổn định.

Có thể thấy, thanh ghi là nơi lưu trữ và xử lý dữ liệu trung gian trong quá trình thực thi chương trình CHIP-8. Mặc dù số lượng và kích thước thanh ghi của CHIP-8 khá nhỏ, nhưng chúng tham gia vào hầu hết các hoạt động quan trọng của hệ thống, từ tính toán, điều kiện, truy cập bộ nhớ cho đến xử lý đồ họa. Vì vậy, việc hiểu rõ vai trò của các thanh ghi là cơ sở quan trọng để phân tích tập lệnh CHIP-8 và xây dựng phần lõi xử lý opcode trong trình giả lập.

2.3.3. Stack

Stack, hay còn gọi là ngăn xếp, là một thành phần quan trọng trong kiến trúc CHIP-8, được sử dụng để lưu trữ tạm thời địa chỉ quay về khi chương trình thực hiện lệnh gọi chương trình con. Trong lập trình, chương trình con là một đoạn lệnh được tách riêng để thực hiện một nhiệm vụ cụ thể và có thể được gọi nhiều lần trong quá trình chương trình hoạt động. Khi một chương trình con được gọi, hệ thống cần ghi nhớ vị

trí hiện tại của chương trình để sau khi chương trình con thực hiện xong, quá trình xử lý có thể quay lại đúng vị trí trước đó. Stack chính là nơi đảm nhiệm vai trò lưu trữ địa chỉ quay về này.

CHIP-8 sử dụng Stack theo nguyên tắc vào sau ra trước, thường được gọi là LIFO. Theo nguyên tắc này, giá trị nào được đưa vào Stack sau cùng sẽ là giá trị được lấy ra đầu tiên. Cơ chế này phù hợp với quá trình gọi chương trình con, vì khi một chương trình con gọi tiếp một chương trình con khác, hệ thống cần quay về chương trình con gần nhất trước, sau đó mới quay về các vị trí gọi trước đó. Nhờ Stack, chương trình có thể tổ chức các đoạn xử lý theo dạng phân cấp mà vẫn đảm bảo luồng thực thi được điều khiển chính xác.

Trong CHIP-8, Stack chủ yếu được sử dụng bởi hai nhóm lệnh liên quan đến chương trình con. Khi gặp lệnh gọi chương trình con, địa chỉ hiện tại của Program Counter sẽ được lưu vào Stack, sau đó Program Counter được thay đổi để trỏ đến địa chỉ bắt đầu của chương trình con. Khi chương trình con kết thúc và gặp lệnh quay về, địa chỉ đã lưu trước đó sẽ được lấy ra khỏi Stack và gán lại cho Program Counter. Nhờ vậy, chương trình có thể tiếp tục thực thi từ vị trí ngay sau lệnh gọi chương trình con.

Stack có quan hệ chặt chẽ với Program Counter. Nếu Program Counter cho biết địa chỉ lệnh tiếp theo cần thực hiện, thì Stack giúp lưu lại những địa chỉ cần quay về khi chương trình thay đổi luồng thực thi để đi vào chương trình con. Khi một lệnh gọi chương trình con được thực hiện, luồng chương trình tạm thời chuyển sang một vùng lệnh khác trong bộ nhớ. Tuy nhiên, nhờ địa chỉ quay về được lưu trong Stack, chương trình không bị mất vị trí ban đầu và có thể tiếp tục hoạt động đúng sau khi chương trình con hoàn thành.

Trong trình giả lập CHIP-8, việc mô phỏng Stack cần được thực hiện chính xác để các ROM có sử dụng chương trình con hoạt động ổn định. Nếu địa chỉ quay về được lưu sai hoặc lấy ra sai, chương trình có thể nhảy đến nhầm vị trí trong bộ nhớ, dẫn đến việc thực thi sai opcode hoặc làm chương trình bị lỗi. Vì vậy, ngoài Stack, trình giả lập còn cần một biến hỗ trợ để quản lý vị trí hiện tại trong Stack, thường được gọi là Stack Pointer. Stack Pointer cho biết vị trí tiếp theo trong Stack sẽ được ghi hoặc vị trí cần lấy dữ liệu khi quay về từ chương trình con.

Trong project của nhóm, Stack được mô phỏng trong lớp Chip8 để phục vụ các opcode liên quan đến gọi và trả về từ chương trình con. Khi một opcode yêu cầu gọi chương trình con, chương trình lưu giá trị hiện tại của Program Counter vào Stack, sau đó cập nhật Program Counter đến địa chỉ cần thực thi. Khi opcode quay về được xử lý, giá trị địa chỉ đã lưu trong Stack sẽ được lấy ra để khôi phục luồng thực thi. Cách cài đặt này giúp trình giả lập mô phỏng đúng cơ chế điều khiển chương trình của CHIP-8.

Mặc dù Stack trong CHIP-8 có cấu trúc đơn giản, nhưng nó giữ vai trò quan trọng trong việc tổ chức và điều khiển luồng chương trình. Nhờ Stack, các chương trình CHIP-8 có thể chia nhỏ chức năng thành nhiều chương trình con, giúp mã lệnh dễ tổ chức hơn và tránh việc lặp lại các đoạn lệnh giống nhau. Đối với trình giả lập, Stack là thành phần bắt buộc phải có để xử lý chính xác các opcode liên quan đến gọi hàm và quay về.

Có thể thấy, Stack là một thành phần hỗ trợ quan trọng trong kiến trúc CHIP-8, đặc biệt đối với các chương trình có sử dụng chương trình con. Stack phối hợp với Program Counter để đảm bảo chương trình có thể chuyển sang vùng lệnh khác và quay lại đúng vị trí ban đầu sau khi xử lý xong. Việc hiểu rõ cơ chế hoạt động của Stack giúp làm rõ hơn cách CHIP-8 điều khiển luồng chương trình, đồng thời là cơ sở để phân tích các nhóm lệnh điều khiển trong phần tập lệnh CHIP-8.

2.3.4. Bộ đếm chương trình

Bộ đếm chương trình, hay còn gọi là Program Counter (PC), là một thành phần quan trọng trong kiến trúc CHIP-8, có nhiệm vụ lưu trữ địa chỉ của opcode tiếp theo sẽ được đọc và thực thi. Trong quá trình chương trình hoạt động, hệ thống không thực hiện các lệnh một cách ngẫu nhiên mà phải biết chính xác lệnh nào đang được xử lý và lệnh nào sẽ được thực hiện tiếp theo. Program Counter chính là thành phần đảm nhiệm vai trò điều khiển thứ tự thực thi lệnh trong bộ nhớ.

Trong CHIP-8, mỗi opcode có độ dài 2 byte. Vì vậy, trong quá trình thực thi tuần tự, sau khi một opcode được đọc từ bộ nhớ, Program Counter thường được tăng thêm 2 để trỏ đến opcode kế tiếp. Cơ chế này giúp chương trình có thể lần lượt thực hiện các lệnh theo đúng thứ tự đã được lưu trong ROM. Khi ROM được nạp vào bộ nhớ bắt đầu từ địa chỉ 0x200, Program Counter cũng thường được khởi tạo tại địa chỉ này để bắt đầu thực thi lệnh đầu tiên của chương trình.

Tuy nhiên, Program Counter không phải lúc nào cũng tăng tuần tự. Trong nhiều trường hợp, giá trị của Program Counter có thể thay đổi tùy theo loại opcode đang được thực thi. Đối với các lệnh nhảy, Program Counter sẽ được gán trực tiếp đến một địa chỉ mới trong bộ nhớ, làm cho chương trình chuyển sang thực thi lệnh tại vị trí khác. Đối với các lệnh gọi chương trình con, Program Counter cũng được thay đổi để chuyển đến đoạn chương trình con cần thực hiện, đồng thời địa chỉ quay về sẽ được lưu vào Stack. Khi chương trình con kết thúc, địa chỉ đã lưu trong Stack sẽ được khôi phục lại cho Program Counter để chương trình tiếp tục thực hiện từ vị trí ban đầu.

Ngoài các lệnh nhảy và gọi chương trình con, Program Counter còn liên quan đến các lệnh rẽ nhánh có điều kiện. Một số opcode của CHIP-8 có chức năng kiểm tra giá trị trong thanh ghi hoặc trạng thái phím bấm. Nếu điều kiện đúng, Program Counter sẽ bỏ qua opcode kế tiếp bằng cách tăng thêm một bước tương ứng. Nếu điều kiện sai, chương trình sẽ tiếp tục thực hiện lệnh kế tiếp như bình thường. Nhờ cơ chế này, CHIP-8 có thể thực hiện các cấu trúc điều kiện đơn giản trong chương trình.

Trong trình giả lập CHIP-8, việc quản lý Program Counter cần được thực hiện chính xác vì nó ảnh hưởng trực tiếp đến toàn bộ luồng thực thi của chương trình. Nếu Program Counter trỏ sai địa chỉ, Emulator có thể đọc nhầm dữ liệu trong bộ nhớ và giải mã thành opcode không đúng. Điều này có thể khiến chương trình chạy sai logic, hiển thị sai hình ảnh hoặc dừng hoạt động. Vì vậy, mỗi khi một opcode được xử lý, trình giả lập cần xác định rõ Program Counter sẽ được tăng tuần tự hay được thay đổi theo yêu cầu của opcode.

Trong project của nhóm, Program Counter được mô phỏng trong lớp Chip8. Khi đối tượng Chip8 được khởi tạo, Program Counter được thiết lập tại địa chỉ bắt đầu của chương trình CHIP-8. Trong mỗi chu kỳ xử lý, trình giả lập sẽ sử dụng Program Counter để lấy opcode từ bộ nhớ, sau đó cập nhật Program Counter trước hoặc sau khi thực thi lệnh tùy theo cách cài đặt. Các opcode liên quan đến nhảy, gọi chương trình con, quay về hoặc bỏ qua lệnh tiếp theo đều trực tiếp tác động đến giá trị của Program Counter.

Có thể thấy, Program Counter đóng vai trò như bộ điều hướng của chương trình trong kiến trúc CHIP-8. Nó xác định lệnh nào sẽ được thực hiện, điều khiển thứ tự thực thi và hỗ trợ các thao tác thay đổi luồng chương trình. Việc hiểu rõ vai trò của Program Counter giúp làm rõ cơ chế hoạt động của chu trình Fetch – Decode – Execute, đồng

thời là cơ sở để phân tích các nhóm opcode điều khiển chương trình trong phần tập lệnh CHIP-8.

2.3.5. Bộ định thời

Bộ định thời là một thành phần quan trọng trong kiến trúc CHIP-8, được sử dụng để hỗ trợ các thao tác liên quan đến thời gian trong quá trình chương trình hoạt động. Mặc dù CHIP-8 có kiến trúc đơn giản, hệ thống vẫn cần cơ chế định thời để điều khiển một số hành vi của chương trình, đặc biệt là trong các trò chơi. Nhờ có bộ định thời, chương trình có thể tạo độ trễ, kiểm soát nhịp xử lý hoặc thực hiện các hiệu ứng liên quan đến thời gian một cách ổn định hơn.

Trong CHIP-8 có hai bộ định thời chính là Delay Timer và Sound Timer. Cả hai bộ định thời này đều có giá trị 8 bit và thường được giảm dần theo thời gian khi giá trị của chúng lớn hơn 0. Khi một chương trình cần sử dụng timer, giá trị sẽ được gán vào bộ định thời thông qua opcode tương ứng. Sau đó, trong quá trình trình giả lập hoạt động, giá trị của timer sẽ được cập nhật liên tục cho đến khi giảm về 0.

Delay Timer thường được sử dụng để tạo độ trễ hoặc kiểm soát một số trạng thái trong chương trình. Ví dụ, một ROM có thể đặt giá trị cho Delay Timer, sau đó kiểm tra giá trị này để xác định khi nào cần tiếp tục thực hiện một hành động nào đó. Cơ chế này giúp chương trình không phụ thuộc hoàn toàn vào tốc độ xử lý của máy tính hiện đại. Nếu không có Delay Timer, chương trình có thể chạy quá nhanh hoặc không kiểm soát được thời gian giữa các thao tác, đặc biệt là đối với các trò chơi có chuyển động hoặc trạng thái thay đổi liên tục.

Sound Timer được sử dụng cho các thao tác liên quan đến âm thanh. Khi Sound Timer có giá trị lớn hơn 0, hệ thống CHIP-8 có thể phát ra âm thanh báo hiệu. Khi giá trị của Sound Timer giảm về 0, âm thanh sẽ dừng lại. Trong nhiều trình giả lập CHIP-8 cơ bản, phần xử lý âm thanh có thể được triển khai đơn giản hoặc chưa được chú trọng bằng phần hiển thị và xử lý opcode. Tuy nhiên, về mặt kiến trúc, Sound Timer vẫn là một thành phần cần được mô phỏng để đảm bảo hệ thống hoạt động đúng với đặc tả của CHIP-8.

Bộ định thời có mối quan hệ chặt chẽ với chu trình thực thi lệnh của trình giả lập. Trong mỗi vòng lặp hoạt động, trình giả lập không chỉ đọc, giải mã và thực thi opcode mà còn phải cập nhật trạng thái của các timer. Khi giá trị của Delay Timer hoặc

Sound Timer lớn hơn 0, chúng sẽ được giảm dần theo thời gian. Việc cập nhật timer cần được thực hiện đều đặn để chương trình hoạt động ổn định và tránh tình trạng tốc độ chạy bị phụ thuộc quá nhiều vào tốc độ xử lý của phần cứng máy tính.

Trong project của nhóm, bộ định thời được mô phỏng trong lớp Chip8 thông qua các biến lưu giá trị của Delay Timer và Sound Timer. Khi chương trình thực thi các opcode liên quan đến timer, các giá trị này sẽ được đọc hoặc cập nhật tương ứng. Trong quá trình xử lý chu kỳ lệnh, nếu giá trị của timer lớn hơn 0 thì chương trình sẽ giảm giá trị của timer xuống. Cách mô phỏng này giúp trình giả lập duy trì được trạng thái thời gian cơ bản của hệ thống CHIP-8.

Việc cài đặt bộ định thời tuy không phức tạp như xử lý opcode đồ họa hay quản lý bộ nhớ, nhưng vẫn có vai trò quan trọng đối với hoạt động của Emulator. Nếu timer không được cập nhật đúng, một số ROM có thể hoạt động sai về tốc độ, hiệu ứng hoặc trạng thái xử lý. Đặc biệt, các trò chơi thường sử dụng timer để kiểm soát nhịp chuyển động, thời gian chờ hoặc tín hiệu âm thanh. Vì vậy, trình giả lập cần đảm bảo Delay Timer và Sound Timer được mô phỏng đúng về mặt logic.

Có thể thấy, bộ định thời trong CHIP-8 là thành phần hỗ trợ quan trọng giúp chương trình kiểm soát các yếu tố liên quan đến thời gian. Delay Timer phục vụ cho các thao tác tạo độ trễ và kiểm soát nhịp xử lý, trong khi Sound Timer liên quan đến tín hiệu âm thanh. Mặc dù đơn giản, hai bộ định thời này góp phần làm cho các chương trình CHIP-8 hoạt động ổn định và đúng với nguyên lý thiết kế ban đầu của hệ thống.

2.3.6. Màn hình và bàn phím

Màn hình và bàn phím là hai thành phần giúp CHIP-8 giao tiếp trực tiếp với người dùng. Nếu bộ nhớ, thanh ghi, Stack, Program Counter và Timer là các thành phần xử lý bên trong hệ thống, thì màn hình và bàn phím đóng vai trò là phần nhập xuất, giúp chương trình hiển thị kết quả và nhận thao tác điều khiển. Đối với các chương trình trò chơi, hai thành phần này đặc biệt quan trọng vì chúng quyết định khả năng tương tác giữa người chơi và chương trình đang được thực thi.

Màn hình của CHIP-8 có độ phân giải 64×32 pixel. Đây là độ phân giải rất nhỏ so với các thiết bị hiển thị hiện đại, nhưng phù hợp với mục tiêu thiết kế ban đầu của CHIP-8 là thực thi các chương trình đơn giản. Màn hình CHIP-8 là màn hình đơn sắc, nghĩa là mỗi điểm ảnh chỉ có hai trạng thái: bật hoặc tắt. Khi một điểm ảnh được bật,

nó sẽ được hiển thị trên màn hình; khi điểm ảnh bị tắt, vị trí đó sẽ trở về trạng thái nền. Cách hiển thị này giúp việc xử lý đồ họa trong CHIP-8 trở nên đơn giản hơn so với các hệ thống có màu sắc và độ phân giải cao.

Trong CHIP-8, hình ảnh thường được tạo ra bằng cách vẽ các sprite lên màn hình. Sprite là một tập hợp dữ liệu nhị phân biểu diễn hình dạng của đối tượng cần hiển thị. Mỗi bit trong dữ liệu sprite tương ứng với trạng thái của một điểm ảnh. Khi chương trình thực hiện lệnh vẽ, trình giả lập sẽ đọc dữ liệu sprite từ bộ nhớ, sau đó cập nhật các điểm ảnh tương ứng trên màn hình. Việc vẽ sprite trong CHIP-8 thường sử dụng phép XOR, nghĩa là nếu một điểm ảnh đang tắt thì sẽ được bật, còn nếu đang bật thì sẽ bị tắt. Cơ chế này cũng cho phép hệ thống phát hiện va chạm khi vẽ hình ảnh.

Dữ liệu hiển thị của CHIP-8 thường được lưu trong một vùng đệm màn hình, còn gọi là video buffer. Vùng đệm này lưu trạng thái của toàn bộ các điểm ảnh trên màn hình 64×32. Khi một opcode đồ họa được thực thi, dữ liệu trong video buffer sẽ được cập nhật trước. Sau đó, trình giả lập mới đưa dữ liệu này ra cửa sổ hiển thị để người dùng quan sát. Cách tổ chức này giúp tách biệt giữa phần xử lý logic của CHIP-8 và phần hiển thị trên hệ điều hành hiện đại.

Do màn hình CHIP-8 có kích thước rất nhỏ, khi xây dựng trình giả lập trên máy tính hiện đại cần phóng đại hình ảnh lên nhiều lần để người dùng dễ nhìn. Trong project của nhóm, dữ liệu màn hình 64×32 pixel được xử lý trong phần lõi CHIP-8, sau đó được lớp Platform sử dụng thư viện SDL2 để hiển thị lên cửa sổ chương trình. Việc sử dụng SDL2 giúp chương trình tạo cửa sổ, cập nhật texture và render hình ảnh lên màn hình một cách thuận tiện. Nhờ đó, mặc dù dữ liệu gốc của CHIP-8 chỉ có độ phân giải thấp, người dùng vẫn có thể quan sát trò chơi rõ ràng trên màn hình máy tính.

Bên cạnh màn hình, bàn phím cũng là thành phần quan trọng trong hệ thống CHIP-8. CHIP-8 sử dụng bàn phím gồm 16 phím, thường được biểu diễn bằng các giá trị hệ thập lục phân từ 0 đến F. Bố cục bàn phím CHIP-8 không giống với bàn phím máy tính hiện đại, vì vậy khi xây dựng trình giả lập cần có cơ chế ánh xạ phím. Cơ chế này cho phép một phím trên bàn phím máy tính đại diện cho một phím tương ứng trong hệ thống CHIP-8.

Trong quá trình hoạt động, một số opcode của CHIP-8 có nhiệm vụ kiểm tra trạng thái bàn phím. Chương trình có thể kiểm tra xem một phím có đang được nhấn

hay không, bỏ qua lệnh kế tiếp nếu điều kiện phím đúng, hoặc tạm dừng để chờ người dùng nhấn phím. Vì vậy, trạng thái bàn phím cần được cập nhật liên tục trong vòng lặp chính của trình giả lập. Nếu việc xử lý bàn phím không chính xác, người dùng có thể không điều khiển được trò chơi hoặc chương trình có thể phản hồi sai với thao tác nhập liệu.

Trong project của nhóm, việc xử lý bàn phím được thực hiện thông qua lớp Platform. Lớp này nhận các sự kiện từ SDL2, sau đó chuyển đổi các phím trên bàn phím máy tính thành trạng thái tương ứng trong mảng bàn phím của CHIP-8. Khi người dùng nhấn hoặc thả phím, trạng thái của phím đó sẽ được cập nhật để lớp Chip8 có thể sử dụng trong quá trình xử lý opcode. Cách làm này giúp tách biệt phần nhận sự kiện từ hệ điều hành với phần xử lý logic của Emulator.

Màn hình và bàn phím có mối quan hệ chặt chẽ với quá trình thực thi ROM. Trong một trò chơi CHIP-8, các opcode đồ họa sẽ cập nhật màn hình để hiển thị nhân vật, vật thể hoặc trạng thái trò chơi. Đồng thời, các opcode bàn phím sẽ kiểm tra thao tác của người dùng để điều khiển nhân vật hoặc thực hiện hành động trong trò chơi. Nhờ sự phối hợp giữa hai thành phần này, người dùng có thể tương tác với ROM giống như đang sử dụng hệ thống CHIP-8 gốc.

Có thể thấy, màn hình và bàn phím là hai thành phần không thể thiếu trong kiến trúc CHIP-8. Màn hình giúp hiển thị kết quả xử lý của chương trình, trong khi bàn phím cho phép người dùng nhập dữ liệu điều khiển. Đối với trình giả lập CHIP-8, việc mô phỏng chính xác hai thành phần này có ý nghĩa quan trọng trong việc đảm bảo các ROM trò chơi hoạt động ổn định, hiển thị đúng hình ảnh và phản hồi chính xác với thao tác của người dùng.

2.4. Tập lệnh CHIP-8

Tập lệnh là một thành phần quan trọng trong kiến trúc CHIP-8, quy định các thao tác mà hệ thống có thể thực hiện trong quá trình chạy chương trình. Mỗi chương trình hoặc trò chơi CHIP-8 được tạo thành từ nhiều lệnh khác nhau, được lưu trong ROM dưới dạng các opcode. Khi ROM được nạp vào bộ nhớ, trình giả lập sẽ lần lượt đọc từng opcode, giải mã ý nghĩa của opcode đó và thực hiện thao tác tương ứng.

Trong CHIP-8, mỗi opcode thường có độ dài 2 byte và được biểu diễn dưới dạng hệ thập lục phân. Mỗi opcode mang một ý nghĩa cụ thể, chẳng hạn như xóa màn hình,

gán giá trị vào thanh ghi, thực hiện phép toán, nhảy đến một địa chỉ trong bộ nhớ, gọi chương trình con, vẽ sprite lên màn hình hoặc kiểm tra trạng thái bàn phím. Nhờ tập lệnh này, các chương trình CHIP-8 có thể điều khiển toàn bộ hoạt động của hệ thống, từ xử lý dữ liệu bên trong cho đến hiển thị hình ảnh và nhận tương tác từ người dùng.

Đối với trình giả lập CHIP-8, việc cài đặt tập lệnh là phần cốt lõi quyết định khả năng hoạt động của chương trình. Nếu các opcode được giải mã và xử lý đúng, ROM sẽ có thể chạy ổn định và cho kết quả chính xác. Ngược lại, nếu một opcode bị xử lý sai hoặc chưa được hỗ trợ, chương trình có thể chạy sai logic, hiển thị sai hình ảnh hoặc không phản hồi đúng với thao tác của người dùng. Vì vậy, quá trình xây dựng Emulator cần đặc biệt chú ý đến việc phân loại và cài đặt chính xác các nhóm lệnh của CHIP-8.

Trong đề tài này, tập lệnh CHIP-8 được tìm hiểu và phân chia thành các nhóm chính như nhóm lệnh điều khiển chương trình, nhóm lệnh xử lý dữ liệu, nhóm lệnh đồ họa và nhóm lệnh bàn phím. Bên cạnh đó, chu trình Fetch – Decode – Execute cũng được trình bày nhằm làm rõ cách trình giả lập đọc, giải mã và thực thi opcode trong từng vòng lặp xử lý. Các nội dung này là cơ sở quan trọng để phân tích quá trình cài đặt lớp Chip8 và các hàm xử lý opcode ở những chương sau.

2.4.1. Chu trình Fetch - Decode - Execute

Chu trình Fetch - Decode - Execute là nguyên lý hoạt động cơ bản của hầu hết các hệ thống xử lý lệnh, trong đó có CHIP-8. Đây là quá trình mà hệ thống sử dụng để đọc lệnh từ bộ nhớ, xác định ý nghĩa của lệnh và thực hiện thao tác tương ứng. Đối với một trình giả lập CHIP-8, chu trình này giữ vai trò trung tâm vì toàn bộ hoạt động của Emulator đều xoay quanh việc đọc và xử lý các opcode có trong ROM.

Giai đoạn đầu tiên của chu trình là Fetch, tức là lấy lệnh từ bộ nhớ. Trong CHIP-8, mỗi opcode có độ dài 2 byte, vì vậy trình giả lập cần đọc hai byte liên tiếp trong bộ nhớ để tạo thành một opcode hoàn chỉnh. Vị trí đọc lệnh được xác định bởi Program Counter. Khi chương trình bắt đầu chạy, Program Counter thường trỏ đến địa chỉ 0x200, là vị trí bắt đầu của chương trình CHIP-8 trong bộ nhớ. Sau khi opcode được lấy ra, Program Counter thường được tăng thêm 2 để chuẩn bị trỏ đến opcode kế tiếp.

Sau khi lấy được opcode, trình giả lập chuyển sang giai đoạn Decode, tức là giải mã lệnh. Ở giai đoạn này, Emulator phân tích các bit hoặc các chữ số hệ thập lục phân trong opcode để xác định lệnh cần thực hiện. Một opcode CHIP-8 thường chứa nhiều

phần thông tin khác nhau như nhóm lệnh, địa chỉ bộ nhớ, chỉ số thanh ghi hoặc giá trị hằng số. Vì vậy, trình giả lập cần tách các thành phần này ra để biết opcode đang yêu cầu thao tác gì và thao tác đó sẽ tác động đến thành phần nào của hệ thống.

Giai đoạn cuối cùng là Execute, tức là thực thi lệnh. Sau khi opcode đã được giải mã, trình giả lập sẽ thực hiện thao tác tương ứng với opcode đó. Tùy theo từng loại lệnh, quá trình thực thi có thể làm thay đổi giá trị của thanh ghi, cập nhật bộ nhớ, thay đổi Program Counter, ghi dữ liệu vào Stack, cập nhật Timer, kiểm tra trạng thái bàn phím hoặc vẽ dữ liệu lên màn hình. Đây là giai đoạn tạo ra sự thay đổi thực tế trong trạng thái của hệ thống CHIP-8.

Chu trình Fetch - Decode - Execute được thực hiện lặp đi lặp lại trong suốt quá trình chương trình hoạt động. Mỗi lần chu trình được thực hiện, một opcode sẽ được xử lý và trạng thái của hệ thống có thể thay đổi. Khi các chu kỳ này diễn ra liên tục với tốc độ đủ nhanh, chương trình CHIP-8 sẽ tạo ra chuyển động, phản hồi bàn phím và hiển thị hình ảnh giống như một trò chơi đang chạy. Vì vậy, có thể xem chu trình này là "nhịp hoạt động" chính của trình giả lập.

Trong trình giả lập CHIP-8, việc cài đặt chính xác chu trình Fetch - Decode - Execute có ý nghĩa rất quan trọng. Nếu quá trình lấy lệnh sai, opcode được tạo ra sẽ không đúng. Nếu quá trình giải mã sai, trình giả lập có thể gọi nhầm chức năng xử lý. Nếu quá trình thực thi sai, trạng thái của hệ thống sẽ bị thay đổi không đúng với yêu cầu của ROM. Chỉ cần một trong ba giai đoạn này hoạt động không chính xác, chương trình có thể bị lỗi, hiển thị sai hoặc không thể tiếp tục chạy.

Trong project của nhóm, chu trình này được hiện thực trong phần lõi của Emulator, cụ thể là trong lớp Chip8. Lớp này chịu trách nhiệm đọc opcode từ bộ nhớ dựa trên vị trí của Program Counter, giải mã opcode để xác định nhóm lệnh cần xử lý, sau đó gọi hàm tương ứng để thực hiện lệnh. Sau mỗi chu kỳ xử lý, các thành phần như thanh ghi, bộ nhớ, màn hình, bàn phím và timer có thể được cập nhật tùy theo opcode được thực thi.

Có thể thấy, chu trình Fetch - Decode - Execute là nền tảng để CHIP-8 thực thi chương trình. Đây cũng là cơ sở để hiểu cách một ROM được chuyển từ dữ liệu nhị phân trong bộ nhớ thành các hành động cụ thể trên màn hình và bàn phím. Việc nắm rõ chu trình này giúp người đọc dễ dàng hiểu hơn các nhóm lệnh CHIP-8 được trình bày

ở các mục tiếp theo, đồng thời tạo nền tảng cho phần phân tích cài đặt opcode trong chương trình giả lập.

2.4.2. Nhóm lệnh điều khiển

Nhóm lệnh điều khiển trong CHIP-8 là nhóm lệnh có nhiệm vụ thay đổi luồng thực thi của chương trình. Trong quá trình hoạt động bình thường, Program Counter sẽ lần lượt trỏ đến các opcode kế tiếp trong bộ nhớ. Tuy nhiên, không phải lúc nào chương trình cũng thực hiện lệnh theo thứ tự từ trên xuống dưới. Để tạo ra các cấu trúc như nhảy đến một vị trí khác, gọi chương trình con, quay về từ chương trình con hoặc bỏ qua một lệnh khi thỏa điều kiện, CHIP-8 sử dụng các opcode thuộc nhóm lệnh điều khiển.

Một trong những lệnh điều khiển cơ bản của CHIP-8 là lệnh nhảy đến một địa chỉ xác định trong bộ nhớ. Khi opcode dạng này được thực thi, Program Counter sẽ không tiếp tục tăng tuần tự mà được gán trực tiếp đến địa chỉ được chỉ định trong lệnh. Nhờ đó, chương trình có thể chuyển sang thực thi một vùng lệnh khác trong ROM. Cơ chế này thường được sử dụng để tạo vòng lặp, chuyển trạng thái chương trình hoặc điều hướng luồng xử lý trong trò chơi.

Bên cạnh lệnh nhảy, CHIP-8 còn hỗ trợ lệnh gọi chương trình con. Chương trình con là một đoạn lệnh được tách riêng để thực hiện một nhiệm vụ cụ thể và có thể được sử dụng nhiều lần trong chương trình. Khi lệnh gọi chương trình con được thực hiện, địa chỉ quay về sẽ được lưu vào Stack, sau đó Program Counter được cập nhật đến địa chỉ bắt đầu của chương trình con. Khi chương trình con thực hiện xong, lệnh quay về sẽ lấy địa chỉ đã lưu trong Stack và gán lại cho Program Counter để chương trình tiếp tục chạy từ vị trí trước đó. Cơ chế này giúp chương trình CHIP-8 được tổ chức linh hoạt hơn và tránh việc lặp lại nhiều đoạn lệnh giống nhau.

Ngoài các lệnh nhảy và gọi chương trình con, nhóm lệnh điều khiển còn bao gồm các lệnh bỏ qua opcode kế tiếp dựa trên điều kiện. Các lệnh này thường kiểm tra giá trị trong thanh ghi hoặc so sánh giữa hai thanh ghi với nhau. Nếu điều kiện đúng, Program Counter sẽ được tăng thêm để bỏ qua opcode tiếp theo. Nếu điều kiện sai, chương trình sẽ tiếp tục thực hiện opcode kế tiếp như bình thường. Đây là cách CHIP-8 tạo ra các cấu trúc rẽ nhánh đơn giản, tương tự như câu lệnh điều kiện trong các ngôn ngữ lập trình bậc cao.

Nhóm lệnh điều khiển có vai trò rất quan trọng đối với hoạt động của các ROM trò chơi. Trong một trò chơi, chương trình cần liên tục kiểm tra trạng thái, xử lý điều kiện thắng thua, cập nhật vị trí đối tượng, lặp lại vòng chơi và chuyển đổi giữa các phần khác nhau của chương trình. Những thao tác này đều cần đến khả năng điều khiển luồng thực thi. Nếu nhóm lệnh điều khiển không được cài đặt chính xác, ROM có thể bị nhảy sai địa chỉ, không thể quay về từ chương trình con hoặc chạy sai logic.

Trong trình giả lập CHIP-8 của đề tài, các lệnh điều khiển được xử lý trong lớp Chip8 thông qua các hàm xử lý opcode tương ứng. Khi một opcode thuộc nhóm điều khiển được giải mã, chương trình sẽ cập nhật Program Counter hoặc Stack tùy theo yêu cầu của lệnh. Đối với các lệnh nhảy, Program Counter được thay đổi trực tiếp. Đối với các lệnh gọi chương trình con, địa chỉ hiện tại được lưu vào Stack trước khi chuyển sang địa chỉ mới. Đối với các lệnh quay về, địa chỉ đã lưu trong Stack được khôi phục để tiếp tục quá trình thực thi.

Có thể thấy, nhóm lệnh điều khiển là nền tảng để CHIP-8 thực hiện các thao tác thay đổi luồng chương trình. Nhóm lệnh này không trực tiếp tạo ra hình ảnh hay xử lý dữ liệu đồ họa, nhưng lại quyết định chương trình sẽ đi theo hướng nào, lặp lại đoạn lệnh nào hoặc chuyển sang vùng xử lý nào trong bộ nhớ. Vì vậy, việc hiểu và cài đặt đúng nhóm lệnh điều khiển là điều kiện cần thiết để trình giả lập CHIP-8 có thể chạy ổn định các ROM trò chơi.

2.4.3. Nhóm lệnh xử lý dữ liệu

Nhóm lệnh xử lý dữ liệu trong CHIP-8 là nhóm lệnh dùng để thao tác với các giá trị được lưu trong thanh ghi, bộ nhớ và một số thành phần liên quan đến quá trình thực thi chương trình. Nếu nhóm lệnh điều khiển quyết định luồng chạy của chương trình, thì nhóm lệnh xử lý dữ liệu quyết định cách chương trình lưu trữ, thay đổi và tính toán các giá trị trong quá trình hoạt động. Đây là nhóm lệnh xuất hiện rất thường xuyên trong các ROM CHIP-8, đặc biệt là trong các trò chơi cần xử lý vị trí, điểm số, trạng thái nhân vật hoặc các biến điều khiển.

Trong CHIP-8, dữ liệu thường được lưu tạm thời trong các thanh ghi đa dụng từ V0 đến VF. Các lệnh xử lý dữ liệu có thể gán một giá trị trực tiếp vào thanh ghi, cộng thêm một giá trị vào thanh ghi, sao chép dữ liệu giữa các thanh ghi hoặc thực hiện các phép toán số học và logic. Những thao tác này tuy đơn giản nhưng là nền tảng để chương

trình thay đổi trạng thái trong quá trình chạy. Ví dụ, một trò chơi có thể sử dụng thanh ghi để lưu tọa độ của nhân vật, sau đó thay đổi giá trị thanh ghi khi người dùng nhấn phím điều khiển.

Một nhóm thao tác quan trọng trong xử lý dữ liệu là các phép toán số học. CHIP-8 hỗ trợ những thao tác như cộng, trừ và cập nhật cờ trạng thái khi xảy ra nhớ hoặc mượn. Trong các phép toán này, thanh ghi VF thường được sử dụng như một cờ báo trạng thái. Việc cập nhật đúng thanh ghi VF có ý nghĩa quan trọng vì một số chương trình có thể dựa vào giá trị của cờ này để quyết định các bước xử lý tiếp theo. Nếu cờ trạng thái bị xử lý sai, chương trình có thể đưa ra kết quả không chính xác hoặc chạy sai logic.

Bên cạnh các phép toán số học, CHIP-8 còn có các lệnh logic như AND, OR, XOR hoặc dịch bit. Các lệnh này làm việc trực tiếp trên dữ liệu nhị phân trong thanh ghi. Mặc dù ở mức báo cáo người dùng thường không quan sát trực tiếp các phép toán bit, nhưng chúng có vai trò quan trọng trong việc xử lý trạng thái, tạo hiệu ứng hoặc biến đổi dữ liệu trong chương trình. Đối với trình giả lập, các thao tác logic cần được cài đặt đúng theo đặc tả của opcode để đảm bảo ROM hoạt động ổn định.

Nhóm lệnh xử lý dữ liệu cũng bao gồm các lệnh liên quan đến thanh ghi địa chỉ I và bộ nhớ. Thanh ghi I thường được dùng để lưu địa chỉ của một vùng nhớ cần truy cập, chẳng hạn như vùng chứa sprite, fontset hoặc dữ liệu cần lưu trữ. Một số opcode có nhiệm vụ gán địa chỉ vào thanh ghi I, cộng giá trị của một thanh ghi vào I, lưu nhiều thanh ghi vào bộ nhớ hoặc đọc dữ liệu từ bộ nhớ trở lại thanh ghi. Nhờ các lệnh này, chương trình CHIP-8 có thể trao đổi dữ liệu giữa thanh ghi và bộ nhớ trong quá trình thực thi.

Ngoài ra, CHIP-8 còn có các lệnh phục vụ cho việc chuyển đổi và lưu trữ dữ liệu theo định dạng đặc biệt. Ví dụ, một số lệnh có thể chuyển giá trị trong thanh ghi sang dạng thập phân để lưu vào bộ nhớ, thường được sử dụng khi chương trình cần hiển thị số lên màn hình. Những lệnh này cho thấy mặc dù CHIP-8 có kiến trúc đơn giản, hệ thống vẫn cung cấp các thao tác cần thiết để chương trình có thể xử lý dữ liệu phục vụ cho hiển thị và logic trò chơi.

Trong project của nhóm, các lệnh xử lý dữ liệu được hiện thực trong lớp Chip8 thông qua các hàm xử lý opcode tương ứng. Khi một opcode thuộc nhóm xử lý dữ liệu

được giải mã, chương trình sẽ xác định thanh ghi hoặc vùng nhớ liên quan, sau đó thực hiện thao tác cần thiết như gán giá trị, cộng trừ, thực hiện phép toán logic hoặc đọc ghi bộ nhớ. Các thao tác này làm thay đổi trạng thái bên trong của Emulator và ảnh hưởng trực tiếp đến kết quả thực thi của ROM.

Có thể thấy, nhóm lệnh xử lý dữ liệu giữ vai trò nền tảng trong hoạt động của CHIP-8. Hầu hết các chương trình đều cần sử dụng nhóm lệnh này để lưu trữ trạng thái, tính toán giá trị, cập nhật tọa độ, xử lý điều kiện và chuẩn bị dữ liệu cho các thao tác đồ họa hoặc bàn phím. Vì vậy, việc cài đặt chính xác nhóm lệnh xử lý dữ liệu là yêu cầu quan trọng để trình giả lập CHIP-8 có thể thực thi đúng các ROM trò chơi.

2.4.4. Nhóm lệnh đồ họa

Nhóm lệnh đồ họa trong CHIP-8 là nhóm lệnh có nhiệm vụ điều khiển việc hiển thị hình ảnh trên màn hình. Đây là nhóm lệnh rất quan trọng đối với các ROM trò chơi, vì hầu hết các trò chơi đều cần hiển thị nhân vật, vật thể, điểm số hoặc trạng thái trò chơi lên màn hình. Nếu các lệnh điều khiển và xử lý dữ liệu quyết định logic bên trong của chương trình, thì nhóm lệnh đồ họa là thành phần giúp người dùng quan sát được kết quả của quá trình xử lý đó.

Màn hình của CHIP-8 có độ phân giải 64×32 pixel và chỉ sử dụng hai trạng thái điểm ảnh là bật hoặc tắt. Do đó, việc xử lý đồ họa trong CHIP-8 đơn giản hơn nhiều so với các hệ thống hiện đại có màu sắc và độ phân giải cao. Hình ảnh trong CHIP-8 thường được tạo ra bằng các sprite. Sprite là một dãy dữ liệu nhị phân được lưu trong bộ nhớ, trong đó mỗi bit đại diện cho trạng thái của một điểm ảnh. Khi một sprite được vẽ lên màn hình, trình giả lập sẽ đọc dữ liệu từ bộ nhớ và cập nhật các điểm ảnh tương ứng trong vùng hiển thị.

Opcode đồ họa quan trọng nhất trong CHIP-8 là lệnh vẽ sprite lên màn hình. Lệnh này thường sử dụng giá trị trong các thanh ghi để xác định tọa độ vẽ, đồng thời sử dụng thanh ghi địa chỉ I để xác định vị trí dữ liệu sprite trong bộ nhớ. Khi lệnh vẽ được thực hiện, trình giả lập sẽ lấy từng byte dữ liệu sprite từ bộ nhớ, kiểm tra từng bit trong byte đó và cập nhật các điểm ảnh trên màn hình. Quá trình này giúp chương trình hiển thị các đối tượng như nhân vật, chữ số, vật cản hoặc các thành phần khác của trò chơi.

Một đặc điểm quan trọng của cơ chế vẽ trong CHIP-8 là việc sử dụng phép XOR để cập nhật điểm ảnh. Theo cơ chế này, nếu một điểm ảnh đang tắt và sprite yêu cầu bật

điểm ảnh đó, điểm ảnh sẽ được bật. Ngược lại, nếu một điểm ảnh đang bật và sprite tiếp tục vẽ lên cùng vị trí đó, điểm ảnh sẽ bị tắt. Cách xử lý này giúp CHIP-8 có thể tạo hiệu ứng xóa và vẽ lại hình ảnh một cách đơn giản mà không cần hệ thống đồ họa phức tạp. Đồng thời, cơ chế XOR cũng hỗ trợ phát hiện va chạm giữa các đối tượng trên màn hình.

Trong quá trình vẽ sprite, thanh ghi VF thường được sử dụng như một cờ va chạm. Nếu có điểm ảnh nào bị tắt do quá trình vẽ XOR, VF sẽ được đặt để báo hiệu rằng đã xảy ra va chạm. Cơ chế này rất hữu ích trong các trò chơi, chẳng hạn như khi nhân vật chạm vào vật cản, đạn va chạm với mục tiêu hoặc các đối tượng giao nhau trên màn hình. Nhờ cờ va chạm, chương trình có thể xử lý các tình huống trong trò chơi mà không cần một hệ thống kiểm tra va chạm phức tạp.

Ngoài lệnh vẽ sprite, nhóm lệnh đồ họa còn bao gồm lệnh xóa màn hình. Khi lệnh này được thực thi, toàn bộ vùng hiển thị của CHIP-8 sẽ được đưa về trạng thái ban đầu, tức là các điểm ảnh đều tắt. Lệnh xóa màn hình thường được sử dụng khi bắt đầu chương trình, chuyển sang màn chơi mới hoặc cần làm mới trạng thái hiển thị. Đây là một thao tác đơn giản nhưng cần thiết để đảm bảo hình ảnh trên màn hình không bị chồng lấn hoặc giữ lại dữ liệu cũ không còn phù hợp.

Trong trình giả lập CHIP-8, dữ liệu màn hình thường được lưu trong một vùng đệm hiển thị. Khi opcode đồ họa được thực thi, trình giả lập sẽ cập nhật dữ liệu trong vùng đệm này trước. Sau đó, phần hiển thị của chương trình sẽ đưa dữ liệu từ vùng đệm lên cửa sổ hiển thị thực tế. Cách tổ chức này giúp tách biệt giữa việc xử lý logic đồ họa của CHIP-8 và việc render hình ảnh trên hệ điều hành hiện đại.

Trong project của nhóm, phần xử lý lệnh đồ họa được thực hiện trong lớp Chip8, nơi dữ liệu video được cập nhật dựa trên opcode đang được thực thi. Sau khi dữ liệu màn hình thay đổi, lớp Platform sử dụng thư viện SDL2 để cập nhật hình ảnh lên cửa sổ chương trình. Vì màn hình gốc của CHIP-8 chỉ có kích thước 64×32 pixel, chương trình cần phóng đại hình ảnh khi hiển thị để người dùng có thể quan sát rõ hơn trên màn hình máy tính.

Nhóm lệnh đồ họa có ảnh hưởng trực tiếp đến trải nghiệm của người dùng khi chạy ROM. Nếu lệnh vẽ sprite được xử lý sai, hình ảnh có thể bị méo, mất chi tiết, hiển thị sai vị trí hoặc không thể phát hiện va chạm. Nếu lệnh xóa màn hình hoạt động không

đúng, các điểm ảnh cũ có thể vẫn còn tồn tại và làm sai giao diện trò chơi. Vì vậy, việc cài đặt chính xác nhóm lệnh đồ họa là yêu cầu quan trọng để trình giả lập CHIP-8 có thể chạy ổn định các trò chơi.

Có thể thấy, nhóm lệnh đồ họa là cầu nối giữa dữ liệu xử lý bên trong Emulator và hình ảnh mà người dùng quan sát được. Thông qua các lệnh vẽ sprite, xóa màn hình và cập nhật vùng đệm hiển thị, CHIP-8 có thể tạo ra các trò chơi đơn giản nhưng vẫn có khả năng tương tác trực quan. Đây cũng là một trong những nhóm lệnh quan trọng nhất cần được phân tích và cài đặt cẩn thận trong quá trình xây dựng trình giả lập.

2.4.5. Nhóm lệnh bàn phím

Nhóm lệnh bàn phím trong CHIP-8 là nhóm lệnh dùng để kiểm tra và xử lý thao tác nhập liệu từ người dùng. Đối với các chương trình trò chơi, bàn phím là phương tiện chính để người chơi điều khiển nhân vật, lựa chọn hành động hoặc thay đổi trạng thái của trò chơi. Vì vậy, bên cạnh các lệnh xử lý dữ liệu và đồ họa, nhóm lệnh bàn phím giữ vai trò quan trọng trong việc tạo ra sự tương tác giữa người dùng và chương trình đang chạy trên trình giả lập.

CHIP-8 sử dụng bàn phím gồm 16 phím, được biểu diễn bằng các giá trị hệ thập lục phân từ 0 đến F. Cách bố trí này khác với bàn phím máy tính hiện đại, do đó khi xây dựng trình giả lập cần có cơ chế ánh xạ phím. Cơ chế ánh xạ cho phép các phím trên bàn phím máy tính được chuyển đổi thành các phím tương ứng trong hệ thống CHIP-8. Nhờ vậy, khi người dùng nhấn một phím trên máy tính, chương trình CHIP-8 vẫn có thể nhận biết như đang thao tác trên bàn phím gốc của hệ thống.

Các opcode liên quan đến bàn phím thường được sử dụng để kiểm tra xem một phím có đang được nhấn hay không. Một số lệnh có chức năng bỏ qua opcode kế tiếp nếu phím được chỉ định đang được nhấn. Ngược lại, một số lệnh khác sẽ bỏ qua opcode kế tiếp nếu phím đó không được nhấn. Cơ chế này giúp chương trình CHIP-8 có thể tạo ra các điều kiện xử lý dựa trên thao tác của người dùng. Ví dụ, trong một trò chơi, nếu người dùng nhấn phím di chuyển sang trái, chương trình có thể thay đổi tọa độ của nhân vật; nếu không có phím nào được nhấn, chương trình sẽ tiếp tục giữ nguyên trạng thái hiện tại.

Ngoài các lệnh kiểm tra trạng thái phím, CHIP-8 còn có lệnh chờ người dùng nhấn phím. Khi opcode này được thực thi, chương trình sẽ tạm dừng ở trạng thái hiện

tại cho đến khi nhận được một phím hợp lệ từ bàn phím. Sau khi người dùng nhấn phím, giá trị của phím đó sẽ được lưu vào thanh ghi được chỉ định, sau đó chương trình tiếp tục thực hiện các lệnh tiếp theo. Lệnh này thường được sử dụng trong các chương trình cần người dùng nhập dữ liệu hoặc chờ thao tác xác nhận trước khi tiếp tục xử lý.

Trong trình giả lập, việc xử lý bàn phím không chỉ đơn giản là đọc một phím tại một thời điểm, mà cần quản lý trạng thái của toàn bộ bàn phím CHIP-8. Mỗi phím có thể ở trạng thái được nhấn hoặc không được nhấn. Trạng thái này cần được cập nhật liên tục trong vòng lặp chính của chương trình, bởi vì các ROM trò chơi thường kiểm tra bàn phím nhiều lần trong quá trình hoạt động. Nếu trạng thái phím không được cập nhật chính xác, trò chơi có thể không nhận thao tác điều khiển hoặc phản hồi sai với hành động của người dùng.

Trong project của nhóm, phần nhận sự kiện bàn phím được xử lý thông qua lớp Platform bằng thư viện SDL2. Khi người dùng nhấn hoặc thả một phím trên bàn phím máy tính, SDL2 sẽ tạo ra sự kiện tương ứng. Lớp Platform tiếp nhận sự kiện này và cập nhật trạng thái trong mảng bàn phím của CHIP-8. Sau đó, lớp Chip8 sử dụng trạng thái bàn phím này khi thực thi các opcode liên quan đến nhập liệu. Cách tổ chức này giúp tách biệt rõ ràng giữa phần giao tiếp với hệ điều hành và phần xử lý logic của Emulator.

Nhóm lệnh bàn phím có liên hệ chặt chẽ với nhóm lệnh điều khiển chương trình. Khi một opcode kiểm tra trạng thái phím được thực thi, kết quả kiểm tra có thể làm thay đổi Program Counter bằng cách bỏ qua hoặc không bỏ qua lệnh kế tiếp. Vì vậy, các lệnh bàn phím không chỉ dùng để nhận dữ liệu nhập mà còn có ảnh hưởng trực tiếp đến luồng thực thi của chương trình. Đây là lý do việc xử lý bàn phím cần được thực hiện chính xác, đặc biệt đối với các trò chơi yêu cầu phản hồi nhanh từ người dùng.

Trong các ROM trò chơi CHIP-8, nhóm lệnh bàn phím thường được sử dụng để điều khiển nhân vật, bắn, di chuyển, chọn chế độ hoặc bắt đầu trò chơi. Ví dụ, một trò chơi có thể kiểm tra phím di chuyển để thay đổi vị trí đối tượng trên màn hình, hoặc kiểm tra phím hành động để thực hiện thao tác bắn, thả vật thể hoặc bắt đầu màn chơi. Khi các lệnh bàn phím phối hợp với lệnh đồ họa và lệnh xử lý dữ liệu, chương trình có thể tạo ra trải nghiệm tương tác hoàn chỉnh cho người dùng.

Có thể thấy, nhóm lệnh bàn phím là thành phần quan trọng giúp CHIP-8 nhận dữ liệu điều khiển từ người dùng. Mặc dù số lượng opcode liên quan đến bàn phím không

nhiều, nhưng chúng ảnh hưởng trực tiếp đến khả năng tương tác của chương trình. Việc cài đặt chính xác nhóm lệnh này giúp trình giả lập phản hồi đúng thao tác nhập liệu, từ đó đảm bảo các ROM trò chơi có thể hoạt động ổn định và mang lại trải nghiệm gần giống với hệ thống CHIP-8 gốc.

2.5. Lập trình hướng đối tượng trong C++

Lập trình hướng đối tượng là một phương pháp lập trình được sử dụng phổ biến trong quá trình phát triển phần mềm hiện nay. Thay vì chỉ tổ chức chương trình thành các hàm xử lý riêng lẻ, lập trình hướng đối tượng cho phép xây dựng chương trình dựa trên các đối tượng có dữ liệu và hành vi riêng. Mỗi đối tượng đại diện cho một thành phần cụ thể trong hệ thống, giúp chương trình được tổ chức rõ ràng, dễ quản lý và dễ mở rộng hơn.

Trong ngôn ngữ C++, lập trình hướng đối tượng được thể hiện thông qua các khái niệm như lớp, đối tượng, thuộc tính, phương thức và cơ chế đóng gói dữ liệu. Lớp được dùng để mô tả cấu trúc và hành vi chung của một nhóm đối tượng, trong khi đối tượng là thể hiện cụ thể của lớp trong chương trình. Nhờ cách tổ chức này, các thành phần trong phần mềm có thể được chia thành những đơn vị độc lập, mỗi đơn vị đảm nhiệm một chức năng riêng.

Đối với đề tài xây dựng trình giả lập CHIP-8, lập trình hướng đối tượng giữ vai trò quan trọng trong việc tổ chức mã nguồn. Chương trình không được viết toàn bộ trong một file duy nhất mà được chia thành các lớp có trách nhiệm rõ ràng. Phần mô phỏng hệ thống CHIP-8 được đặt trong lớp Chip8, còn phần hiển thị màn hình và xử lý bàn phím được đặt trong lớp Platform. Cách phân chia này giúp tách biệt phần lõi giả lập với phần giao tiếp bên ngoài, làm cho chương trình dễ hiểu và thuận tiện hơn trong quá trình kiểm thử.

Việc áp dụng lập trình hướng đối tượng cũng giúp chương trình thể hiện được nguyên tắc phân chia trách nhiệm giữa các thành phần. Lớp Chip8 tập trung vào các chức năng liên quan đến bộ nhớ, thanh ghi, Stack, Program Counter, Timer và xử lý opcode. Trong khi đó, lớp Platform chịu trách nhiệm làm việc với thư viện SDL2 để tạo cửa sổ hiển thị, cập nhật hình ảnh và tiếp nhận sự kiện bàn phím. Nhờ đó, khi cần chỉnh sửa phần xử lý đồ họa hoặc bàn phím, nhóm có thể tập trung vào lớp Platform mà không làm ảnh hưởng lớn đến phần xử lý lõi của Emulator.

Có thể thấy, lập trình hướng đối tượng không chỉ là kiến thức lý thuyết mà còn được vận dụng trực tiếp trong quá trình xây dựng chương trình của đề tài. Việc sử dụng lớp và đối tượng giúp mã nguồn có cấu trúc rõ ràng, tăng khả năng bảo trì và tạo nền tảng cho việc mở rộng chương trình trong tương lai. Đây cũng là nội dung phù hợp với mục tiêu của học phần Lập trình hướng đối tượng và Kỹ thuật phần mềm, khi sinh viên cần biết cách áp dụng kiến thức đã học vào một sản phẩm phần mềm cụ thể.

2.5.1. Lớp và đối tượng

Trong lập trình hướng đối tượng, lớp và đối tượng là hai khái niệm cơ bản và quan trọng nhất. Lớp có thể được hiểu là một khuôn mẫu dùng để mô tả đặc điểm và hành vi chung của một nhóm đối tượng. Trong một lớp thường bao gồm các thuộc tính dùng để lưu trữ dữ liệu và các phương thức dùng để xử lý dữ liệu hoặc thực hiện một chức năng nào đó. Từ một lớp, chương trình có thể tạo ra các đối tượng cụ thể để sử dụng trong quá trình thực thi.

Đối tượng là một thể hiện cụ thể của lớp. Nếu lớp là bản thiết kế, thì đối tượng là thành phần được tạo ra từ bản thiết kế đó trong chương trình. Mỗi đối tượng có thể có dữ liệu riêng nhưng vẫn tuân theo cấu trúc và hành vi được định nghĩa trong lớp. Cách tổ chức này giúp chương trình mô hình hóa các thành phần trong hệ thống một cách rõ ràng hơn, đặc biệt đối với những chương trình có nhiều thành phần cần phối hợp với nhau.

Trong ngôn ngữ C++, lớp được khai báo bằng từ khóa class. Bên trong lớp có thể chứa các biến thành viên, hàm thành viên và các phạm vi truy cập như public, private hoặc protected. Các biến thành viên thường biểu diễn trạng thái của đối tượng, trong khi các hàm thành viên biểu diễn hành vi của đối tượng. Nhờ vậy, dữ liệu và chức năng xử lý dữ liệu được đặt trong cùng một đơn vị, giúp chương trình dễ quản lý hơn so với cách viết toàn bộ chương trình bằng các biến và hàm rời rạc.

Đối với đề tài xây dựng trình giả lập CHIP-8, việc sử dụng lớp và đối tượng giúp mô hình hóa các thành phần chính của chương trình một cách hợp lý. Thay vì viết toàn bộ mã nguồn trong một file duy nhất, chương trình được chia thành các lớp có nhiệm vụ riêng. Lớp Chip8 được dùng để mô phỏng hệ thống CHIP-8, bao gồm các thành phần như bộ nhớ, thanh ghi, Stack, Program Counter, Timer, màn hình, bàn phím và các hàm xử lý opcode. Đây là lớp giữ vai trò lõi của trình giả lập.

Bên cạnh lớp Chip8, chương trình còn sử dụng lớp Platform để xử lý các chức năng liên quan đến thư viện SDL2. Lớp này chịu trách nhiệm tạo cửa sổ hiển thị, cập nhật dữ liệu hình ảnh lên màn hình và xử lý các sự kiện bàn phím từ người dùng. Việc tách riêng lớp Platform khỏi lớp Chip8 giúp chương trình phân biệt rõ phần mô phỏng logic của hệ thống CHIP-8 với phần giao tiếp cùng hệ điều hành và thiết bị ngoại vi.

Trong quá trình chương trình chạy, các đối tượng được tạo ra từ những lớp này sẽ phối hợp với nhau để thực hiện chức năng của Emulator. Đối tượng Chip8 đảm nhiệm việc nạp ROM, xử lý opcode và cập nhật trạng thái bên trong của hệ thống. Đối tượng Platform đảm nhiệm việc hiển thị dữ liệu video và nhận thao tác bàn phím. Chương trình chính sẽ điều khiển sự phối hợp giữa các đối tượng này thông qua vòng lặp chính của Emulator.

Việc sử dụng lớp và đối tượng mang lại nhiều lợi ích cho project. Trước hết, mã nguồn trở nên rõ ràng hơn vì mỗi lớp có một trách nhiệm cụ thể. Khi cần tìm hiểu phần xử lý opcode, người lập trình có thể tập trung vào lớp Chip8. Khi cần kiểm tra phần hiển thị hoặc bàn phím, người lập trình có thể xem lớp Platform. Ngoài ra, cách tổ chức này cũng giúp chương trình dễ bảo trì hơn, vì việc sửa đổi một thành phần sẽ ít ảnh hưởng đến các thành phần còn lại nếu các lớp được thiết kế hợp lý.

Có thể thấy, lớp và đối tượng là nền tảng quan trọng trong lập trình hướng đối tượng bằng C++. Trong đề tài này, việc áp dụng hai khái niệm này giúp chương trình giả lập CHIP-8 được tổ chức khoa học hơn, phù hợp với yêu cầu của môn Lập trình hướng đối tượng và Kỹ thuật phần mềm. Đây cũng là cơ sở để tiếp tục phân tích cơ chế đóng gói dữ liệu và cách áp dụng OOP trong chương trình ở các mục tiếp theo.

2.5.2. Đóng gói dữ liệu

Đóng gói dữ liệu là một trong những đặc trưng quan trọng của lập trình hướng đối tượng. Trong C++, đóng gói dữ liệu cho phép kết hợp dữ liệu và các phương thức xử lý dữ liệu vào bên trong cùng một lớp. Thay vì để các biến được truy cập và thay đổi trực tiếp từ nhiều nơi trong chương trình, dữ liệu sẽ được quản lý thông qua các phương thức của lớp. Cách tổ chức này giúp bảo vệ trạng thái bên trong của đối tượng, đồng thời làm cho chương trình rõ ràng và an toàn hơn trong quá trình phát triển.

Trong lập trình hướng đối tượng, một đối tượng thường có trạng thái bên trong và các hành vi tương ứng. Trạng thái bên trong được biểu diễn thông qua các thuộc tính,

còn hành vi được biểu diễn thông qua các phương thức. Nếu các thuộc tính được truy cập tự do từ bên ngoài, chương trình rất dễ xảy ra lỗi do dữ liệu bị thay đổi không đúng cách. Vì vậy, cơ chế đóng gói giúp giới hạn quyền truy cập trực tiếp vào dữ liệu, đồng thời cung cấp các phương thức phù hợp để thao tác với dữ liệu đó.

Trong C++, đóng gói dữ liệu thường được thực hiện thông qua các phạm vi truy cập như private, public và protected. Các thành phần được khai báo trong phạm vi private chỉ có thể được truy cập bên trong lớp, không thể truy cập trực tiếp từ bên ngoài. Các thành phần public là những phương thức hoặc thuộc tính được phép sử dụng từ bên ngoài lớp. Nhờ cơ chế này, người lập trình có thể che giấu chi tiết cài đặt bên trong và chỉ cung cấp những chức năng cần thiết cho các thành phần khác của chương trình.

Đối với đề tài xây dựng trình giả lập CHIP-8, đóng gói dữ liệu có vai trò rất quan trọng vì hệ thống giả lập có nhiều thành phần trạng thái cần được quản lý chính xác. Các thành phần như bộ nhớ, thanh ghi, Stack, Program Counter, Timer, bàn phím và vùng đệm hiển thị đều là dữ liệu bên trong của hệ thống CHIP-8. Những dữ liệu này không nên bị thay đổi tùy ý từ bên ngoài, vì chỉ cần một giá trị bị cập nhật sai, toàn bộ quá trình thực thi ROM có thể bị ảnh hưởng.

Trong project của nhóm, lớp Chip8 đóng vai trò quản lý trạng thái bên trong của hệ thống CHIP-8. Các dữ liệu như bộ nhớ, thanh ghi, Stack, Program Counter, Timer và video buffer được tổ chức trong lớp này. Những thành phần bên ngoài không cần trực tiếp can thiệp vào cách bộ nhớ hoặc thanh ghi được xử lý, mà chỉ cần gọi các phương thức phù hợp như nạp ROM hoặc thực hiện một chu kỳ xử lý lệnh. Cách tổ chức này thể hiện rõ nguyên tắc đóng gói, vì phần lõi xử lý của hệ thống được quản lý tập trung trong một lớp.

Tương tự, lớp Platform cũng thể hiện cơ chế đóng gói trong quá trình làm việc với thư viện SDL2. Việc khởi tạo cửa sổ, renderer, texture, cập nhật màn hình và xử lý sự kiện bàn phím đều được đặt trong lớp Platform. Chương trình chính không cần biết chi tiết SDL2 tạo cửa sổ như thế nào hay cập nhật texture ra sao, mà chỉ cần gọi các phương thức đã được lớp Platform cung cấp. Điều này giúp phần giao tiếp với SDL2 được tách biệt khỏi phần xử lý logic của Emulator.

Đóng gói dữ liệu còn giúp chương trình dễ bảo trì hơn. Khi cần thay đổi cách xử lý bộ nhớ hoặc cách cập nhật màn hình, người lập trình có thể chỉnh sửa bên trong lớp

tương ứng mà không cần thay đổi quá nhiều ở các phần khác của chương trình. Ví dụ, nếu cần thay đổi cách render hình ảnh bằng SDL2, phần chỉnh sửa chủ yếu nằm trong lớp Platform. Nếu cần chỉnh sửa cách xử lý opcode, phần thay đổi chủ yếu nằm trong lớp Chip8. Cách phân chia này giúp giảm sự phụ thuộc giữa các thành phần và hạn chế lỗi phát sinh khi mở rộng chương trình.

Ngoài ra, đóng gói dữ liệu còn giúp chương trình dễ đọc và dễ kiểm thử hơn. Khi mỗi lớp chịu trách nhiệm quản lý một nhóm dữ liệu và chức năng riêng, người đọc mã nguồn có thể nhanh chóng xác định phần nào của chương trình đảm nhiệm nhiệm vụ nào. Đối với một trình giả lập, đây là điều rất cần thiết vì chương trình có nhiều thành phần hoạt động cùng lúc như xử lý opcode, cập nhật timer, xử lý bàn phím và hiển thị màn hình. Nếu không có đóng gói, mã nguồn sẽ dễ bị rối và khó kiểm soát.

Có thể thấy, đóng gói dữ liệu không chỉ là một khái niệm lý thuyết trong lập trình hướng đối tượng mà còn được áp dụng trực tiếp trong project CHIP-8 Emulator. Nhờ cơ chế đóng gói, chương trình được tổ chức thành các lớp có trách nhiệm rõ ràng, dữ liệu bên trong được quản lý tốt hơn và mã nguồn trở nên dễ bảo trì hơn. Đây là một yếu tố quan trọng giúp chương trình đáp ứng được yêu cầu của môn Lập trình hướng đối tượng và Kỹ thuật phần mềm.

2.5.3. Áp dụng OOP trong chương trình

Trong đề tài xây dựng trình giả lập CHIP-8, lập trình hướng đối tượng không chỉ được trình bày như một phần lý thuyết mà còn được áp dụng trực tiếp vào quá trình tổ chức và phát triển chương trình. Thay vì viết toàn bộ mã nguồn theo dạng tuần tự hoặc đặt tất cả chức năng trong một file duy nhất, chương trình được chia thành các lớp có nhiệm vụ riêng biệt. Cách tổ chức này giúp mã nguồn rõ ràng hơn, dễ theo dõi hơn và phù hợp với yêu cầu của học phần Lập trình hướng đối tượng và Kỹ thuật phần mềm.

Thành phần quan trọng nhất trong chương trình là lớp Chip8. Lớp này đảm nhiệm vai trò mô phỏng phần lõi của hệ thống CHIP-8. Các thành phần bên trong của CHIP-8 như bộ nhớ, thanh ghi, Stack, Program Counter, Timer, bàn phím và vùng dữ liệu hiển thị được quản lý trong lớp này. Ngoài ra, lớp Chip8 còn chịu trách nhiệm nạp ROM, thực hiện chu trình xử lý lệnh và xử lý các opcode của hệ thống. Có thể xem lớp Chip8 là phần mô phỏng trực tiếp kiến trúc CHIP-8 trong chương trình.

Việc đặt các thành phần xử lý chính vào lớp Chip8 giúp chương trình thể hiện rõ nguyên tắc đóng gói và phân chia trách nhiệm. Các dữ liệu nội bộ của CHIP-8 không bị xử lý rải rác ở nhiều nơi mà được quản lý tập trung trong một lớp. Khi chương trình cần thực hiện một chu kỳ xử lý, đối tượng Chip8 sẽ tự đảm nhiệm việc lấy opcode từ bộ nhớ, giải mã opcode, thực thi lệnh và cập nhật trạng thái của hệ thống. Điều này giúp phần lõi giả lập được tổ chức rõ ràng và dễ kiểm soát hơn.

Bên cạnh lớp Chip8, chương trình còn sử dụng lớp Platform để xử lý các chức năng liên quan đến thư viện SDL2. Lớp này chịu trách nhiệm tạo cửa sổ hiển thị, cập nhật hình ảnh lên màn hình và xử lý sự kiện bàn phím từ người dùng. Việc tách riêng lớp Platform giúp chương trình không trộn lẫn phần xử lý logic của CHIP-8 với phần giao tiếp cùng hệ điều hành. Nhờ đó, lớp Chip8 chỉ cần tập trung vào việc mô phỏng hệ thống, còn lớp Platform tập trung vào việc hiển thị và nhập liệu.

Sự phân chia giữa lớp Chip8 và lớp Platform thể hiện rõ tư duy thiết kế hướng đối tượng trong chương trình. Mỗi lớp có một nhiệm vụ riêng, dữ liệu riêng và các phương thức riêng để xử lý nhiệm vụ đó. Lớp Chip8 không cần quan tâm SDL2 tạo cửa sổ như thế nào, còn lớp Platform không cần biết chi tiết từng opcode được xử lý ra sao. Sự tách biệt này làm giảm sự phụ thuộc giữa các thành phần, đồng thời giúp chương trình dễ bảo trì và mở rộng hơn.

Trong chương trình chính, các đối tượng được tạo ra từ những lớp trên và phối hợp với nhau thông qua vòng lặp chính của Emulator. Đối tượng Chip8 chịu trách nhiệm nạp ROM và xử lý opcode, trong khi đối tượng Platform nhận tín hiệu bàn phím và hiển thị dữ liệu màn hình. Sau mỗi chu kỳ xử lý, trạng thái video của CHIP-8 được cập nhật và chuyển sang Platform để hiển thị lên cửa sổ chương trình. Quá trình này cho thấy các đối tượng trong chương trình không hoạt động riêng lẻ mà phối hợp với nhau để tạo thành một hệ thống hoàn chỉnh.

Việc áp dụng OOP còn giúp chương trình thuận tiện hơn trong quá trình kiểm thử và chỉnh sửa. Khi xảy ra lỗi liên quan đến xử lý opcode, nhóm có thể tập trung kiểm tra lớp Chip8. Khi lỗi liên quan đến hiển thị màn hình hoặc bàn phím, nhóm có thể tập trung vào lớp Platform. Cách tổ chức này giúp việc xác định nguyên nhân lỗi nhanh hơn so với việc toàn bộ chương trình được viết chung trong một khối mã lớn.

Ngoài ra, thiết kế hướng đối tượng cũng tạo điều kiện cho việc mở rộng chương trình trong tương lai. Nếu muốn bổ sung thêm tính năng như âm thanh, giao diện chọn ROM, điều chỉnh tốc độ chạy hoặc công cụ hỗ trợ kiểm thử, nhóm có thể xây dựng thêm các lớp mới hoặc mở rộng các lớp hiện có mà không cần thay đổi toàn bộ cấu trúc chương trình. Đây là một ưu điểm quan trọng của lập trình hướng đối tượng trong quá trình phát triển phần mềm.

Có thể thấy, việc áp dụng lập trình hướng đối tượng trong chương trình CHIP-8 Emulator giúp mã nguồn được tổ chức khoa học, rõ ràng và phù hợp với quy trình phát triển phần mềm. Các lớp trong chương trình được thiết kế theo từng nhóm chức năng cụ thể, trong đó lớp Chip8 đảm nhiệm phần lõi giả lập, lớp Platform đảm nhiệm phần hiển thị và nhập liệu, còn chương trình chính điều phối hoạt động giữa các đối tượng. Cách tổ chức này không chỉ giúp chương trình hoạt động ổn định mà còn thể hiện được khả năng vận dụng kiến thức của môn Lập trình hướng đối tượng và Kỹ thuật phần mềm vào một sản phẩm thực tế.

2.6. Thư viện SDL2

SDL2 là một thư viện lập trình đa phương tiện được sử dụng phổ biến trong việc xây dựng các ứng dụng có liên quan đến đồ họa, âm thanh, bàn phím, chuột và các thiết bị nhập xuất khác. Thư viện này thường được dùng trong phát triển trò chơi, trình giả lập, phần mềm mô phỏng hoặc các ứng dụng cần hiển thị hình ảnh theo thời gian thực. Với khả năng hỗ trợ đa nền tảng, SDL2 cho phép chương trình có thể làm việc với cửa sổ hiển thị, xử lý sự kiện và cập nhật hình ảnh mà không cần phụ thuộc quá nhiều vào các thư viện đồ họa phức tạp của từng hệ điều hành.

Trong đề tài xây dựng trình giả lập CHIP-8, SDL2 được sử dụng để hỗ trợ phần giao tiếp giữa chương trình và người dùng. Bản thân lớp Chip8 chỉ đảm nhiệm việc mô phỏng hệ thống CHIP-8, xử lý opcode và cập nhật vùng dữ liệu hiển thị. Tuy nhiên, để người dùng có thể quan sát kết quả xử lý trên màn hình máy tính, chương trình cần một thành phần trung gian để tạo cửa sổ, render hình ảnh và nhận thao tác bàn phím. Đây chính là vai trò của SDL2 trong project.

Việc sử dụng SDL2 giúp chương trình hiển thị màn hình CHIP-8 một cách trực quan hơn. Do màn hình gốc của CHIP-8 chỉ có độ phân giải 64×32 pixel, nếu hiển thị đúng kích thước thật thì hình ảnh sẽ rất nhỏ và khó quan sát trên máy tính hiện đại. Vì

vậy, chương trình cần phóng đại dữ liệu hiển thị lên theo một tỷ lệ nhất định trước khi đưa ra cửa sổ. SDL2 hỗ trợ quá trình này thông qua các thành phần như Window, Renderer và Texture, giúp việc cập nhật hình ảnh lên màn hình trở nên thuận tiện.

Bên cạnh chức năng hiển thị, SDL2 còn được sử dụng để xử lý sự kiện bàn phím. CHIP-8 có hệ thống bàn phím gồm 16 phím, trong khi bàn phím máy tính hiện đại có bố cục khác hoàn toàn. Vì vậy, trình giả lập cần ánh xạ các phím trên bàn phím máy tính sang các phím tương ứng của CHIP-8. Thông qua cơ chế xử lý sự kiện của SDL2, chương trình có thể nhận biết khi người dùng nhấn hoặc thả phím, sau đó cập nhật trạng thái bàn phím bên trong Emulator.

Trong project của nhóm, các chức năng làm việc với SDL2 được tổ chức trong lớp Platform. Lớp này chịu trách nhiệm khởi tạo SDL2, tạo cửa sổ hiển thị, cập nhật dữ liệu hình ảnh và xử lý sự kiện bàn phím. Việc tách riêng phần SDL2 vào lớp Platform giúp chương trình có cấu trúc rõ ràng hơn, tránh việc trộn lẫn giữa phần lõi giả lập và phần hiển thị. Nhờ đó, lớp Chip8 chỉ tập trung vào việc mô phỏng hệ thống CHIP-8, còn lớp Platform đảm nhiệm việc giao tiếp với hệ điều hành và thiết bị nhập xuất.

Có thể thấy, SDL2 giữ vai trò quan trọng trong việc hoàn thiện trình giả lập CHIP-8 ở mặt giao diện và tương tác. Nếu không có SDL2 hoặc một thư viện tương tự, chương trình vẫn có thể xử lý opcode bên trong nhưng người dùng sẽ không thể quan sát kết quả hiển thị hoặc điều khiển trò chơi một cách trực quan. Vì vậy, việc sử dụng SDL2 là lựa chọn phù hợp với đề tài, giúp chương trình vừa đảm bảo phần xử lý giả lập, vừa có khả năng hiển thị hình ảnh và nhận dữ liệu nhập từ người dùng.

2.6.1. Tạo cửa sổ hiển thị

Trong chương trình giả lập CHIP-8, cửa sổ hiển thị là thành phần dùng để trình bày kết quả xử lý hình ảnh của Emulator ra màn hình máy tính. Bản thân hệ thống CHIP-8 chỉ có màn hình đơn sắc với độ phân giải 64×32 pixel, tuy nhiên dữ liệu này cần được hiển thị thông qua một cửa sổ đồ họa trên hệ điều hành hiện đại để người dùng có thể quan sát và tương tác với chương trình. Vì vậy, việc tạo cửa sổ hiển thị là bước cần thiết để kết nối phần lõi xử lý của trình giả lập với giao diện trực quan bên ngoài.

SDL2 cung cấp các chức năng hỗ trợ tạo cửa sổ chương trình một cách thuận tiện. Khi chương trình bắt đầu chạy, thư viện SDL2 cần được khởi tạo trước để chuẩn bị các thành phần liên quan đến đồ họa và sự kiện. Sau đó, chương trình tiến hành tạo

một cửa sổ hiển thị với tiêu đề, vị trí và kích thước phù hợp. Đối với trình giả lập CHIP-8, kích thước cửa sổ không nhất thiết bằng đúng độ phân giải gốc 64×32 pixel, vì nếu hiển thị ở kích thước thật thì hình ảnh sẽ rất nhỏ. Do đó, chương trình thường sử dụng một hệ số phóng đại để nhân kích thước màn hình CHIP-8 lên nhiều lần, giúp người dùng dễ quan sát hơn.

Trong project của nhóm, phần tạo cửa sổ được xử lý trong lớp Platform. Khi đối tượng Platform được khởi tạo, chương trình thiết lập cửa sổ hiển thị cho Emulator. Tiêu đề cửa sổ có thể chứa tên chương trình hoặc tên ROM đang được chạy, giúp người dùng biết trò chơi nào đang được thực thi. Kích thước cửa sổ được tính dựa trên độ phân giải gốc của CHIP-8 kết hợp với hệ số phóng đại do chương trình quy định. Nhờ đó, màn hình 64×32 pixel của CHIP-8 có thể được hiển thị với kích thước lớn hơn trên máy tính hiện đại.

Sau khi cửa sổ được tạo, chương trình cần thêm các thành phần hỗ trợ render hình ảnh. SDL2 thường sử dụng Renderer để thực hiện việc vẽ dữ liệu lên cửa sổ. Renderer đóng vai trò trung gian giữa dữ liệu hình ảnh của chương trình và cửa sổ hiển thị. Thông qua Renderer, chương trình có thể xóa màn hình cũ, vẽ dữ liệu mới và cập nhật kết quả hiển thị cho người dùng. Đây là cơ chế cần thiết đối với trình giả lập vì hình ảnh trong trò chơi có thể thay đổi liên tục sau mỗi chu kỳ xử lý opcode.

Bên cạnh Renderer, chương trình còn sử dụng Texture để lưu dữ liệu hình ảnh trước khi đưa lên cửa sổ. Trong CHIP-8 Emulator, dữ liệu màn hình được lưu trong vùng đệm video của hệ thống CHIP-8. Khi cần hiển thị, dữ liệu này sẽ được chuyển sang dạng phù hợp để cập nhật vào Texture. Sau đó, Texture được Renderer đưa lên cửa sổ chương trình. Cách làm này giúp quá trình cập nhật hình ảnh diễn ra hiệu quả hơn, đồng thời phù hợp với mô hình xử lý đồ họa của SDL2.

Việc tạo cửa sổ hiển thị không chỉ đơn thuần là mở một khung giao diện trên màn hình mà còn là bước chuẩn bị môi trường đồ họa cho toàn bộ chương trình. Nếu cửa sổ hoặc Renderer không được khởi tạo đúng, Emulator sẽ không thể hiển thị kết quả xử lý của ROM. Người dùng khi đó có thể vẫn chạy được chương trình ở phần lõi, nhưng không thể quan sát được hình ảnh trò chơi. Vì vậy, phần tạo cửa sổ hiển thị có vai trò quan trọng trong việc đảm bảo chương trình hoạt động trực quan và dễ sử dụng.

Ngoài ra, việc đặt phần tạo cửa sổ trong lớp Platform giúp chương trình có cấu trúc rõ ràng hơn. Lớp Chip8 không cần quan tâm đến cách SDL2 tạo cửa sổ hay render hình ảnh, mà chỉ cần cập nhật dữ liệu màn hình bên trong hệ thống giả lập. Lớp Platform sẽ đảm nhiệm việc lấy dữ liệu đó và hiển thị ra màn hình máy tính. Cách phân chia này phù hợp với nguyên tắc lập trình hướng đối tượng, trong đó mỗi lớp đảm nhận một nhóm chức năng riêng biệt.

Có thể thấy, tạo cửa sổ hiển thị là bước quan trọng trong quá trình xây dựng trình giả lập CHIP-8. Thông qua SDL2, chương trình có thể tạo ra một cửa sổ đồ họa, phóng đại màn hình gốc của CHIP-8 và chuẩn bị các thành phần cần thiết để cập nhật hình ảnh trong quá trình chạy ROM. Đây là cơ sở để các phần tiếp theo như xử lý sự kiện bàn phím và cập nhật hình ảnh lên màn hình hoạt động hiệu quả.

2.6.2. Ánh xạ và xử lý bàn phím

Trong trình giả lập CHIP-8, bàn phím là thành phần giúp người dùng tương tác trực tiếp với chương trình đang được thực thi. Đối với các ROM trò chơi, thao tác bàn phím được sử dụng để điều khiển nhân vật, bắt đầu trò chơi, di chuyển đối tượng, thực hiện hành động hoặc thay đổi trạng thái của chương trình. Vì vậy, bên cạnh việc hiển thị hình ảnh, trình giả lập cần có cơ chế ánh xạ và xử lý bàn phím một cách chính xác.

CHIP-8 sử dụng hệ thống bàn phím gồm 16 phím, được biểu diễn bằng các giá trị hệ thập lục phân từ 0 đến F. Tuy nhiên, bàn phím máy tính hiện đại không có bố cục giống với bàn phím CHIP-8. Do đó, khi xây dựng Emulator, chương trình cần ánh xạ các phím vật lý trên bàn phím máy tính sang các phím tương ứng của CHIP-8. Việc ánh xạ này giúp ROM vẫn nhận được dữ liệu nhập đúng theo cấu trúc bàn phím gốc, mặc dù người dùng đang thao tác trên máy tính hiện đại.

SDL2 hỗ trợ xử lý bàn phím thông qua cơ chế sự kiện. Khi người dùng nhấn hoặc thả một phím, SDL2 sẽ tạo ra sự kiện tương ứng. Chương trình liên tục kiểm tra các sự kiện này trong vòng lặp chính để xác định thao tác của người dùng. Nếu sự kiện là nhấn phím, trạng thái của phím tương ứng trong hệ thống CHIP-8 sẽ được cập nhật thành đang được nhấn. Nếu sự kiện là thả phím, trạng thái đó sẽ được đưa về không được nhấn. Nhờ cơ chế này, Emulator có thể theo dõi trạng thái bàn phím trong suốt quá trình ROM hoạt động.

Trong project của nhóm, phần xử lý bàn phím được đặt trong lớp Platform. Lớp này tiếp nhận các sự kiện từ SDL2, sau đó ánh xạ phím trên bàn phím máy tính sang phím tương ứng trong hệ thống CHIP-8. Trạng thái bàn phím sau khi được cập nhật sẽ được lớp Chip8 sử dụng trong quá trình xử lý các opcode liên quan đến nhập liệu. Nhờ đó, khi ROM thực hiện các lệnh kiểm tra bàn phím, chương trình có thể xác định chính xác phím nào đang được người dùng nhấn hoặc thả.

Việc xử lý bàn phím cần được thực hiện liên tục trong suốt quá trình Emulator hoạt động. Trong mỗi vòng lặp chính, chương trình không chỉ xử lý opcode và cập nhật màn hình mà còn phải kiểm tra các sự kiện nhập liệu từ người dùng. Nếu quá trình kiểm tra bàn phím bị chậm hoặc không chính xác, trò chơi có thể phản hồi sai, không nhận phím hoặc nhận phím trễ. Điều này ảnh hưởng trực tiếp đến trải nghiệm của người dùng, đặc biệt là với các trò chơi yêu cầu phản xạ nhanh như Space Invaders hoặc Dodge.

Một điểm quan trọng trong xử lý bàn phím là phân biệt giữa trạng thái nhấn phím và thả phím. Nếu chỉ kiểm tra thời điểm nhấn phím mà không cập nhật khi phím được thả, chương trình có thể hiểu nhầm rằng phím đó vẫn đang được giữ. Ngược lại, nếu không ghi nhận đúng sự kiện nhấn phím, ROM có thể không phản hồi với thao tác của người chơi. Vì vậy, trình giả lập cần duy trì một mảng trạng thái bàn phím, trong đó mỗi phần tử tương ứng với một phím của CHIP-8.

Ngoài các phím điều khiển trò chơi, chương trình cũng cần xử lý một số phím đặc biệt phục vụ cho quá trình vận hành Emulator. Ví dụ, phím thoát chương trình có thể được sử dụng để đóng cửa sổ và kết thúc vòng lặp chính. Điều này giúp người dùng có thể dừng chương trình một cách thuận tiện thay vì phải tắt cửa sổ bằng các thao tác bên ngoài.

Có thể thấy, ánh xạ và xử lý bàn phím là một phần không thể thiếu trong trình giả lập CHIP-8. Thông qua SDL2, chương trình có thể nhận biết thao tác của người dùng, chuyển đổi phím máy tính sang hệ phím CHIP-8 và cập nhật trạng thái nhập liệu cho Emulator. Việc xử lý bàn phím chính xác giúp các ROM trò chơi phản hồi đúng với thao tác điều khiển, từ đó tạo ra trải nghiệm tương tác ổn định và gần giống với hệ thống CHIP-8 gốc.

2.6.3. Cập nhật hình ảnh lên màn hình

Trong trình giả lập CHIP-8, việc cập nhật hình ảnh lên màn hình là bước giúp người dùng quan sát được kết quả xử lý của chương trình. Sau khi ROM được nạp và các opcode được thực thi, trạng thái bên trong của hệ thống CHIP-8 sẽ liên tục thay đổi. Những thay đổi liên quan đến đồ họa sẽ được lưu vào vùng dữ liệu hiển thị của Emulator. Tuy nhiên, để người dùng có thể nhìn thấy hình ảnh trên màn hình máy tính, dữ liệu này cần được chuyển sang cửa sổ hiển thị thông qua thư viện SDL2.

Màn hình gốc của CHIP-8 có độ phân giải 64×32 pixel và chỉ sử dụng hai trạng thái điểm ảnh là bật hoặc tắt. Khi một opcode đồ họa được thực thi, chương trình sẽ cập nhật trạng thái của các điểm ảnh trong vùng đệm hiển thị. Vùng đệm này có thể được xem như bản sao trạng thái màn hình của CHIP-8 bên trong bộ nhớ chương trình. Mỗi phần tử trong vùng đệm đại diện cho một điểm ảnh trên màn hình CHIP-8. Khi điểm ảnh thay đổi, dữ liệu trong vùng đệm cũng được cập nhật tương ứng.

Trong project của nhóm, phần lõi xử lý đồ họa được thực hiện trong lớp Chip8. Khi opcode vẽ sprite được thực thi, lớp Chip8 sẽ cập nhật dữ liệu video dựa trên nội dung sprite được lấy từ bộ nhớ. Sau đó, dữ liệu video này được lớp Platform sử dụng để hiển thị lên cửa sổ chương trình. Cách tổ chức này giúp tách biệt rõ ràng giữa phần xử lý logic đồ họa của CHIP-8 và phần hiển thị hình ảnh bằng SDL2.

SDL2 hỗ trợ quá trình cập nhật hình ảnh thông qua các thành phần như Texture và Renderer. Texture được dùng để lưu dữ liệu hình ảnh cần hiển thị, còn Renderer đảm nhiệm việc đưa dữ liệu đó lên cửa sổ chương trình. Khi dữ liệu video của CHIP-8 thay đổi, chương trình sẽ cập nhật lại Texture, sau đó sử dụng Renderer để vẽ Texture lên cửa sổ. Quá trình này được lặp lại liên tục trong vòng lặp chính của Emulator, giúp hình ảnh trên màn hình luôn phản ánh trạng thái mới nhất của chương trình đang chạy.

Do độ phân giải gốc của CHIP-8 rất nhỏ, chương trình cần phóng đại hình ảnh trước khi hiển thị để người dùng dễ quan sát. Thay vì hiển thị trực tiếp màn hình 64×32 pixel với kích thước thật, Emulator sử dụng hệ số phóng đại để mở rộng kích thước cửa sổ. Nhờ đó, các trò chơi CHIP-8 có thể được hiển thị rõ ràng hơn trên màn hình máy tính hiện đại mà vẫn giữ nguyên logic hiển thị ban đầu của hệ thống.

Việc cập nhật hình ảnh cần được thực hiện đồng bộ với quá trình xử lý opcode. Trong mỗi vòng lặp chính, chương trình sẽ kiểm tra sự kiện bàn phím, thực hiện chu kỳ

xử lý lệnh của CHIP-8 và cập nhật lại màn hình khi cần thiết. Nếu quá trình cập nhật hình ảnh không được thực hiện đúng, người dùng có thể gặp tình trạng màn hình không thay đổi, hình ảnh bị sai, bị nhấp nháy hoặc không phản ánh đúng trạng thái của ROM. Vì vậy, việc cập nhật hình ảnh là một phần quan trọng trong quá trình vận hành của trình giả lập.

Ngoài ra, việc sử dụng SDL2 giúp chương trình đơn giản hóa quá trình render hình ảnh. Thay vì phải làm việc trực tiếp với các API đồ họa phức tạp của hệ điều hành, chương trình có thể sử dụng các hàm của SDL2 để cập nhật texture, xóa nội dung cũ trên cửa sổ, vẽ dữ liệu mới và hiển thị kết quả. Điều này giúp phần hiển thị của Emulator dễ cài đặt hơn, đồng thời vẫn đảm bảo đáp ứng được yêu cầu của một trình giả lập CHIP-8 cơ bản.

Có thể thấy, cập nhật hình ảnh lên màn hình là bước quan trọng giúp kết quả xử lý bên trong Emulator được thể hiện trực quan cho người dùng. Thông qua SDL2, dữ liệu video của CHIP-8 được chuyển thành hình ảnh hiển thị trên cửa sổ chương trình. Việc tổ chức phần cập nhật hình ảnh trong lớp Platform giúp chương trình có cấu trúc rõ ràng, tách biệt giữa lõi giả lập và giao diện hiển thị, đồng thời phù hợp với phương pháp lập trình hướng đối tượng được áp dụng trong đề tài.

CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

Phân tích cách hệ thống giả lập CHIP-8 được tổ chức trước khi đi vào giai đoạn cài đặt. Đối tượng cần xây dựng không phải là một trò chơi riêng lẻ, mà là một môi trường có khả năng đọc và thực thi các ROM CHIP-8. Vì vậy, hệ thống phải đồng thời mô phỏng trạng thái của một máy CHIP-8, tiếp nhận tương tác từ người dùng và hiển thị kết quả đồ họa trên máy tính hiện đại.

Về phía người dùng, đầu vào gồm tên ROM hoặc đường dẫn ROM, hai tham số tùy chọn là hệ số phóng to và độ trễ chu kỳ, cùng các thao tác bàn phím trong lúc chạy. Đầu ra quan sát được là cửa sổ SDL2 hiển thị khung hình 64 x 32 điểm ảnh đã được phóng to; ngoài ra chương trình còn in danh sách ROM, hướng dẫn sử dụng hoặc thông báo lỗi lên cửa sổ dòng lệnh khi cần thiết.

Về phía hệ thống, mọi dữ liệu cần được duy trì nhất quán giữa các chu kỳ lệnh. Bộ nhớ phải chứa fontset và ROM tại đúng vùng địa chỉ; thanh ghi, bộ đếm chương trình, stack, timer, bàn phím và vùng đệm video phải được cập nhật theo opcode hiện hành. Các yêu cầu này được chuyển thành các tiêu chí chức năng, tiêu chí chất lượng và các ràng buộc kỹ thuật trong những bảng dưới đây.

Bảng 1 Các yêu cầu chức năng của hệ thống

Mã	**Yêu cầu**	**Điều kiện đáp ứng**
FR-01	Chọn và tìm ROM	Nhận đường dẫn, tên tệp hoặc một phần tên game; tìm trong roms và ../roms.
FR-02	Liệt kê ROM	Khi nhận --list, chỉ in danh sách tệp .ch8 và kết thúc bình thường.
FR-03	Nạp ROM an toàn	Từ chối tệp không mở được, tệp rỗng hoặc ROM vượt vùng nhớ từ 0x200 đến 0xFFF.
FR-04	Giả lập chu kỳ lệnh	Fetch opcode 16 bit, giải mã qua bảng hàm, thực thi và cập nhật timer.

Mã	Yêu cầu	Điều kiện đáp ứng
FR-05	Hiển thị đồ họa	Cập nhật texture RGBA8888 từ video[64 x 32], render và present lên cửa sổ.
FR-06	Nhập và thoát	Ánh xạ sự kiện KEYDOWN/KEYUP sang keypad; nhận SDL_QUIT hoặc Escape để thoát.

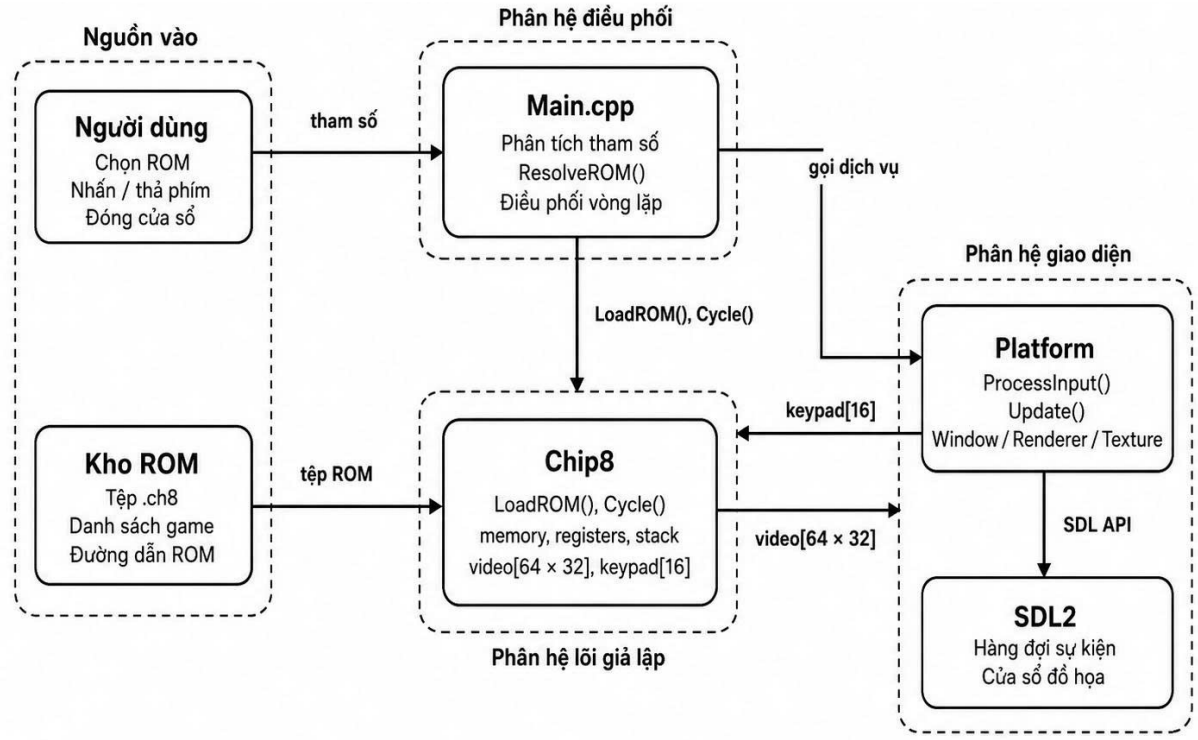
Bảng 2 Các yêu cầu chất lượng và ràng buộc kỹ thuật

Nhóm	Ràng buộc hoặc tiêu chí	Ý nghĩa thiết kế
Tính đúng	Bộ nhớ 4096 byte, 16 thanh ghi, stack 16 mức, video 64 x 32, keypad 16 phím.	Bám sát cấu trúc dữ liệu được khai báo trong Chip8.hpp.
Khả năng bảo trì	Tách lớp Chip8, Platform và phần điều phối Main.cpp.	Giảm phụ thuộc giữa mô phỏng, SDL2 và xử lý tham số.
Tính ổn định	Kiểm tra kết quả mở tệp và kích thước ROM trước khi sao chép.	Ngăn ghi dữ liệu ROM ra ngoài vùng bộ nhớ giả lập.
Khả năng sử dụng	Có cú pháp gọi rõ ràng; hỗ trợ tên game ngắn và thông báo hướng dẫn.	Giảm thao tác gõ đường dẫn dài khi chạy thử ROM.
Môi trường	C++17, CMake và SDL2; texture dùng SDL_PIXELFORMAT_RGBA8888.	Đảm bảo cấu trúc build và giao diện đồ họa phù hợp với dự án.

Phạm vi chức năng được xác định theo mã nguồn hiện có. Hệ thống hỗ trợ các opcode đã được cài đặt trong lớp Chip8, bộ sinh số ngẫu nhiên cho Cxkk, cơ chế vẽ sprite bằng XOR, bàn phím và hai timer. Tuy nhiên, soundTimer hiện mới là trạng thái logic và chưa có thành phần âm thanh SDL2; báo cáo vì thế chỉ mô tả âm thanh ở mức timer, không xem việc phát loa là một chức năng đã hoàn thành.

3.2. Thiết kế kiến trúc hệ thống

Kiến trúc được xây dựng theo hướng phân tách trách nhiệm. Main.cpp không tự mô phỏng opcode và cũng không thao tác trực tiếp lên từng pixel; nó chỉ thiết lập cấu hình, chọn ROM và điều phối trình tự chạy. Chip8 không phụ thuộc SDL2 nên có thể được kiểm tra như một lõi mô phỏng độc lập. Platform không biết chi tiết về opcode mà chỉ nhận bộ đệm video và trạng thái phím cần chuyển đổi.



Hình 2 Sơ đồ kiến trúc thành phần và chiều trao đổi dữ liệu của hệ thống

Mũi tên từ người dùng đến Main.cpp biểu diễn tham số khởi động; mũi tên từ kho ROM đến Chip8 biểu diễn dữ liệu nhị phân cần nạp. Sau khi chạy, Platform ghi trạng thái phím vào keypad của Chip8, trong khi Chip8 xuất vùng đệm video cho Platform. Nhờ hai luồng trao đổi này, dữ liệu đầu vào và kết quả đồ họa được tách bạch, tránh việc SDL2 can thiệp trực tiếp vào memory hoặc registers.

Bảng 3 Vai trò và giao diện giữa các thành phần chính

Thành phần	Nhận vào	Cung cấp ra	Trách nhiệm
Main.cpp	argc, argv, thời gian hiện tại	Cấu hình, lời gọi Cycle/Update	Điều phối toàn bộ tiến trình chạy.
Chip8	ROM, keypad, opcode hiện hành	video, trạng thái CPU	Mô phỏng bộ nhớ và thực thi lệnh CHIP-8.
Platform	video buffer, pitch, sự kiện SDL	trạng thái thoát, keypad	Bao bọc SDL2 cho hiển thị và bàn phím.
SDL2	Lời gọi từ Platform	sự kiện và khung hình	Cung cấp cửa sổ, renderer, texture và event queue.

3.2.1. Thành phần xử lý CHIP-8

Thành phần xử lý CHIP-8 tập trung toàn bộ trạng thái cần có trong quá trình mô phỏng. Constructor đặt pc về START_ADDRESS là 0x200, sao chép 80 byte fontset vào địa chỉ 0x50, khởi tạo bộ sinh số ngẫu nhiên và gán các phần tử của bảng con trỏ hàm. Nhờ đó, đối tượng đã sẵn sàng nạp ROM và thực thi opcode ngay sau khi được tạo.

Trong thiết kế này, public interface được giữ nhỏ: LoadROM() trả về kết quả thành công hay thất bại, Cycle() chạy một opcode, còn keypad và video là hai vùng dữ liệu trao đổi với Platform. Các thuộc tính như memory, registers, index, pc, stack và timer đều là private để chỉ các hàm xử lý opcode có thể thay đổi chúng. Đây là cách áp dụng đóng gói phù hợp với bài toán giả lập, vì mọi chuyển trạng thái phải đi qua cơ chế opcode xác định.

Bảng opcode là quyết định then chốt đối với khả năng mở rộng. Bảng chính nhận nibble cao, còn các nhóm nhiều hậu tố như 0x0, 0x8, 0xE và 0xF được chuyển đến bảng phụ. Nếu sau này bổ sung opcode, lập trình viên chỉ cần cài đặt hàm OP_xxx và gán nó vào đúng vị trí trong bảng, thay vì thay đổi cấu trúc giải mã của toàn bộ chương trình.

3.2.2. Thành phần hiển thị và nhập liệu

Platform được thiết kế như lớp thích nghi giữa mô hình video đơn giản của CHIP-8 và SDL2. Chip8 lưu mỗi pixel dưới dạng uint32_t, trong đó pixel bật có giá trị 0xFFFFFFFF và pixel tắt có giá trị 0. Platform dùng texture streaming RGBA8888 với kích thước texture đúng bằng VIDEO_WIDTH x VIDEO_HEIGHT; SDL_RenderCopy() tự co giãn texture lên cửa sổ có kích thước lớn hơn theo videoScale.

Đối với nhập liệu, ProcessInput() không trả trực tiếp phím nào cho opcode. Hàm nhận một con trỏ đến mảng keys, thay đổi phần tử tương ứng khi nhận SDL_KEYDOWN hoặc SDL_KEYUP và trả về duy nhất cờ quit. Cách tổ chức này bảo đảm các opcode Ex9E, ExA1 và Fx0A có thể đọc keypad tại thời điểm Cycle() chạy mà không phụ thuộc vào kiểu sự kiện của SDL2.

Khi việc khởi tạo SDL_Init, SDL_CreateWindow, SDL_CreateRenderer hoặc SDL_CreateTexture thất bại, constructor ném std::runtime_error với nội dung SDL_GetError(). Nhánh renderer ưu tiên SDL_RENDERER_ACCELERATED và có phương án dự phòng SDL_RENDERER_SOFTWARE. Do đó, Platform vừa đóng vai trò hiển thị vừa là biên xử lý lỗi tài nguyên đồ họa của chương trình.

3.2.3. Thành phần điều khiển chương trình chính

Phần điều khiển chính thực hiện ba việc: phân tích tham số, chuẩn hóa tên ROM và vận hành vòng lặp. Chương trình nhận một đối số ROM hoặc ba đối số scale, delay, ROM. Khi nhận --list, ListROM() duyệt các thư mục xác định, lọc tệp có phần mở rộng .ch8, sắp xếp tên và in ra kết quả. Các trường hợp cú pháp khác đều gọi PrintUsage() để thông báo cách sử dụng.

ResolveROM() cho phép sử dụng các bí danh IBM, breakout và dodge; sau đó thử đường dẫn gốc, các đường dẫn trong roms, ../roms và biến thể có đuôi .ch8. Khi vẫn chưa tìm thấy, FindROMMatches() chuẩn hóa tên bằng NormalizeName(), so khớp chính xác trước rồi mới so khớp một phần. Quy trình này nâng cao trải nghiệm sử dụng mà không làm thay đổi phần nạp ROM của lớp Chip8.

Sau khi nạp ROM thành công, Main.cpp tạo Platform, tính videoPitch bằng sizeof(chip8.video[0]) * VIDEO_WIDTH và lưu lastCycleTime. Vòng lặp gọi ProcessInput(), tính dt bằng mili giây, chỉ gọi Cycle() và Update() khi dt lớn hơn

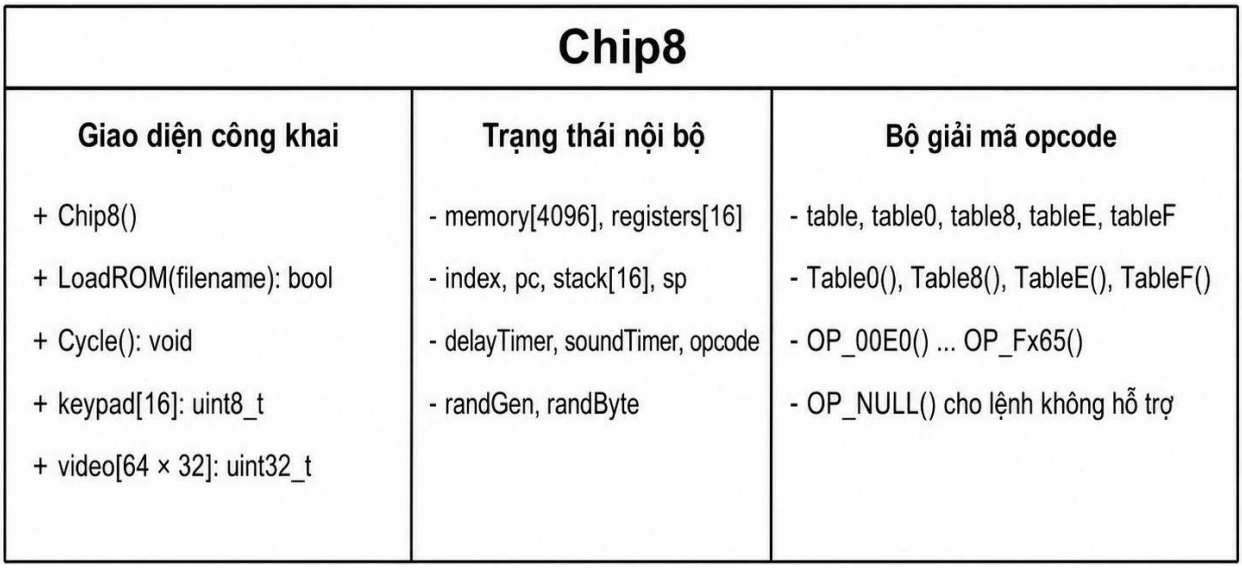
cycleDelay. Chính Main.cpp quyết định tần suất gọi Chip8::Cycle(), còn Chip8 chỉ thực hiện đúng một chu kỳ tại mỗi lần được gọi.

3.3. Thiết kế lớp Chip8

Lớp Chip8 là lõi nghiệp vụ của hệ thống. Thiết kế lớp không mô phỏng phần cứng ở mức điện tử mà mô phỏng trạng thái và tác động của tập lệnh: khi một opcode được thực thi, lớp đọc dữ liệu cần thiết từ bộ nhớ hoặc thanh ghi, cập nhật trạng thái rồi phản ánh kết quả qua video, keypad, pc hoặc timer. Sơ đồ lớp bên dưới tóm tắt giao diện và ba nhóm dữ liệu nội bộ quan trọng.

Việc gom dữ liệu và hàm thực thi vào cùng một lớp giúp mô hình có tính nhất quán: opcode luôn thao tác trên trạng thái của đúng đối tượng đang chạy, còn dữ liệu không bị chia sẻ tùy tiện giữa những phần khác của chương trình. Khi cần kiểm thử, một đối tượng Chip8 có thể được tạo, nạp một ROM nhỏ và gọi Cycle() mà không cần khởi tạo cửa sổ SDL2.

Thiết kế này cũng làm rõ đường phụ thuộc của dự án: Platform phụ thuộc vào hai vùng trao đổi keypad và video, trong khi Chip8 không phụ thuộc ngược vào Platform. Main.cpp là nơi liên kết hai đối tượng, vì vậy việc mở rộng giao diện hoặc thay đổi cách gọi chương trình không buộc phải sửa thuật toán xử lý opcode.



Hình 3 Sơ đồ lớp Chip8 với giao diện công khai, trạng thái nội bộ và cơ chế

3.3.1. Thuộc tính của lớp

Các thuộc tính được chia thành vùng giao tiếp và vùng trạng thái nội bộ. keypad và video là dữ liệu public vì Platform cần truy cập nhanh ở mỗi khung hình. Việc để hai mảng này công khai không làm lộ cơ chế giải mã opcode, bởi phần còn lại của trạng thái CPU vẫn được bảo vệ trong private. Kích thước các mảng sử dụng hằng số KEY_COUNT, MEMORY_SIZE, REGISTER_COUNT, STACK_LEVELS, VIDEO_WIDTH và VIDEO_HEIGHT để tránh số ma thuật trong mã nguồn.

Bảng 4 Thuộc tính và ý nghĩa mô phỏng trong lớp Chip8

Nhóm	Thuộc tính	Vai trò
Giao tiếp	keypad[KEY_COUNT]	Nhận 0 hoặc 1 cho từng phím CHIP-8 từ lớp Platform.
Giao tiếp	video[VIDEO_WIDTH * VIDEO_HEIGHT]	Giữ ảnh RGBA để Platform cập nhật texture SDL2.
Bộ nhớ	memory[MEMORY_SIZE]	Chứa fontset, ROM, opcode, sprite và vùng dữ liệu làm việc.
Thanh ghi	registers[REGISTER_COUNT], index, pc	Lưu V0–VF, địa chỉ chỉ mục và vị trí lệnh tiếp theo.
Ngăn xếp	stack[STACK_LEVELS], sp	Lưu địa chỉ quay về của chương trình con.
Timer	delayTimer, soundTimer	Lưu trạng thái định thời được giảm sau Cycle().
Điều phối	opcode, table, table0, table8, tableE, tableF	Giữ opcode hiện tại và các bảng con trỏ hàm giải mã.

Ở đây VF là phần tử registers[0xF] nhưng có vai trò cờ trong nhiều opcode. OP_8xy4 ghi VF bằng 1 khi phép cộng tràn quá 255; OP_8xy5 và OP_8xy7 phản ánh

điều kiện mượn; OP_8xy6 và OP_8xyE lưu bit thấp hoặc bit cao trước khi dịch; OP_Dxyn dùng VF để báo va chạm pixel. Vì cờ được đặt trong cùng mảng thanh ghi, cách biểu diễn này bám sát quy ước CHIP-8 và giảm số thuộc tính phụ cần duy trì.

3.3.2. Phương thức tải ROM

LoadROM() mở tệp bằng std::ifstream ở chế độ binary và ate. Chế độ ate đưa con trỏ đọc tới cuối tệp để hàm lấy được kích thước bằng tellg(); sau đó hàm quay về đầu bằng seekg(0, std::ios::beg) để đọc nội dung. Kết quả mở tệp được kiểm tra đầu tiên, vì nếu file không hợp lệ thì việc lấy kích thước hoặc đọc dữ liệu không còn ý nghĩa.

Sau bước xác định kích thước, điều kiện size <= 0 hoặc START_ADDRESS + size > MEMORY_SIZE được dùng để từ chối ROM rỗng hoặc ROM quá lớn. START_ADDRESS là 0x200 và MEMORY_SIZE là 4096, do đó ROM chỉ được phép chiếm vùng từ 0x200 đến hết bộ nhớ. Đây là kiểm tra trước khi sao chép, giúp không ghi đè ra ngoài mảng memory.

Khi hợp lệ, hàm cấp phát buffer theo size, đọc toàn bộ tệp, lặp từng byte để gán memory[START_ADDRESS + i] và giải phóng buffer trước khi trả về true. Main.cpp dựa vào giá trị trả về để quyết định có tạo Platform hay không. Như vậy lỗi nạp ROM được phát hiện sớm, không đưa chương trình vào vòng lặp với trạng thái bộ nhớ không hợp lệ.

Bảng 5 Các quyết định trong quy trình tải ROM

Điểm kiểm tra	Điều kiện	Xử lý
Mở tệp	file.is_open() là false	In lỗi Failed to open ROM và trả về false.
Kích thước	size <= 0	Từ chối ROM rỗng vì không có opcode để thực thi.
Giới hạn bộ nhớ	0x200 + size > 4096	Từ chối ROM vượt dung lượng bộ nhớ giả lập.
Đọc dữ liệu	Các điều kiện trên đều hợp lệ	Đọc binary vào buffer và sao chép vào memory[0x200 + i].
Báo kết quả	Sao chép hoàn tất	Giải phóng buffer và trả về true cho Main.cpp.

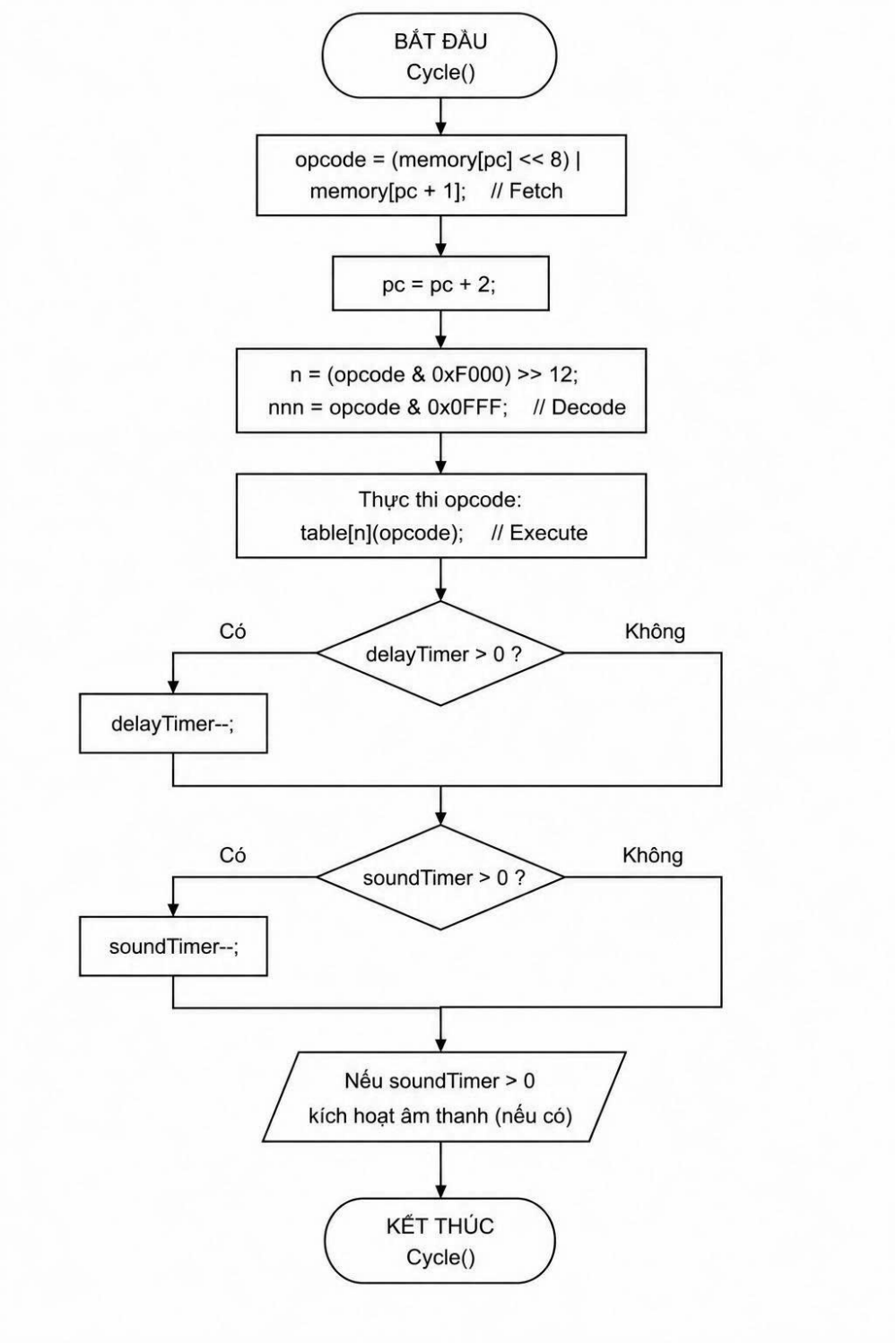
3.3.3. Phương thức xử lý chu kỳ lệnh

Cycle() là phương thức trung tâm của Chip8. Bước fetch ghép hai byte liên tiếp theo công thức (memory[pc] << 8u) | memory[pc + 1]. Vì opcode CHIP-8 dài 16 bit, cách ghép này bảo đảm byte ở địa chỉ pc là byte cao và byte ở pc + 1 là byte thấp. Ngay sau fetch, pc được tăng 2 để trạng thái mặc định luôn trỏ đến lệnh kế tiếp.

Bước decode và execute sử dụng biểu thức (opcode & 0xF000u) >> 12u để lấy nibble cao, từ đó truy xuất table. Phần tử trong table là con trỏ đến hàm thành viên, nên lời gọi ((*this).*(table[index]))() thực thi đúng opcode trên đối tượng hiện tại. Các opcode làm thay đổi luồng, như OP_1nnn, OP_2nnn, OP_00EE hoặc các lệnh skip, sẽ gán lại pc hoặc tăng thêm pc theo quy tắc riêng.

Sau opcode, Cycle() giảm delayTimer và soundTimer nếu giá trị còn dương. Trong phiên bản hiện tại, timer giảm theo số chu kỳ mô phỏng thay vì một bộ điều khiển 60 Hz độc lập; tần số này do điều kiện dt > cycleDelay trong Main.cpp tác động. Báo

cáo nêu rõ thiết kế thực tế để tránh suy diễn rằng timer đã được tách thời gian chuẩn 60 Hz.



Hình 4 Lưu đồ fetch - decode - execute và cập nhật timer của Chip8::Cycle()

Bảng 6 Thuật toán xử lý một chu kỳ lệnh CHIP-8

Bước	Mô tả thuật toán
1	Đọc memory[pc] và memory[pc + 1], ghép thành opcode 16 bit.
2	Tăng pc thêm 2 để chuẩn bị cho lệnh liên tiếp.
3	Lấy nibble cao của opcode, tra table hoặc gọi bảng phụ phù hợp.
4	Thực thi hàm OP_xxx; hàm có thể thay đổi registers, memory, video, stack hoặc pc.
5	Nếu delayTimer hoặc soundTimer lớn hơn 0, giảm từng timer một đơn vị.
6	Kết thúc chu kỳ và trao quyền điều khiển lại cho vòng lặp Main.cpp.

3.3.4. Các hàm xử lý opcode

Các hàm OP_xxx được đặt tên theo mẫu opcode để liên hệ trực tiếp giữa đặc tả và mã nguồn. Ví dụ, OP_00E0() gọi memset để xóa video, OP_1nnn() gán pc bằng địa chỉ nnn, OP_2nnn() đẩy pc hiện tại lên stack trước khi nhảy, và OP_Dxyn() đọc sprite từ memory[index + row] để thay đổi pixel theo phép XOR. Những hàm này chỉ xử lý một loại lệnh, nhờ đó chức năng dễ kiểm tra và dễ đối chiếu.

Nhóm 8xy* có nhiều thao tác số học và logic nên được Table8() điều phối bằng nibble thấp. Các hàm cộng/trừ dùng VF làm cờ, còn hai hàm dịch lưu bit cần thiết vào VF trước khi thay đổi Vx. Nhóm Fxyy chứa các thao tác với timer, fontset, BCD và sao chép dữ liệu; TableF() dùng tám bit thấp để lựa chọn đúng phần tử bảng phụ.

Bảng 7 Nhóm opcode đã tổ chức trong bảng giải mã

Nhóm	Điều phối	Tác động tiêu biểu
0x0	Table0()	Xóa màn hình 00E0; quay về từ chương trình con 00EE.
0x1–0x7	table trực tiếp	Nhảy, gọi, skip, gán Vx và cộng hằng số.
0x8	Table8()	Gán, OR, AND, XOR, cộng/trừ và dịch bit thanh ghi.
0x9–0xD	table trực tiếp	Skip khác, index, nhảy Bnnn, random Cxkk, vẽ Dxyn.
0xE	TableE()	Kiểm tra key pressed Ex9E và key not pressed ExA1.
0xF	TableF()	Timer, chờ phím, fontset, BCD, lưu/tải thanh ghi.

3.4. Thiết kế lớp Platform

Lớp Platform đóng vai trò adapter cho SDL2. Lớp không biết ROM nào đang chạy và cũng không giải mã opcode; nó nhận một buffer pixel cùng pitch để vẽ, nhận một con trỏ keys để cập nhật bàn phím và trả cờ thoát. Nhờ phạm vi trách nhiệm hẹp, các thay đổi về thư viện hiển thị trong tương lai sẽ tập trung trong Platform thay vì lan sang Chip8.

Constructor nhận title, windowWidth, windowHeight, textureWidth và textureHeight. Cửa số được đặt giữa màn hình bằng SDL_WINDOWPOS_CENTERED, renderer ưu tiên chế độ accelerated nhưng có nhánh dự phòng software, texture dùng SDL_TEXTUREACCESS_STREAMING để có thể cập nhật dữ liệu video nhiều lần. Các bước tạo tài nguyên được kiểm tra tuần tự và giải phóng phần đã tạo nếu một bước sau thất bại.

3.4.1. Thuộc tính của lớp

Bảng 8 Thuộc tính và vòng đời tài nguyên của Platform

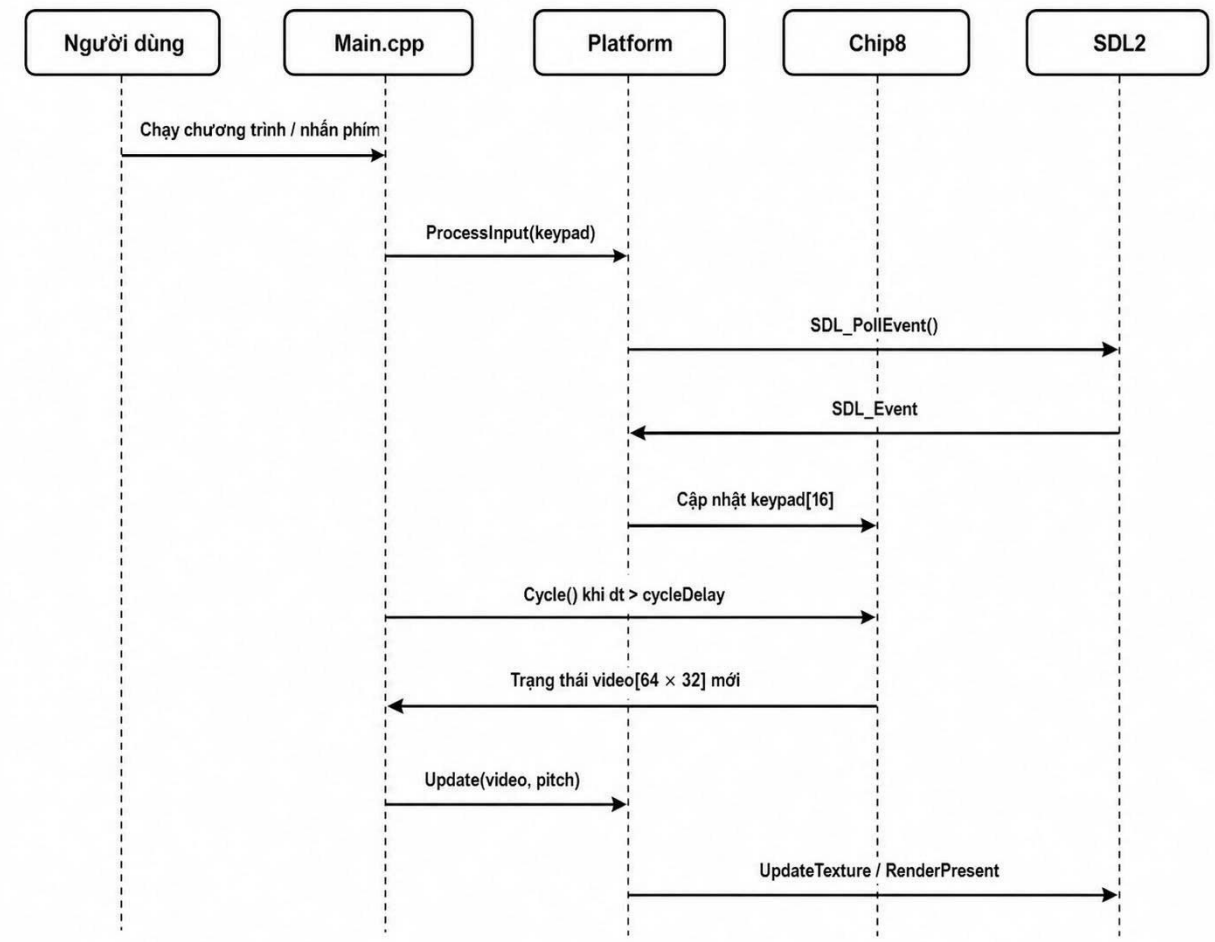
Thuộc tính	Tạo tại	Dùng cho	Giải phóng tại
window	SDL_CreateWindow	Hiển thị giao diện và nhận SDL_QUIT.	SDL_DestroyWindow
renderer	SDL_CreateRenderer	Xóa, sao chép texture, trình bày khung hình.	SDL_DestroyRenderer
texture	SDL_CreateTexture	Nhận buffer video RGBA8888 từ Chip8.	SDL_DestroyTexture

Thứ tự hủy trong destructor là texture, renderer, window, rồi SDL_Quit(). Trình tự ngược với quá trình tạo này giúp mỗi đối tượng SDL được giải phóng trong khi đối tượng phụ thuộc của nó vẫn còn hợp lệ. Khi chương trình trả về từ main() theo nhánh chạy bình thường, destructor tự động được gọi do Platform là biến cục bộ trong phạm vi hàm main().

3.4.2. Phương thức cập nhật màn hình

Update() thực hiện bốn thao tác theo đúng thứ tự khung hình: SDL_UpdateTexture() sao chép dữ liệu mới từ buffer vào texture; SDL_RenderClear() xóa nội dung renderer; SDL_RenderCopy() co giãn texture để lấp vùng vẽ; SDL_RenderPresent() đưa ảnh hoàn chỉnh lên màn hình. pitch được truyền từ Main.cpp, bằng số byte trong một hàng video, do đó SDL2 biết cách bước qua từng dòng dữ liệu nguồn.

Lõi Chip8 chỉ cần thay đổi video trong opcode Dxyn hoặc xóa nó trong 00E0, không cần biết cách điểm ảnh được render. Khi OP_Dxyn bật một bit sprite, mã nguồn lấy con trỏ tới video[wrappedY * VIDEO_WIDTH + wrappedX] và XOR với 0xFFFFFFFF. Platform sau đó nhận đúng mảng này để thể hiện pixel trắng/tắt theo trạng thái 32 bit đã có.



Hình 5 Sơ đồ tuần tự từ sự kiện người dùng đến khung hình SDL2

3.4.3. Phương thức xử lý bàn phím

ProcessInput() lặp đến khi SDL_PollEvent() không còn sự kiện. Nếu event.type là SDL_QUIT, biến quit trở thành true. Với SDL_KEYDOWN hoặc SDL_KEYUP, biến pressed được gán theo phép so sánh event.type == SDL_KEYDOWN; cùng một switch ánh xạ cả hai sự kiện nên keys[index] sẽ trở về 0 đúng lúc người dùng nhả phím.

Các phím 0x4 và 0x6 có thêm lựa chọn mũi tên trái/phải, A/Q và D/E để thuận tiện cho một số game. Bảng sau mô tả chính xác những case hiện có trong Platform.cpp. Việc nêu rõ 0x7 và 0x9 chưa có case tương ứng là cần thiết vì báo cáo phải phản ánh đúng mã nguồn hiện tại, không suy diễn toàn bộ 16 phím đã có ánh xạ vật lý.

Bảng 9 Ánh xạ bàn phím hiện có trong Platform::ProcessInput()

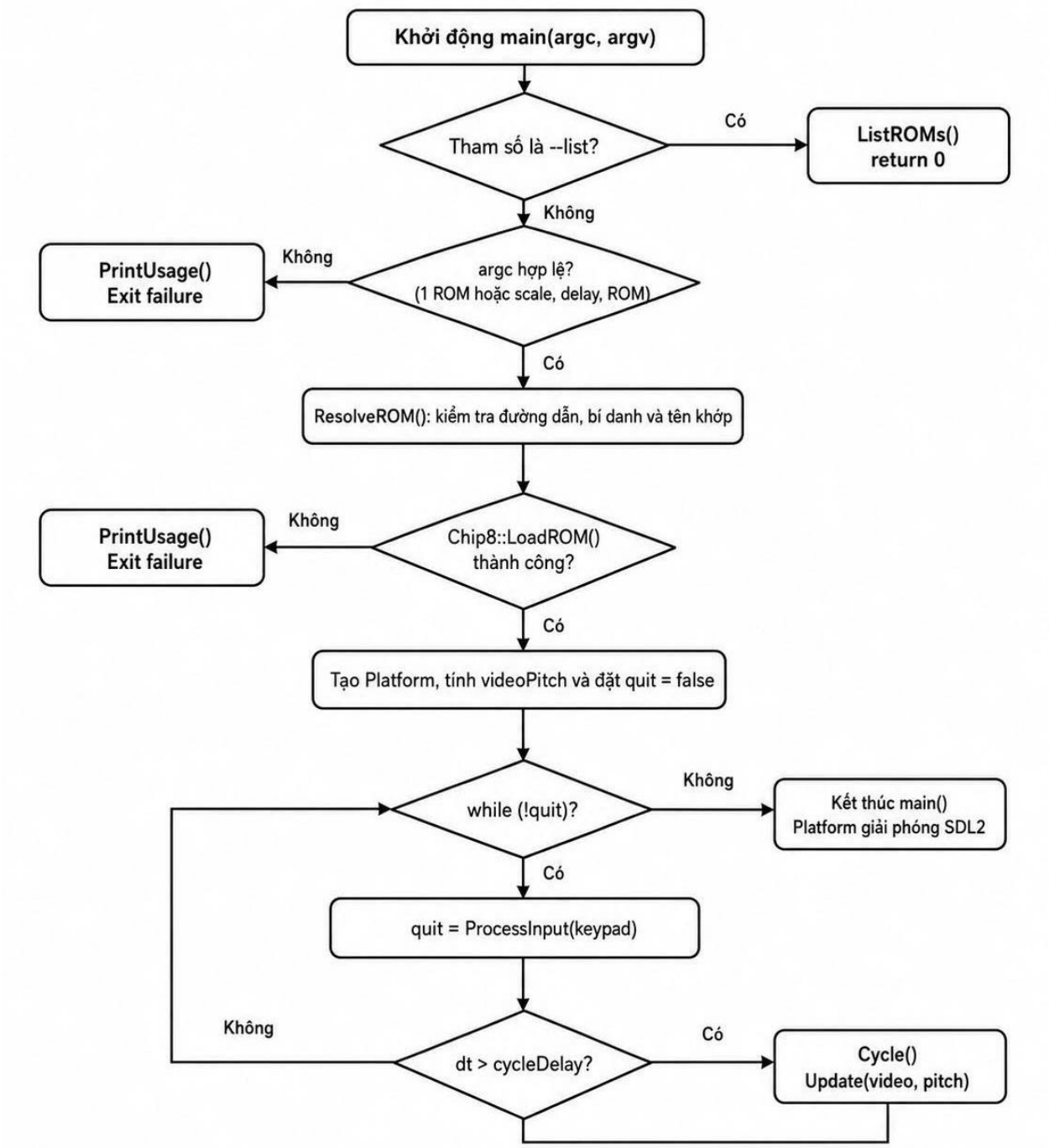
Phím CHIP-8	SDLK xử lý	Tác động
0x0	X	Ghi pressed vào keys[0x0].
0x1, 0x2, 0x3	1, 2, 3	Ghi vào ba phần tử đầu của keypad.
0x4	Q, A, ←	Ba phím vật lý cùng điều khiển keys[0x4].
0x5	W	Ghi pressed vào keys[0x5].
0x6	E, D, →	Ba phím vật lý cùng điều khiển keys[0x6].
0x8	S	Ghi pressed vào keys[0x8].
0xA, 0xB	Z, C	Ghi vào keys[0xA] và keys[0xB].
0xC–0xF	4, R, F, V	Ghi vào các keys 0xC, 0xD, 0xE, 0xF.
0x7, 0x9	Chưa gán	Không có case SDL tương ứng trong mã nguồn.



Hình 6 Lưu đồ tiếp nhận và ánh xạ sự kiện bàn phím của Platform::ProcessInput()

3.5. Thiết kế luồng hoạt động

Luồng hoạt động tổng quát kết nối các thiết kế ở trên thành một trình tự có thể chạy được. Giai đoạn đầu đảm bảo cấu hình và ROM hợp lệ; giai đoạn giữa lặp lại việc lấy input, kiểm tra thời gian, mô phỏng và render; giai đoạn cuối giải phóng tài nguyên. Sự phân kỳ rõ ràng giữa đường chạy bình thường và đường lỗi giúp chương trình không tạo cửa sổ SDL2 khi dữ liệu đầu vào chưa đạt yêu cầu.



Hình 7 Lưu đồ khởi động, thực thi và kết thúc của chương trình chính

3.5.1. Luồng khởi động chương trình

Luồng khởi động bắt đầu bằng việc khởi tạo videoScale = 10 và cycleDelay = 8. Đây là giá trị mặc định khi người dùng chỉ cung cấp tên game. Nếu argc bằng 2 và argv[1] là --list, chương trình liệt kê ROM rồi return 0; nếu argc bằng 2 theo cách khác, chuỗi đó được đưa vào ResolveROM(). Khi argc bằng 4, std::stoi chuyển hai tham số đầu thành số nguyên, sau đó ROM nằm tại argv[3].

Lỗi chuyển đổi scale hoặc delay được catch dưới dạng std::exception, in nội dung lỗi và gọi PrintUsage() trước khi std::exit(EXIT_FAILURE). Nếu số lượng tham số không thuộc hai dạng hợp lệ, Main.cpp cũng gọi PrintUsage() ngay. Nhờ đó, các bước tạo Chip8 và Platform chỉ xảy ra sau khi cấu hình đầu vào đã được kiểm tra ở mức cú pháp.

3.5.2. Luồng tải ROM

ResolveROM() xử lý tên yêu cầu trước khi Chip8::LoadROM() làm việc. Các hàm CanOpen(), ToLower(), NormalizeName(), ROMDirectories(), FindROMMatches() và ListROMs() chia nhỏ trách nhiệm tìm kiếm: CanOpen kiểm tra đọc tệp, NormalizeName bỏ các ký tự không phải chữ số/chữ cái và chuyển thường, ROMDirectories trả về hai vị trí cần duyệt, còn FindROMMatches ưu tiên tên trùng hoàn toàn trước khi giữ các tên chứa chuỗi yêu cầu.

Sau khi ResolveROM() trả lại một đường dẫn, đối tượng Chip8 mới được tạo và LoadROM() được gọi. Nếu LoadROM() thất bại, PrintUsage() được in để nhắc cú pháp và main() kết thúc lỗi. Nếu thành công, luồng chuyển sang tạo Platform; điều này cho thấy việc mở ROM là một cổng kiểm soát bắt buộc trước mọi thao tác hiển thị.

Bảng 10 Các nhánh kết quả của quá trình khởi động và tải ROM

Tình huống	Kết quả xử lý	Trạng thái tiếp theo
--list	ListROM() in các tệp .ch8 theo thứ tự.	Kết thúc bình thường.
1 tham số ROM	ResolveROM() chuẩn hóa và tìm ROM.	Tạo Chip8, gọi LoadROM().
3 tham số	Chuyển scale/delay bằng stoi, sau đó ResolveROM().	Tạo Chip8, gọi LoadROM().
Sai tham số	PrintUsage() và exit(EXIT_FAILURE).	Không tạo Chip8/Platform.
ROM không hợp lệ	LoadROM() trả false, PrintUsage().	Không tạo Platform.
ROM hợp lệ	LoadROM() trả true.	Tạo Platform và vào vòng lặp chính.

3.5.3. Luồng xử lý vòng lặp game

Vòng lặp chính dùng while (!quit). Ở đầu mỗi lần lặp, quit nhận giá trị trả về của platform.ProcessInput(chip8.keypad). Nếu người dùng đóng cửa sổ hoặc nhấn Escape, điều kiện vòng lặp sẽ không còn đúng ở lần kiểm tra kế tiếp. Nếu chưa thoát, chương trình lấy currentTime và tính dt là độ chênh bằng mili giây so với lastCycleTime. Khi dt > cycleDelay, lastCycleTime được thay bằng currentTime để đánh dấu chu kỳ vừa thực hiện.

Sau đó Chip8::Cycle() cập nhật trạng thái giả lập và platform.Update(chip8.video, videoPitch) trình bày bộ đệm mới. Nếu dt chưa vượt ngưỡng, vòng lặp vẫn tiếp tục xử lý sự kiện nhưng không thực thi opcode; cách này giúp input không bị bỏ đói trong các khoảng giữa hai chu kỳ mô phỏng.

Thứ tự ProcessInput trước Cycle có ý nghĩa với các opcode phụ thuộc bàn phím. Ví dụ, khi một ROM thực hiện Ex9E hoặc ExA1, trạng thái keypad đã phản ánh sự kiện

SDL gần nhất; khi thực hiện Fx0A mà chưa có phím nào nhấn, hàm giảm pc đi 2 để lặp lại opcode chờ phím ở chu kỳ tiếp theo. Đây là ví dụ trực tiếp cho sự phối hợp giữa Platform, Main.cpp và Chip8.

Bảng 11 Thuật toán vòng lặp chính

Bước	Mô tả thuật toán
1	Gọi ProcessInput(chip8.keypad) và gán kết quả cho quit.
2	Nếu quit là true thì kết thúc vòng lặp.
3	Tính dt từ currentTime - lastCycleTime theo đơn vị mili giây.
4	Nếu dt không lớn hơn cycleDelay thì quay lại Bước 1.
5	Cập nhật lastCycleTime = currentTime.
6	Gọi chip8.Cycle() để thực thi đúng một opcode.
7	Gọi platform.Update(chip8.video, videoPitch) để trình bày khung hình.
8	Quay lại Bước 1 khi người dùng chưa yêu cầu thoát.

3.5.4. Luồng thoát chương trình

Có hai đường thoát chính trong lúc chạy: SDL_QUIT và SDL_KEYDOWN với SDLK_ESCAPE. Cả hai đều được ProcessInput() xử lý bằng cách đặt quit = true. Khi hàm trả giá trị này về Main.cpp, điều kiện while (!quit) dừng và main() đi đến return 0. Vì đối tượng Platform nằm trong phạm vi hàm main(), destructor của nó được gọi tự động trước khi tiến trình kết thúc.

Destructor gọi SDL_DestroyTexture(texture), SDL_DestroyRenderer(renderer), SDL_DestroyWindow(window) và SDL_Quit(). Trình tự này đảm bảo tài nguyên đồ họa cấp thấp được hủy trước khi môi trường SDL kết thúc. Đường lỗi trong giai đoạn tham số hoặc LoadROM() thoát sớm hơn, trước khi Platform được tạo, nên không tồn tại tài nguyên SDL2 cần dọn; đây là lý do kiểm tra ROM được đặt trước constructor Platform.

Từ góc nhìn thiết kế, luồng thoát gọn gàng khép kín vòng đời của tất cả thành phần: Main.cpp kết thúc điều phối, Platform giải phóng SDL2, còn Chip8 là đối tượng tự động chỉ chứa dữ liệu trong bộ nhớ tiến trình. Điều này phù hợp với kiến trúc có trách nhiệm rõ ràng, giúp hệ thống dễ quan sát lỗi và dễ mở rộng ở các giai đoạn sau, chẳng hạn thêm âm thanh hoặc hoàn thiện ánh xạ bàn phím còn thiếu.

CHƯƠNG 4 CÀI ĐẶT CHƯƠNG TRÌNH VÀ KẾT QUẢ KIỂM THỬ

4.1. Môi trường cài đặt

Môi trường cài đặt được xây dựng theo hướng phục vụ việc phát triển ứng dụng C++ có giao diện hiển thị đồ họa đơn giản. Chương trình giả lập CHIP-8 cần thực hiện hai nhóm công việc chính: xử lý logic giả lập ở mức CPU và hiển thị kết quả lên cửa sổ đồ họa. Vì vậy, môi trường phát triển phải đáp ứng được các yêu cầu cơ bản như biên dịch C++17, liên kết được thư viện SDL2, hỗ trợ cấu hình bằng CMake và tạo được tệp thực thi có thể chạy trực tiếp trên Windows.

Trong quá trình triển khai, dự án được tổ chức để có thể cấu hình bằng CMake, biên dịch bằng Ninja và sinh ra chương trình thực thi tên chip8.exe trong thư mục build. Cách làm này giúp tách riêng mã nguồn với sản phẩm biên dịch, hạn chế việc trộn lẫn tệp sinh tự động vào thư mục mã nguồn và thuận tiện khi cần cấu hình lại hoặc build lại chương trình.

Bảng 12 Tổng hợp môi trường cài đặt và công cụ sử dụng

Thành phần	Thông tin sử dụng trong dự án	Vai trò trong quá trình cài đặt
Hệ điều hành	Windows	Là môi trường chạy chương trình, sử dụng đường dẫn dạng C:\... và terminal PowerShell.
Trình soạn thảo	Visual Studio Code	Dùng để mở cấu trúc dự án, quan sát thư mục src, roms, build và các tệp cấu hình.
CMake	Yêu cầu tối thiểu 3.20	Cấu hình project, tạo hệ thống build và khai báo các tệp cần biên dịch.
Generator	Ninja	Thực hiện quá trình build nhanh, sinh ra tệp build.ninja trong thư mục build.
Trình biên dịch	GNU g++ trong môi trường MSYS2/UCRT64	Biên dịch các tệp C++ và liên kết chương trình thành tệp chip8.exe.
Chuẩn ngôn ngữ	C++17	Đảm bảo chương trình sử dụng được các thành phần C++ hiện đại, đặc biệt là xử lý tệp và thư mục.
Thư viện đồ họa	SDL2	Hỗ trợ tạo cửa sổ, nhận sự kiện bàn phím và cập nhật màn hình giả lập.
Tệp thực thi	chip8.exe	Sản phẩm sau khi build, dùng để chạy ROM CHIP-8 từ terminal.

4.1.1. Hệ điều hành

Chương trình được cài đặt và chạy trong môi trường Windows. Điều này thể hiện qua cách tổ chức đường dẫn sử dụng dấu gạch chéo ngược, định dạng đường dẫn bắt đầu bằng ổ đĩa và việc chạy lệnh thông qua PowerShell. Đây là môi trường phù hợp với bộ công cụ MSYS2/UCRT64, CMake và Ninja, đồng thời cho phép chương trình SDL2 mở cửa sổ emulator độc lập để hiển thị nội dung ROM.

Về mặt triển khai, Windows đóng vai trò là nền tảng thực thi cuối cùng của chương trình. Sau khi build thành công, tệp chip8.exe được tạo trong thư mục build và có thể chạy trực tiếp bằng terminal. Khi người dùng truyền vào tham số về tỉ lệ hiển thị, độ trễ chu kỳ và đường dẫn ROM, chương trình khởi tạo bộ giả lập, nạp ROM và mở cửa sổ CHIP-8 Emulator để hiển thị kết quả.

Trong phạm vi báo cáo, phiên bản Windows cụ thể không được xác định vì dữ liệu kiểm thử chỉ thể hiện môi trường Windows và PowerShell. Do đó, phần mô tả không ghi thêm số phiên bản hệ điều hành để tránh đưa vào thông tin không có căn cứ. Điểm quan trọng đối với đồ án là chương trình có thể được cấu hình, build và chạy được trên Windows với chuỗi công cụ CMake - Ninja - g++.

Việc sử dụng Windows cũng thuận tiện cho quá trình kiểm thử nhiều ROM CHIP-8. Các lệnh chạy game được nhập trực tiếp trong PowerShell từ thư mục gốc của dự án. Cách chạy này giúp người kiểm thử dễ thay đổi tham số scale, delay và tên ROM mà không cần chỉnh sửa mã nguồn chương trình.

4.1.2. Trình biên dịch

Dự án sử dụng chuỗi công cụ biên dịch dựa trên CMake, Ninja và GNU g++. Trong đó, CMake giữ vai trò cấu hình project và sinh ra tệp build tương ứng với generator Ninja. Ninja tiếp tục thực hiện các bước biên dịch và liên kết. Trình biên dịch g++ trong môi trường MSYS2/UCRT64 đảm nhiệm việc biên dịch các tệp C++ của chương trình.

Cấu hình CMake của dự án đặt chuẩn C++17, yêu cầu CMake tối thiểu 3.20, tìm thư viện SDL2 thông qua cơ chế package config và tạo executable tên chip8. Các tệp được đưa vào quá trình biên dịch gồm Main.cpp, Platform.cpp và Chip8.cpp. Nhờ cách khai báo này, chương trình được build theo đúng cấu trúc đã thiết kế: Main điều khiển vòng lặp chính, Platform xử lý SDL2 và Chip8 xử lý lõi giả lập.

CMakePresets được thiết lập cho môi trường Windows MinGW UCRT64. Preset này sử dụng generator Ninja, đặt thư mục build là build, chọn kiểu build Debug, chỉ định trình biên dịch C là gcc.exe và trình biên dịch C++ là g++.exe trong thư mục C:/msys64/ucrt64/bin. Ngoài ra, biến CMAKE_PREFIX_PATH được trỏ đến C:/msys64/ucrt64 để CMake có thể tìm thấy SDL2 và các thư viện liên quan trong môi trường MSYS2.

Kết quả cấu hình trong thư mục build cho thấy generator được sử dụng là Ninja, chương trình make tương ứng là ninja.exe và project có tên chip8_final. Quá trình nhận diện trình biên dịch xác định GNU g++ trong môi trường x86_64-w64-mingw32, phiên bản 15.2.0. Đây là thông tin quan trọng vì nó cho biết chương trình được build bằng bộ biên dịch MinGW trên Windows, không phải bằng MSVC.

Với thư viện SDL2, dự án sử dụng cơ chế find_package để tìm gói SDL2 đã cài trong môi trường UCRT64. Khi target SDL2::SDL2main tồn tại, chương trình sẽ liên kết thêm target này trước khi liên kết với SDL2::SDL2. Cách liên kết như vậy giúp chương trình tương thích tốt hơn với cách SDL2 tổ chức entry point trên Windows.

Nhìn chung, chuỗi build của dự án có thể mô tả theo trình tự: chuẩn bị môi trường Windows và MSYS2/UCRT64, cấu hình project bằng CMake, sinh hệ thống build Ninja, biên dịch các tệp C++, liên kết SDL2 và tạo ra tệp chip8.exe. Cách tổ chức này phù hợp với đồ án vì vừa rõ ràng, vừa dễ tái lập khi chuyển sang máy khác có cùng bộ công cụ.

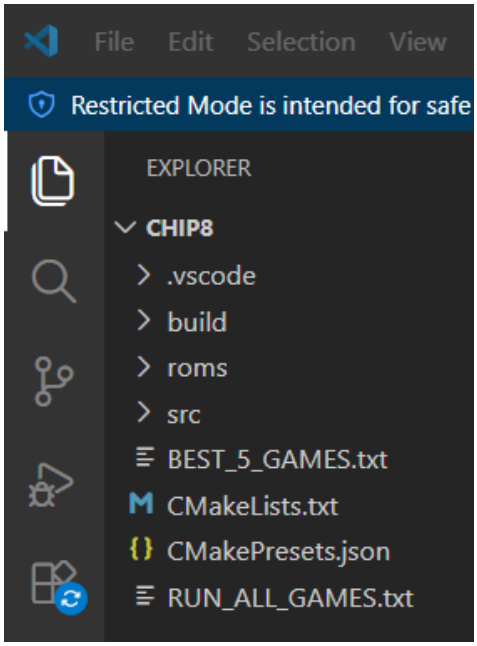
Bảng 13 Vai trò của các công cụ trong quy trình build

Công cụ	Nhiệm vụ chính	Ý nghĩa đối với dự án
CMake	Đọc cấu hình project và sinh hệ thống build.	Giúp chuẩn hóa quá trình cấu hình, không phụ thuộc vào thao tác thủ công.
Ninja	Thực hiện các lệnh biên dịch và liên kết do CMake sinh ra.	Tăng tính tự động và giúp build lại nhanh khi có thay đổi.
GNU g++	Biên dịch mã C++ và tạo tệp đối tượng.	Phù hợp với môi trường MinGW/UCRT64 trên Windows.
SDL2	Cung cấp chức năng cửa sổ, sự kiện và render.	Là thư viện nền cho phần hiển thị và nhập liệu của emulator.
PowerShell	Nhập lệnh build và lệnh chạy chương trình.	Giúp kiểm thử nhiều ROM với các tham số khác nhau mà không cần sửa chương trình.

4.2. Cấu trúc thư mục dự án

Cấu trúc thư mục của dự án được tổ chức theo hướng tách biệt giữa mã nguồn, tài nguyên ROM, tệp cấu hình và kết quả build. Đây là cách tổ chức phù hợp với một chương trình C++ có sử dụng CMake, vì mỗi nhóm tệp có chức năng riêng và có thể quản lý độc lập. Nhờ đó, khi cần chỉnh sửa mã nguồn, thêm ROM kiểm thử hoặc build lại chương trình, người phát triển có thể thao tác đúng vị trí mà không làm ảnh hưởng đến các phần còn lại.

Ở mức tổng quan, thư mục dự án gồm các thành phần chính như .vscode, build, roms, src, BEST_5_GAMES.txt, CMakeLists.txt, CMakePresets.json và RUN_ALL_GAMES.txt. Trong đó, src chứa phần cài đặt chương trình, roms chứa các ROM CHIP-8, build chứa sản phẩm sinh ra sau khi cấu hình và biên dịch, còn các tệp CMake dùng để mô tả cách build chương trình.



Hình 8 Cây thư mục tổng quan của dự án

Bảng 14 Vai trò các thành phần chính trong thư mục dự án

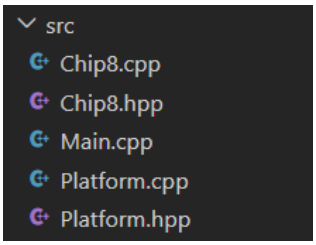
Thành phần	Loại nội dung	Vai trò
.vscode	Cấu hình môi trường làm việc	Hỗ trợ thiết lập khi mở dự án bằng Visual Studio Code.
build	Thư mục sinh tự động	Chứa tệp build.ninja, CMakeCache, tệp trung gian và chip8.exe sau khi build.
roms	Tài nguyên chạy thử	Chứa các ROM .ch8 và một số tệp mô tả đi kèm để kiểm thử emulator.
src	Mã nguồn chương trình	Chứa các tệp triển khai lớp Chip8, lớp Platform và chương trình chính.
BEST_5_GAMES.txt	Ghi chú kiểm thử	Liệt kê một số game phù hợp để chạy thử cùng lệnh và phím điều khiển.
RUN_ALL_GAMES.txt	Danh sách lệnh chạy ROM	Hỗ trợ kiểm thử nhiều ROM bằng cách copy lệnh chạy trong PowerShell.
CMakeLists.txt	Cấu hình build chính	Khai báo project, chuẩn C++17, SDL2 và tệp thực thi chip8.
CMakePresets.json	Preset cấu hình CMake	Xác định generator Ninja, thư mục build, trình biên dịch và đường dẫn UCRT64.

Cách chia thư mục như trên giúp dự án dễ theo dõi hơn trong quá trình phát triển. Mã nguồn chỉ tập trung trong src, dữ liệu đầu vào của emulator nằm trong roms, còn kết quả build nằm trong build. Đây cũng là cơ sở để chương trình có thể tìm ROM theo đường dẫn, theo tên tệp hoặc theo một phần tên game trong quá trình chạy.

4.2.1. Thư mục src

Thư mục src là nơi chứa toàn bộ các tệp C++ trực tiếp tham gia vào quá trình biên dịch chương trình. Theo ảnh cấu trúc, thư mục này gồm năm tệp chính: Chip8.hpp,

Chip8.cpp, Platform.hpp, Platform.cpp và Main.cpp. Cách chia này thể hiện rõ tư duy lập trình hướng đối tượng: phần lõi giả lập được tách khỏi phần hiển thị và xử lý nhập liệu.



Hình 9 Các tệp mã nguồn trong thư mục src

Hai tệp Chip8.hpp và Chip8.cpp tạo thành nhóm xử lý lõi giả lập. Chip8.hpp khai báo lớp, thuộc tính và các phương thức cần thiết để mô phỏng máy CHIP-8. Chip8.cpp triển khai các thao tác như khởi tạo bộ nhớ, nạp fontset, tải ROM, xử lý chu kỳ lệnh và thực thi opcode. Đây là thành phần quan trọng nhất vì nó quyết định độ chính xác của chương trình giả lập.

Hai tệp Platform.hpp và Platform.cpp tạo thành nhóm xử lý nền tảng hiển thị. Platform không trực tiếp thực thi lệnh CHIP-8 mà chịu trách nhiệm làm việc với SDL2, bao gồm tạo cửa sổ, nhận sự kiện bàn phím, cập nhật texture và render vùng màn hình 64x32 của CHIP-8 lên cửa sổ lớn hơn theo tỉ lệ hiển thị. Nhờ vậy, phần giao diện và phần giả lập không bị trộn lẫn trong cùng một lớp.

Tệp Main.cpp giữ vai trò điều phối. Thành phần này xử lý tham số dòng lệnh, tìm ROM cần chạy, khởi tạo đối tượng Chip8, khởi tạo Platform và duy trì vòng lặp chính của chương trình. Trong vòng lặp, Main kết nối dữ liệu giữa Chip8 và Platform: nhận trạng thái phím từ Platform, truyền vào Chip8, sau đó lấy vùng nhớ video của Chip8 để Platform hiển thị lên màn hình.

Bảng 15 Chức năng các tệp trong thư mục src

Tệp	Nhóm chức năng	Mô tả vai trò
Chip8.hpp	Khai báo lõi giả lập	Định nghĩa giao diện lớp Chip8, các hằng số, thuộc tính và phương thức xử lý chính.
Chip8.cpp	Triển khai lõi giả lập	Cài đặt logic nạp ROM, chu kỳ CPU, timer, bộ nhớ, thanh ghi và opcode.
Platform.hpp	Khai báo lớp hiển thị	Định nghĩa giao diện lớp Platform dùng cho cửa sổ, nhập liệu và render bằng SDL2.
Platform.cpp	Triển khai hiển thị và nhập liệu	Cài đặt khởi tạo SDL2, tạo cửa sổ, xử lý bàn phím và cập nhật hình ảnh.
Main.cpp	Điều khiển chương trình	Xử lý tham số chạy, tìm ROM, khởi tạo emulator và duy trì vòng lặp chính.

Về mặt thiết kế, thư mục src thể hiện rõ sự phân tách trách nhiệm. Chip8 tập trung vào trạng thái và hành vi của máy ảo CHIP-8; Platform tập trung vào tác vụ phụ thuộc hệ thống và thư viện SDL2; Main đóng vai trò ghép nối. Cách tổ chức này giúp phần cài đặt dễ kiểm soát hơn, đồng thời phù hợp với yêu cầu của môn lập trình hướng đối tượng.

4.2.2. Thư mục roms

Thư mục roms chứa các chương trình CHIP-8 dùng làm dữ liệu đầu vào cho emulator. Các ROM có phần mở rộng .ch8, mỗi tệp tương ứng với một chương trình hoặc game CHIP-8. Bên cạnh đó, một số ROM có tệp .txt đi kèm để ghi chú thông tin hoặc hướng dẫn liên quan. Việc đặt tất cả ROM trong một thư mục riêng giúp quá trình chạy thử và kiểm thử chương trình thuận tiện hơn.



Hình 10 Một phần danh sách ROM trong thư mục roms

Chương trình hỗ trợ nhiều cách xác định ROM khi chạy. Người dùng có thể nhập đường dẫn trực tiếp đến tệp ROM, nhập tên tệp chính xác hoặc nhập một phần tên game. Khi nhận tên game, chương trình sẽ tìm trong các thư mục ROM được định nghĩa sẵn và ưu tiên các tệp có phần mở rộng .ch8. Cách xử lý này giúp thao tác kiểm thử nhanh hơn, đặc biệt khi thư mục roms chứa nhiều game có tên dài.

Trong quá trình kiểm thử, các ROM được chọn nên là các game có khả năng hiển thị rõ nội dung và dễ kiểm tra phản hồi bàn phím. Danh sách game gợi ý trong dự án bao gồm các game như Space Invaders, Tetris, Blitz, Submarine và Dodge. Các game này giúp kiểm tra nhiều khía cạnh khác nhau của emulator, từ hiển thị sprite, cập nhật màn hình, xử lý phím, đến sự ổn định của vòng lặp xử lý.

Bảng 16 Vai trò dữ liệu trong thư mục roms

Loại tệp	Nội dung	Vai trò trong kiểm thử
.ch8	Chương trình hoặc game CHIP-8.	Dữ liệu chính để nạp vào bộ nhớ và chạy trên emulator.
.txt	Ghi chú hoặc thông tin đi kèm một số ROM.	Hỗ trợ người kiểm thử hiểu thêm về game hoặc cách sử dụng.
ROM đơn giản	Ví dụ các ROM hiển thị cơ bản.	Dùng để kiểm tra nhanh khả năng mở cửa sổ và render màn hình.
ROM game	Các game như Tetris, Blitz, Submarine, Dodge.	Dùng để kiểm tra hiển thị, bàn phím và vòng lặp xử lý trong tình huống thực tế.

Nhờ có thư mục roms, phần kiểm thử của chương trình không phụ thuộc vào việc thay đổi mã nguồn. Người dùng chỉ cần thêm hoặc chọn ROM phù hợp rồi chạy lại chương trình bằng terminal. Điều này giúp quy trình kiểm thử linh hoạt, đồng thời phản ánh đúng cách hoạt động của một emulator: chương trình giả lập giữ nguyên, còn nội dung thực thi thay đổi theo ROM được nạp.

4.2.3. File cấu hình CMake

CMakeLists.txt là tệp cấu hình trung tâm của quá trình build. Tệp này mô tả các thông tin cần thiết để CMake hiểu dự án cần được cấu hình như thế nào, sử dụng chuẩn C++ nào, cần tìm thư viện bên ngoài nào và cần biên dịch những tệp nào để tạo ra chương trình thực thi. Trong dự án, executable được đặt tên là chip8, phù hợp với tên tệp thực thi chip8.exe được sinh ra sau khi build trên Windows.

Một điểm đáng chú ý là dự án đặt chuẩn C++17 và yêu cầu chuẩn này là bắt buộc. Điều này có ý nghĩa vì chương trình sử dụng các thành phần hiện đại của C++, đặc biệt là thao tác với đường dẫn và thư mục trong phần tìm kiếm ROM. Việc cố định chuẩn C++ cũng giúp giảm lỗi không tương thích khi build trên các môi trường khác nhau.

Phần cấu hình SDL2 được thực hiện thông qua cơ chế tìm package. Sau khi SDL2 được tìm thấy, chương trình liên kết với SDL2::SDL2 để sử dụng các chức năng

tạo cửa sổ, xử lý sự kiện và render. Với môi trường Windows, cấu hình cũng kiểm tra thêm target SDL2::SDL2main nếu tồn tại. Đây là cách viết an toàn, giúp chương trình tương thích với cách đóng gói SDL2 trong từng môi trường cài đặt.

Bên cạnh CMakeLists.txt, CMakePresets.json giúp chuẩn hóa thao tác cấu hình. Thay vì phải gõ nhiều tham số thủ công, preset xác định sẵn generator Ninja, thư mục build, kiểu build Debug, trình biên dịch và đường dẫn tiền tố của môi trường UCRT64. Nhờ đó, người phát triển có thể tái lập môi trường build một cách rõ ràng và nhất quán.

Bảng 17 Nội dung chính của cấu hình CMake

Nhóm cấu hình	Giá trị/ý nghĩa	Tác dụng
Phiên bản CMake	Yêu cầu tối thiểu 3.20.	Đảm bảo CMake đủ khả năng xử lý cấu hình dự án.
Tên project	chip8_final.	Định danh dự án trong quá trình cấu hình và build.
Chuẩn C++	C++17 và yêu cầu bắt buộc.	Giữ môi trường biên dịch thống nhất.
Thư viện SDL2	Tìm bằng cơ chế package config.	Liên kết phần hiển thị và nhập liệu với SDL2.
Tệp thực thi	chip8.	Tạo chương trình chạy chính của emulator.
Tệp biên dịch	Main.cpp, Platform.cpp, Chip8.cpp.	Bao gồm đủ ba thành phần điều khiển, hiển thị và lõi giả lập.
Preset Windows	Generator Ninja, binaryDir build, trình biên dịch UCRT64.	Chuẩn hóa quy trình cấu hình trên Windows.

Nhìn tổng thể, cấu trúc thư mục và tệp cấu hình CMake đã hỗ trợ tốt cho quá trình phát triển emulator. Phần mã nguồn được tách rõ theo chức năng, tài nguyên ROM được quản lý riêng, thư mục build chứa sản phẩm sinh tự động và các tệp cấu hình giúp quá trình biên dịch có thể lặp lại. Đây là nền tảng cần thiết để chuyển sang các nội dung

tiếp theo của chương 4 như cài đặt lớp Chip8, cài đặt Platform, hướng dẫn build và kiểm thử các game.

4.3. Cài đặt lớp Chip8

Lớp Chip8 là thành phần trung tâm của chương trình giả lập, đảm nhiệm việc mô phỏng hoạt động bên trong của hệ thống CHIP-8. Đây là lớp xử lý các chức năng quan trọng như quản lý bộ nhớ, thanh ghi, stack, bộ đếm chương trình, bộ định thời, bàn phím, màn hình ảo và tập lệnh opcode. Trong chương trình, lớp Chip8 được xây dựng theo hướng đối tượng nhằm đóng gói toàn bộ logic giả lập vào một lớp riêng. Nhờ đó, phần xử lý của CHIP-8 được tách biệt với phần hiển thị và nhập liệu do lớp Platform đảm nhiệm. Cách thiết kế này giúp mã nguồn rõ ràng hơn, dễ theo dõi và thuận tiện cho việc bảo trì.

Về dữ liệu, lớp Chip8 khai báo các thành phần mô phỏng kiến trúc của CHIP-8 như memory để lưu bộ nhớ 4096 byte, registers để lưu 16 thanh ghi đa dụng, stack để hỗ trợ gọi chương trình con, pc để quản lý địa chỉ lệnh hiện tại, index để lưu địa chỉ bộ nhớ, delayTimer và soundTimer để xử lý thời gian. Ngoài ra, lớp còn có keypad để lưu trạng thái bàn phím và video để lưu dữ liệu màn hình ảo 64 x 32 pixel.

Về xử lý, lớp Chip8 cung cấp hai phương thức chính là LoadROM và Cycle. Phương thức LoadROM dùng để đọc file ROM và nạp dữ liệu vào bộ nhớ từ địa chỉ 0x200. Phương thức Cycle dùng để thực hiện một chu kỳ lệnh, bao gồm lấy opcode từ bộ nhớ, giải mã, thực thi lệnh và cập nhật bộ định thời.

Ngoài ra, lớp Chip8 còn cài đặt nhiều hàm xử lý opcode khác nhau, tương ứng với các nhóm lệnh của CHIP-8 như nhóm lệnh điều khiển chương trình, nhóm lệnh tính toán, nhóm lệnh đồ họa, nhóm lệnh bàn phím và nhóm lệnh liên quan đến bộ nhớ hoặc timer.

Tóm lại, lớp Chip8 đóng vai trò là phần lõi của emulator. Việc cài đặt lớp này quyết định trực tiếp đến khả năng nạp ROM, xử lý lệnh và mô phỏng hoạt động của hệ thống CHIP-8.

4.3.1. Khởi tạo bộ nhớ và thanh ghi

Trong lớp Chip8, việc khởi tạo bộ nhớ và thanh ghi được thực hiện khi đối tượng Chip8 được tạo ra. Đây là bước quan trọng đầu tiên để chuẩn bị môi trường thực thi cho chương trình giả lập. Hệ thống CHIP-8 sử dụng bộ nhớ có kích thước 4096 byte. Trong

chương trình, bộ nhớ này được khai báo bằng mảng memory với kích thước MEMORY_SIZE. Bộ nhớ dùng để lưu dữ liệu hệ thống, fontset và nội dung ROM sau khi được nạp vào.

Bên cạnh bộ nhớ, chương trình còn khai báo 16 thanh ghi đa dụng thông qua mảng registers. Mỗi thanh ghi có kích thước 8 bit và được sử dụng trong quá trình xử lý các opcode. Các thanh ghi này tương ứng với các thanh ghi V0 đến VF trong kiến trúc CHIP-8, trong đó VF thường được dùng làm cờ trạng thái khi thực hiện một số phép toán hoặc thao tác vẽ. Ngoài ra, lớp Chip8 còn khởi tạo các thành phần quan trọng khác như thanh ghi index, bộ đếm chương trình pc, stack, con trỏ stack sp, delayTimer, soundTimer và opcode. Các biến này phục vụ cho quá trình điều khiển luồng thực thi lệnh, gọi chương trình con, xử lý thời gian và lưu mã lệnh hiện tại.

Trong hàm khởi tạo Chip8(), bộ đếm chương trình pc được gán bằng START_ADDRESS, có giá trị là 0x200. Đây là địa chỉ bắt đầu thực thi chương trình của CHIP-8. Lý do là vùng nhớ từ 0x000 đến trước 0x200 thường được dành cho hệ thống và fontset, còn chương trình ROM sẽ được nạp bắt đầu từ địa chỉ 0x200.

Các mảng trong lớp như memory, registers, stack, keypad và video được khai báo với giá trị khởi tạo mặc định bằng 0. Điều này giúp trạng thái ban đầu của bộ giả lập ổn định, tránh việc sử dụng dữ liệu rác trong quá trình chạy chương trình.

4.3.2. Nạp fontset vào bộ nhớ

Trong CHIP-8, fontset là tập dữ liệu dùng để hiển thị các ký tự số và chữ hệ thập lục phân từ 0 đến F. Mỗi ký tự được biểu diễn bằng một sprite có kích thước 4 x 5 pixel. Vì CHIP-8 thường sử dụng các ký tự này để hiển thị điểm số, số mạng hoặc thông tin trong game, fontset cần được nạp sẵn vào bộ nhớ trước khi chương trình ROM bắt đầu chạy. Trong chương trình, fontset được khai báo bằng một mảng gồm 80 byte. Số lượng 80 byte xuất phát từ việc CHIP-8 có 16 ký tự, từ 0 đến F, và mỗi ký tự chiếm 5 byte dữ liệu. Mỗi byte biểu diễn một dòng pixel của ký tự. Fontset được nạp vào bộ nhớ bắt đầu từ địa chỉ FONTSET_START_ADDRESS, có giá trị là 0x50. Đây là vùng nhớ được dành riêng để lưu dữ liệu font. Khi một opcode cần lấy địa chỉ sprite của một ký tự, chương trình sẽ dựa vào địa chỉ bắt đầu này để tính ra vị trí tương ứng trong bộ nhớ.

Việc nạp fontset được thực hiện trong hàm khởi tạo Chip8(). Chương trình sử dụng vòng lặp để sao chép lần lượt từng byte trong mảng fontset vào mảng memory,

bắt đầu từ địa chỉ 0x50. Nhờ vậy, khi emulator chạy, các sprite ký tự đã có sẵn trong bộ nhớ để phục vụ cho các lệnh hiển thị. Cụ thể, mỗi ký tự chiếm 5 byte. Ví dụ, ký tự 0 được lưu bằng 5 byte đầu tiên của mảng fontset, ký tự 1 dùng 5 byte tiếp theo, và cứ tiếp tục như vậy đến ký tự F. Khi cần lấy sprite của một ký tự, chương trình có thể tính địa chỉ bằng công thức: FONTSET_START_ADDRESS + 5 * digit.

Tóm lại, bước nạp fontset giúp emulator có sẵn dữ liệu hình ảnh cho các ký tự cơ bản của CHIP-8. Đây là bước cần thiết để các lệnh vẽ ký tự hoặc hiển thị số trong ROM có thể hoạt động đúng.

Vậy, Fontset là dữ liệu hình ảnh của các ký tự 0 đến F. Chương trình nạp fontset vào vùng nhớ 0x50 để khi ROM cần vẽ ký tự, emulator có thể lấy đúng sprite tương ứng trong bộ nhớ.

4.3.3. Tải ROM vào bộ nhớ

Sau khi khởi tạo bộ nhớ, thanh ghi và nạp fontset, bước tiếp theo của lớp Chip8 là tải ROM vào bộ nhớ. ROM là file chương trình hoặc game CHIP-8, thường có định dạng .ch8. Đây là dữ liệu chính mà emulator sẽ đọc, giải mã và thực thi trong quá trình chạy. Trong chương trình, việc tải ROM được thực hiện thông qua phương thức LoadROM. Phương thức này nhận vào tên file hoặc đường dẫn file ROM, sau đó mở file ở chế độ nhị phân để đọc chính xác từng byte dữ liệu.

Trước khi đọc dữ liệu, chương trình kiểm tra xem file ROM có mở thành công hay không. Nếu file không tồn tại hoặc không thể mở, hàm sẽ trả về false để thông báo quá trình tải ROM thất bại. Ngoài ra, chương trình cũng kiểm tra kích thước ROM để đảm bảo ROM không vượt quá dung lượng bộ nhớ còn trống của CHIP-8. Theo kiến trúc CHIP-8, chương trình ROM thường được nạp bắt đầu từ địa chỉ 0x200. Vì vùng nhớ từ 0x000 đến trước 0x200 thường được dành cho hệ thống và fontset, nên emulator sẽ không ghi ROM vào vùng nhớ này. Trong chương trình, địa chỉ bắt đầu này được định nghĩa bằng hằng START_ADDRESS.

Sau khi kiểm tra hợp lệ, chương trình đọc toàn bộ dữ liệu ROM vào một bộ đệm tạm, sau đó sao chép từng byte của ROM vào mảng memory, bắt đầu từ địa chỉ 0x200. Khi hoàn tất, dữ liệu ROM đã nằm trong bộ nhớ và sẵn sàng để được xử lý bởi chu trình lệnh của CHIP-8. Việc tải ROM vào đúng địa chỉ là rất quan trọng. Nếu ROM được nạp

sai vị trí, bộ đếm chương trình pc sẽ không đọc đúng lệnh đầu tiên, dẫn đến emulator chạy sai hoặc không thể chạy chương trình.

Tóm lại, phương thức LoadROM đảm nhiệm việc đọc file ROM, kiểm tra tính hợp lệ của file và nạp dữ liệu vào bộ nhớ từ địa chỉ 0x200. Đây là bước chuẩn bị trực tiếp trước khi emulator bắt đầu thực thi chương trình.

Vậy, ROM là dữ liệu game CHIP-8. Chương trình đọc ROM dưới dạng nhị phân, kiểm tra kích thước hợp lệ và nạp ROM vào bộ nhớ từ địa chỉ 0x200 để Chip8 bắt đầu thực thi.

4.3.4. Xử lý chu kỳ lệnh

Sau khi ROM được nạp vào bộ nhớ, chương trình bắt đầu thực thi các lệnh CHIP-8 thông qua phương thức Cycle của lớp Chip8. Đây là phương thức mô phỏng một chu kỳ hoạt động của CPU trong hệ thống CHIP-8. Trong mỗi chu kỳ lệnh, chương trình thực hiện ba bước chính: Fetch, Decode và Execute. Đây là nguyên lý cơ bản của quá trình xử lý lệnh trong một trình giả lập.

Đầu tiên là bước Fetch. Ở bước này, chương trình lấy opcode hiện tại từ bộ nhớ dựa vào vị trí của bộ đếm chương trình pc. Mỗi opcode của CHIP-8 có kích thước 2 byte, vì vậy chương trình cần đọc hai ô nhớ liên tiếp và ghép chúng lại thành một mã lệnh 16 bit. Sau khi lấy được opcode, chương trình tăng pc thêm 2 để trỏ đến lệnh tiếp theo. Việc tăng pc được thực hiện trước khi thực thi lệnh, vì một số opcode có thể thay đổi giá trị pc, ví dụ như lệnh nhảy, gọi chương trình con hoặc bỏ qua lệnh.

Tiếp theo là bước Decode và Execute. Chương trình sử dụng 4 bit đầu của opcode để xác định nhóm lệnh cần xử lý. Dựa vào giá trị này, chương trình gọi đến hàm xử lý tương ứng thông qua bảng con trỏ hàm. Cách làm này giúp việc xử lý opcode rõ ràng hơn, thay vì phải viết một câu lệnh switch-case quá dài.

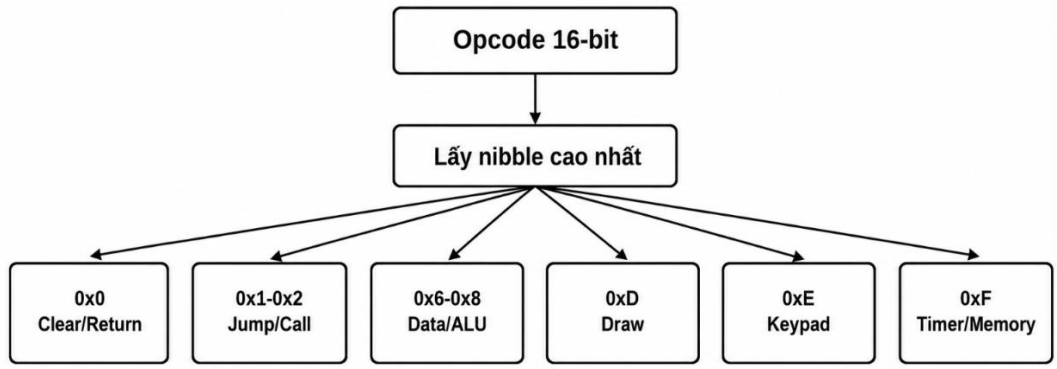
Sau khi opcode được thực thi, chương trình kiểm tra và cập nhật hai bộ định thời là delayTimer và soundTimer. Nếu giá trị của các timer lớn hơn 0, chương trình sẽ giảm mỗi timer đi 1. Điều này giúp mô phỏng hoạt động của bộ định thời trong CHIP-8. Phương thức Cycle được gọi liên tục trong vòng lặp chính của chương trình. Mỗi lần gọi Cycle, emulator sẽ xử lý một lệnh CHIP-8. Khi kết hợp với việc cập nhật màn hình và xử lý bàn phím, chương trình có thể chạy được các ROM/game CHIP-8.

Tóm lại, chu kỳ lệnh là phần quan trọng nhất trong quá trình giả lập. Nó đảm nhiệm việc đọc lệnh, giải mã lệnh, thực thi opcode và cập nhật timer, giúp ROM có thể hoạt động đúng theo kiến trúc CHIP-8.

Tóm lại, phương thức Cycle mô phỏng một chu kỳ xử lý lệnh của CHIP-8. Trong mỗi chu kỳ, chương trình lấy opcode từ bộ nhớ, tăng pc, gọi hàm xử lý opcode tương ứng và cập nhật các bộ định thời.

4.4. Cài đặt xử lý opcode

Tập lệnh CHIP-8 được chia thành nhiều nhóm, mỗi nhóm thao tác trên một phần trạng thái của máy ảo. Project tổ chức việc giải mã opcode bằng cách phân phối theo nibble cao nhất và các bảng phụ cho một số nhóm lệnh đặc biệt. Cách làm này giúp mã nguồn dễ bảo trì hơn so với việc gom toàn bộ logic vào một khối điều kiện dài.



Hình 11 Sơ đồ phân nhóm xử lý opcode trong lớp Chip8

4.4.1. Nhóm lệnh nhảy và gọi chương trình con

Nhóm lệnh điều khiển luồng quyết định thứ tự thực thi chương trình. Các lệnh nhảy thay đổi trực tiếp program counter, trong khi lệnh gọi chương trình con cần lưu địa chỉ quay về vào stack trước khi chuyển đến địa chỉ mới. Khi gặp lệnh trả về, emulator lấy địa chỉ đã lưu trong stack để tiếp tục thực thi sau lời gọi.

Bảng 18 Nhóm opcode điều khiển luồng

Opcode	Chức năng	Thành phần bị ảnh hưởng
1nnn	Nhảy đến địa chỉ nnn	pc
2nnn	Gọi chương trình con tại nnn	stack, sp, pc
00EE	Trả về từ chương trình con	stack, sp, pc
3xkk	Bỏ qua lệnh kế tiếp nếu Vx bằng kk	pc
4xkk	Bỏ qua lệnh kế tiếp nếu Vx khác kk	pc
5xy0	Bỏ qua lệnh kế tiếp nếu Vx bằng Vy	pc
9xy0	Bỏ qua lệnh kế tiếp nếu Vx khác Vy	pc

Nhóm lệnh này cần được kiểm tra kỹ vì chỉ cần program counter hoặc stack pointer sai một bước, ROM có thể nhảy sai địa chỉ, treo chương trình hoặc không quay về đúng vị trí sau khi gọi chương trình con.

4.4.2. Nhóm lệnh gán và tính toán

Nhóm lệnh gán và tính toán thao tác chủ yếu trên các thanh ghi V0 đến VF. Các lệnh này dùng để gán hằng số, cộng dữ liệu, sao chép thanh ghi, thực hiện toán tử logic và dịch bit. Thanh ghi VF thường được dùng làm cờ carry, no-borrow hoặc lưu bit bị dịch ra tùy từng opcode. Theo mô tả CHIP-8, VF có vai trò đặc biệt trong nhiều phép toán và trong lệnh vẽ sprite.

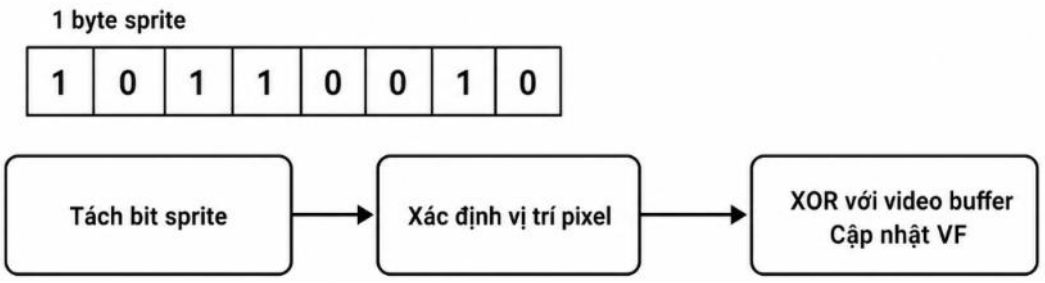
Bảng 19 Một số opcode nhóm gán và tính toán

Opcode	Chức năng
6xkk	Gán giá trị kk vào thanh ghi Vx
7xkk	Cộng kk vào thanh ghi Vx
8xy0	Gán Vx bằng Vy
8xy1	Thực hiện phép OR giữa Vx và Vy
8xy2	Thực hiện phép AND giữa Vx và Vy
8xy3	Thực hiện phép XOR giữa Vx và Vy
8xy4	Cộng Vx và Vy, VF biểu diễn carry
8xy5	Trừ Vy khỏi Vx, VF biểu diễn no-borrow
8xy6	Dịch phải thanh ghi
8xyE	Dịch trái thanh ghi

Trong quá trình cài đặt, các phép toán 8-bit cần được xử lý cẩn thận để vừa giữ đúng kết quả thấp 8 bit, vừa cập nhật cờ VF đúng quy tắc. Đây là cơ sở để các ROM game sử dụng các phép toán điều kiện và va chạm hoạt động ổn định.

4.4.3. Nhóm lệnh vẽ sprite

Nhóm lệnh đồ họa quan trọng nhất là opcode Dxyn. Lệnh này vẽ sprite có chiều rộng 8 pixel và chiều cao n dòng tại vị trí xác định bởi hai thanh ghi tọa độ. Dữ liệu sprite được đọc từ vùng nhớ bắt đầu tại thanh ghi I. Việc vẽ sử dụng phép XOR lên video buffer, vì vậy pixel đang tắt có thể được bật lên và pixel đang bật có thể bị tắt.



Hình 12 Nguyên lý vẽ sprite bằng phép XOR

Khi phép XOR làm một pixel đang bật chuyển thành tắt, thanh ghi VF được đặt để báo có va chạm. Cơ chế này được nhiều game CHIP-8 sử dụng để phát hiện va chạm

giữa bóng, vật thể hoặc sprite. Theo tài liệu kỹ thuật CHIP-8, màn hình gốc có độ phân giải 64 x 32 pixel đơn sắc, sprite rộng 8 pixel và được vẽ bằng XOR.

4.4.4. Nhóm lệnh bàn phím

Nhóm lệnh bàn phím giúp ROM phản hồi thao tác của người dùng. Lõi Chip8 không đọc trực tiếp phím vật lý từ hệ điều hành. Thay vào đó, nó chỉ kiểm tra mảng keypad gồm 16 phần tử. Mảng này được lớp Platform cập nhật dựa trên sự kiện SDL2.

Bảng 20 Nhóm opcode xử lý bàn phím

Opcode	Chức năng
Ex9E	Bỏ qua lệnh kế tiếp nếu phím có mã trong Vx đang được nhấn
ExA1	Bỏ qua lệnh kế tiếp nếu phím có mã trong Vx không được nhấn
Fx0A	Chờ người dùng nhấn một phím và lưu mã phím vào Vx

Với lệnh chờ phím, nếu chưa có phím nào được nhấn, chương trình cần giữ luồng thực thi ở trạng thái chờ. Cách xử lý này giúp ROM có thể dừng ở màn hình chờ thao tác mà không làm sai trạng thái của các thanh ghi khác.

4.4.5. Nhóm lệnh timer và bộ nhớ

Nhóm lệnh timer và bộ nhớ xử lý delay timer, sound timer, địa chỉ font, biểu diễn BCD và trao đổi dữ liệu giữa thanh ghi với bộ nhớ. Đây là nhóm lệnh cần thiết để ROM quản lý thời gian, hiển thị điểm số và lưu khôi phục trạng thái thanh ghi.

Bảng 21 Nhóm opcode timer và bộ nhớ

Opcode	**Vai trò**
Fx07	Đọc delay timer vào Vx
Fx15	Gán delay timer bằng Vx
Fx18	Gán sound timer bằng Vx
Fx1E	Cộng Vx vào thanh ghi I
Fx29	Đặt I đến địa chỉ font của ký tự trong Vx
Fx33	Lưu biểu diễn BCD của Vx vào bộ nhớ
Fx55	Lưu dãy thanh ghi từ V0 đến Vx vào bộ nhớ
Fx65	Đọc dữ liệu từ bộ nhớ vào dãy thanh ghi từ V0 đến Vx

Trong phạm vi project hiện tại, sound timer được duy trì như một phần trạng thái máy ảo. Tuy nhiên, báo cáo chưa đánh giá phát âm thanh thực tế qua thiết bị loa. Do đó, kết quả kiểm thử tập trung vào ROM, opcode, hiển thị và input.

4.5. Cài đặt lớp Platform

Lớp Platform bao bọc các thao tác SDL2. Nhiệm vụ của lớp là tạo cửa sổ, tạo renderer, tạo texture, cập nhật hình ảnh và xử lý sự kiện bàn phím. Việc tách Platform khỏi Chip8 giúp lõi giả lập không phụ thuộc trực tiếp vào chi tiết của thư viện đồ họa.

4.5.1. Khởi tạo SDL2

Khi đối tượng Platform được tạo, chương trình khởi tạo hệ thống video của SDL2, sau đó tạo cửa sổ, renderer và texture. Theo tài liệu SDL2, hàm tạo cửa sổ nhận tiêu đề, vị trí, chiều rộng, chiều cao và các cờ cấu hình để sinh ra một SDL_Window.

Bảng 22 Tài nguyên SDL2 trong lớp Platform

Tài nguyên SDL2	**Vai trò**
SDL_Window	Đại diện cho cửa sổ hiển thị emulator
SDL_Renderer	Thực hiện thao tác vẽ texture lên cửa sổ
SDL_Texture	Lưu dữ liệu hình ảnh trung gian từ video buffer

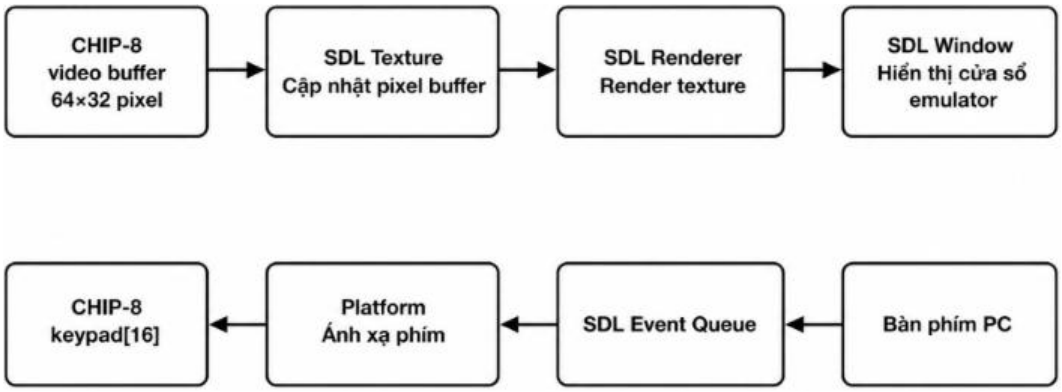
4.5.2. Tạo cửa sổ hiển thị

Màn hình gốc của CHIP-8 chỉ có độ phân giải 64 x 32 pixel. Nếu hiển thị trực tiếp với kích thước này, cửa sổ sẽ rất nhỏ và khó quan sát. Vì vậy, chương trình dùng hệ số videoScale để phóng to màn hình. Kích thước cửa sổ được tính bằng cách nhân chiều rộng và chiều cao gốc với hệ số phóng to.

Ví dụ, với videoScale bằng 12, màn hình 64 x 32 được phóng to thành cửa sổ 768 x 384 pixel. Cách làm này giữ nguyên tỷ lệ hình ảnh, đồng thời giúp mỗi pixel CHIP-8 được hiển thị rõ ràng trên màn hình máy tính.

4.5.3. Render màn hình CHIP-8

Sau khi lõi Chip8 cập nhật video buffer, lớp Platform nhận bộ đệm này và đưa dữ liệu vào texture SDL2. Tiếp theo, renderer xóa khung hình cũ, copy texture mới lên cửa sổ và trình bày khung hình hiện tại. Luồng render được tổ chức một chiều từ video buffer sang texture, renderer và cuối cùng là window.

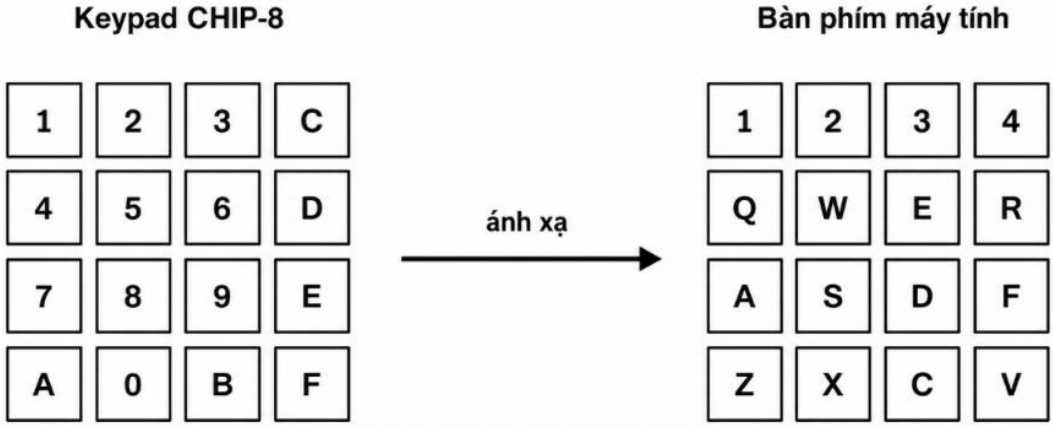


Hình 13 Luồng hiển thị và nhập liệu qua SDL2

Thiết kế này bảo đảm Chip8 chỉ phụ trách dữ liệu hiển thị, còn Platform phụ trách cách đưa dữ liệu đó ra cửa sổ thật. Nếu sau này thay SDL2 bằng thư viện khác, phần lớn logic opcode trong Chip8 vẫn có thể giữ nguyên.

4.5.4. Ánh xạ bàn phím

Bàn phím CHIP-8 gồm 16 phím hexa từ 0 đến F. Vì bàn phím máy tính hiện đại không có bố cục giống keypad CHIP-8, chương trình cần ánh xạ các phím máy tính sang mảng keypad. SDL2 cung cấp cơ chế lấy sự kiện đang chờ trong hàng đợi sự kiện; tài liệu SDL2 khuyến nghị xử lý hàng đợi sự kiện trong vòng lặp chính để không làm treo vòng lặp cập nhật game.



Hình 14 Sơ đồ ánh xạ bàn phím CHIP-8 sang bàn phím máy tính

Bảng 23 Bảng ánh xạ bàn phím trong chương trình

Phím CHIP-8	Phím máy tính	Ghi chú
0x0	X	Phím 0 của keypad CHIP-8
0x1	1	Phím 1
0x2	2	Phím 2
0x3	3	Phím 3
0x4	Q / A / mũi tên trái	Hỗ trợ di chuyển trái
0x5	W	Phím phụ
0x6	E / D / mũi tên phải	Hỗ trợ di chuyển phải
0x8	S	Phím phụ
0xA	Z	Phím A của keypad
0xB	C	Phím B của keypad
0xC	4	Phím C của keypad
0xD	R	Phím D của keypad
0xE	F	Phím E của keypad
0xF	V	Phím F của keypad

4.6. Cài đặt chương trình chính

Chương trình chính nằm trong Main.cpp và có nhiệm vụ điều phối toàn bộ vòng đời ứng dụng. Phần này không xử lý chi tiết opcode và cũng không trực tiếp quản lý texture, mà tập trung vào đọc tham số, tìm ROM, khởi tạo đối tượng, duy trì vòng lặp và kết thúc chương trình an toàn.

4.6.1. Xử lý tham số dòng lệnh

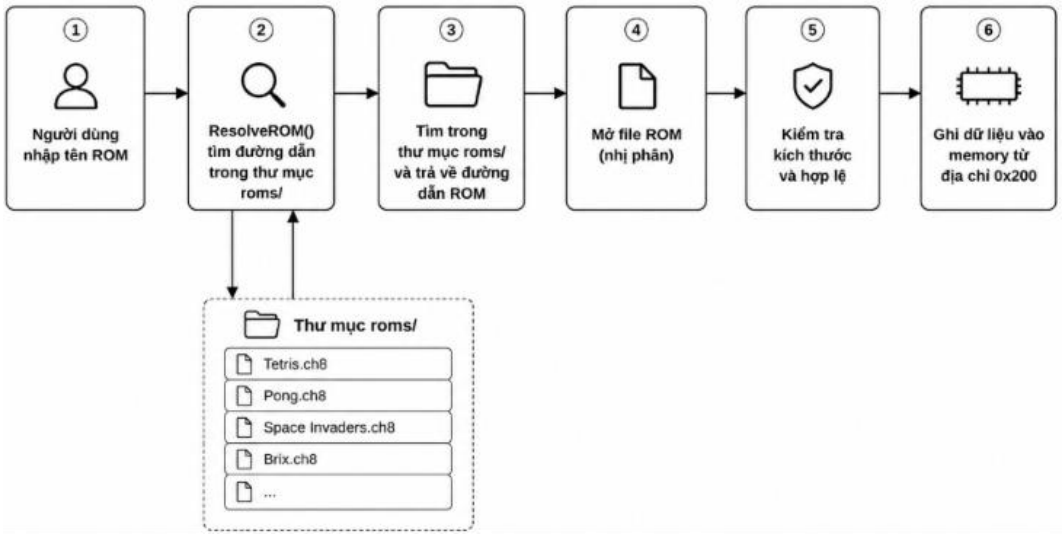
Chương trình hỗ trợ chạy ROM thông qua tham số dòng lệnh. Người dùng có thể truyền tên ROM rút gọn, đường dẫn ROM hoặc các tham số cấu hình như hệ số phóng to màn hình và độ trễ chu kỳ. Cách thiết kế này giúp chương trình linh hoạt hơn: có thể chạy nhanh khi demo bằng tên ROM ngắn, hoặc chạy chính xác hơn khi truyền đủ tham số.

Bảng 24 Ý nghĩa các nhóm tham số dòng lệnh

Loại tham số	**Ý nghĩa**
Tên ROM rút gọn	Cho phép tìm ROM trong thư mục roms theo tên gần đúng
Đường dẫn ROM	Cho phép chạy trực tiếp một file .ch8 cụ thể
videoScale	Điều chỉnh kích thước cửa sổ hiển thị
cycleDelay	Điều chỉnh độ trễ giữa các chu kỳ xử lý
Tùy chọn liệt kê ROM	Hỗ trợ người dùng xem danh sách ROM có thể chạy

4.6.2. Tìm kiếm ROM

Chức năng tìm ROM giúp người dùng không cần nhập đầy đủ tên file dài. Chương trình kiểm tra trước trường hợp người dùng truyền đường dẫn trực tiếp, sau đó tìm trong thư mục roms và so khớp tên gần đúng. Nếu có nhiều ROM phù hợp, chương trình chọn kết quả phù hợp nhất và có thể thông báo các kết quả liên quan.



Hình 15 Luồng tìm kiếm và chuẩn hóa đường dẫn ROM

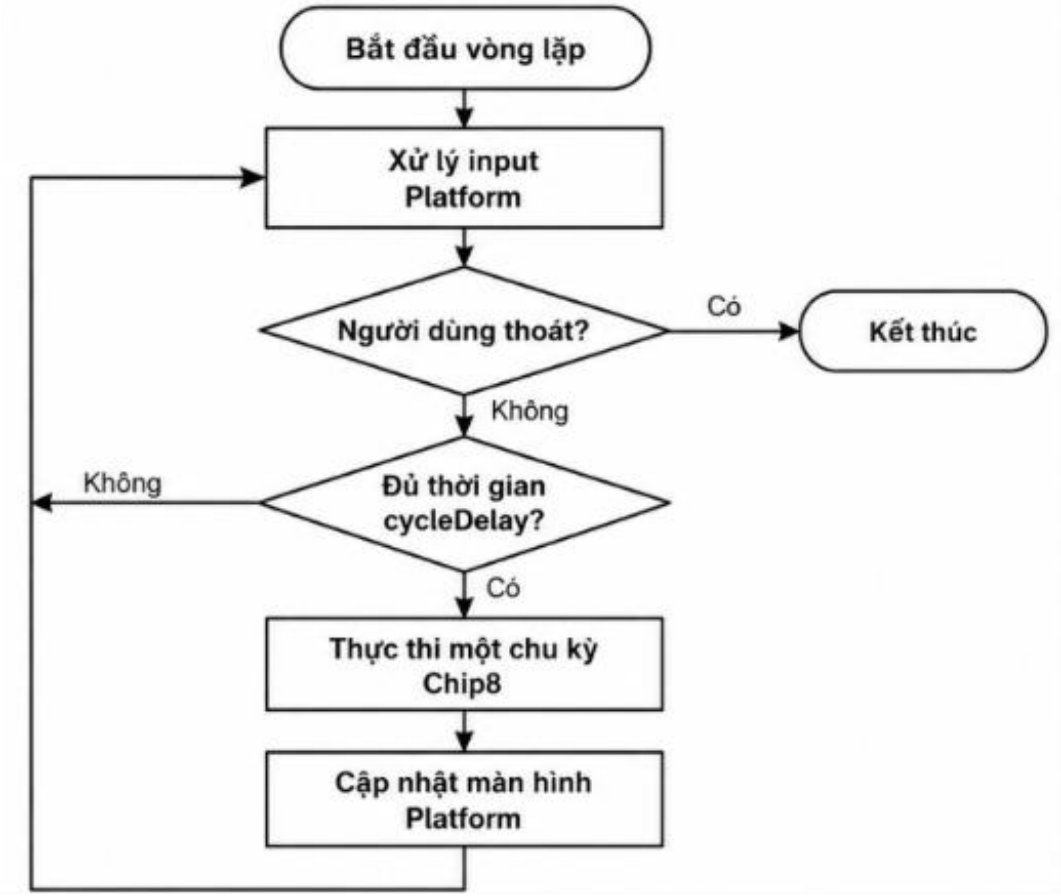
4.6.3. Khởi tạo emulator

Sau khi xác định được ROM, chương trình tạo đối tượng Chip8 và gọi phương thức tải ROM để nạp dữ liệu vào bộ nhớ. Nếu quá trình nạp ROM thất bại, chương trình dừng để tránh chạy trong trạng thái không hợp lệ. Nếu nạp ROM thành công, chương trình tạo đối tượng Platform để mở cửa sổ hiển thị.

Trình tự khởi tạo thể hiện rõ sự phối hợp giữa ba thành phần chính: Main đọc tham số và tìm ROM; Chip8 nạp ROM và xử lý máy ảo; Platform tạo môi trường hiển thị và input. Đây là điểm thể hiện rõ cách phân tách trách nhiệm trong thiết kế hướng đối tượng của project.

4.6.4. Xử lý vòng lặp chính

Vòng lặp chính duy trì chương trình trong suốt thời gian chạy ROM. Ở mỗi vòng lặp, Platform xử lý sự kiện bàn phím và trạng thái thoát. Sau đó, chương trình kiểm tra thời gian đã trôi qua so với cycleDelay. Khi đủ điều kiện, Chip8 thực thi một chu kỳ lệnh và Platform cập nhật màn hình.

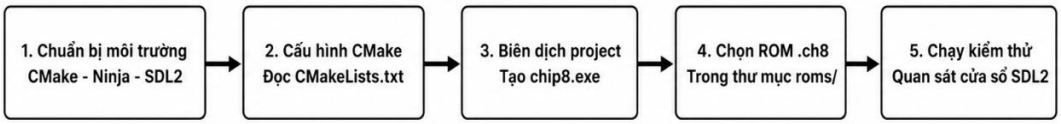


Hình 16 Vòng lặp chính của emulator

Cách tổ chức này giúp chương trình vừa phản hồi input, vừa kiểm soát tốc độ thực thi, vừa cập nhật hiển thị liên tục. Người dùng có thể thoát bằng cách đóng cửa sổ hoặc nhấn phím thoát. Khi vòng lặp kết thúc, tài nguyên SDL2 được giải phóng bởi lớp Platform.

4.7. Hướng dẫn build và chạy chương trình

Để build chương trình, người dùng cần chuẩn bị môi trường gồm CMake, Ninja, trình biên dịch C++ và SDL2. Quy trình build gồm ba giai đoạn chính: cấu hình project, biên dịch project và chạy file thực thi với ROM kiểm thử. Trong báo cáo này, lệnh cụ thể không được trình bày trong thân bài nhằm tránh biến phần cài đặt thành phụ lục mã nguồn hoặc log terminal. Người thực hiện có thể chèn ảnh terminal build thành công vào vị trí hình tương ứng.



Hình 17 Quy trình build và chạy kiểm thử chương trình

Bảng 25 Quy trình build và chạy chương trình

Bước	**Nội dung thực hiện**	**Kết quả mong đợi**
1	Chuẩn bị CMake, Ninja, compiler và SDL2	Môi trường sẵn sàng để cấu hình project
2	Cấu hình project bằng CMake	Sinh thư mục build và file cấu hình build
3	Biên dịch project	Tạo file thực thi chip8.exe
4	Chọn ROM trong thư mục roms	Xác định ROM .ch8 cần kiểm thử
5	Chạy emulator và quan sát cửa sổ SDL2	ROM được hiển thị trong cửa sổ giả lập

Khi import kết quả thực tế, nên chèn ảnh terminal sau khi build thành công và ảnh terminal khi chạy ROM. Hai ảnh này giúp chứng minh chương trình không chỉ được mô tả về mặt thiết kế mà còn biên dịch và chạy được trong môi trường thực tế.

4.8. Kết quả kiểm thử 5 game

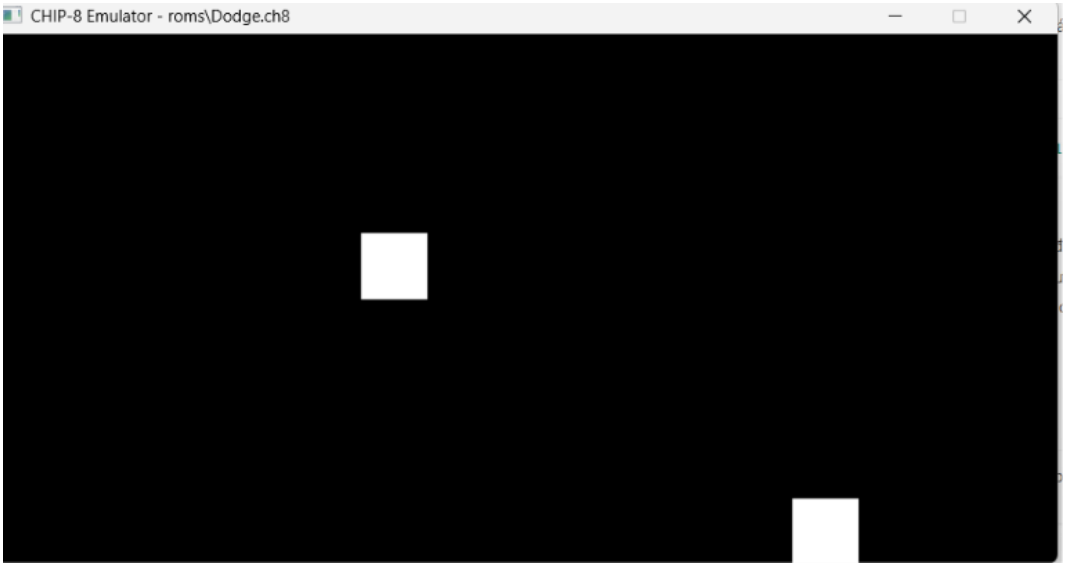
Phần kiểm thử được thực hiện bằng cách chạy một số ROM .ch8 trong thư mục roms và quan sát kết quả hiển thị. Mục tiêu kiểm thử là xác nhận chương trình có thể nạp ROM, thực thi opcode, render hình ảnh và nhận input ở mức trực quan. Các hình

kết quả trong phần này đang được đặt dưới dạng khung chờ để người thực hiện thay bằng ảnh chụp thực tế sau khi chạy chương trình.

Bảng 26 Kiểm thử các ROM

STT	ROM kiểm thử	Mục tiêu quan sát
1	Dodge	- Khi chướng ngại vật rơi trúng nhân vật, emulator phải phát hiện việc vẽ đè pixel và đặt cờ va chạm VF = 1 để dừng trò chơi. - Trò chơi sử dụng bộ sinh số ngẫu nhiên của giả lập để quyết định vị trí xuất hiện của chướng ngại vật rơi xuống. Nếu bộ sinh số ngẫu nhiên lỗi, vật thể sẽ chỉ rơi ở một vài cột cố định. - Đảm bảo emulator ghi nhận phím di chuyển trái/phải lập tức để nhân vật kịp né các vật thể rơi ở tốc độ cao.
2	Brix	- Kiểm tra thuật toán vẽ đè XOR và phản hồi va chạm. - Kiểm tra phản hồi phím điều khiển. - Di chuyển thanh đỡ sát viền để xác minh cơ chế bao màn hình hoạt động ổn định không bị lỗi crash bộ nhớ.
3	Tetris	- Dịch chuyển dữ liệu bit của sprite để xoay khối gạch khi bấm nút. - Phát hiện khối gạch chạm đáy hoặc chạm khối gạch khác đã xếp sẵn để cố định vị trí. - Phản hồi nút xoay gạch, sang trái/phải và thả rơi nhanh.

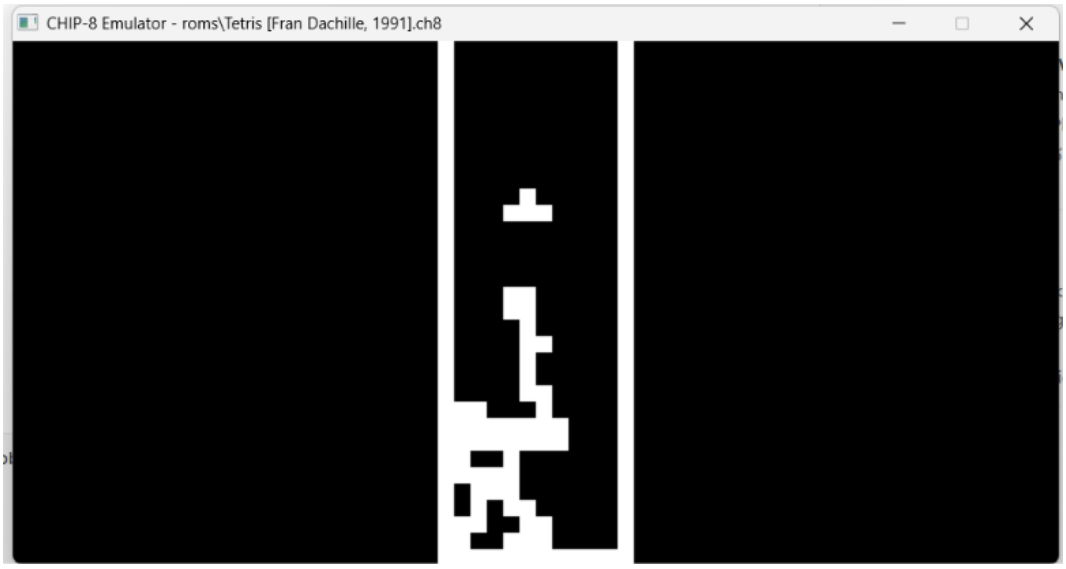
4	Submarine	- Xác minh emulator phát hiện chính xác quả bom chìm chạm vào tàu ngầm để cộng điểm, phá hủy tàu và xóa quả bom. - Tách số thập phân để cập nhật và hiển thị điểm số cùng số lượng bom còn lại lên màn hình. - Sử dụng bộ đếm thời gian để điều khiển tốc độ rơi đều đặn của bom chìm và nhịp di chuyển qua lại của tàu ngầm.
5	Tic-Tac-Toe	- Xác minh emulator tạm dừng CPU đợi và nhận đúng tọa độ phím nhấn (1-9) để đặt quân cờ vào lưới 3x3 - Phát hiện và ngăn chặn (từ chối nước đi) khi người chơi cố tình đánh đè quân cờ lên ô đã có sẵn quân. - Thực thi chuẩn xác các lệnh so sánh nhảy để hoán đổi lượt đi của 2 người chơi và phân định kết quả thắng/thua/hòa.



Hình 18 Kết quả chạy ROM Dodge



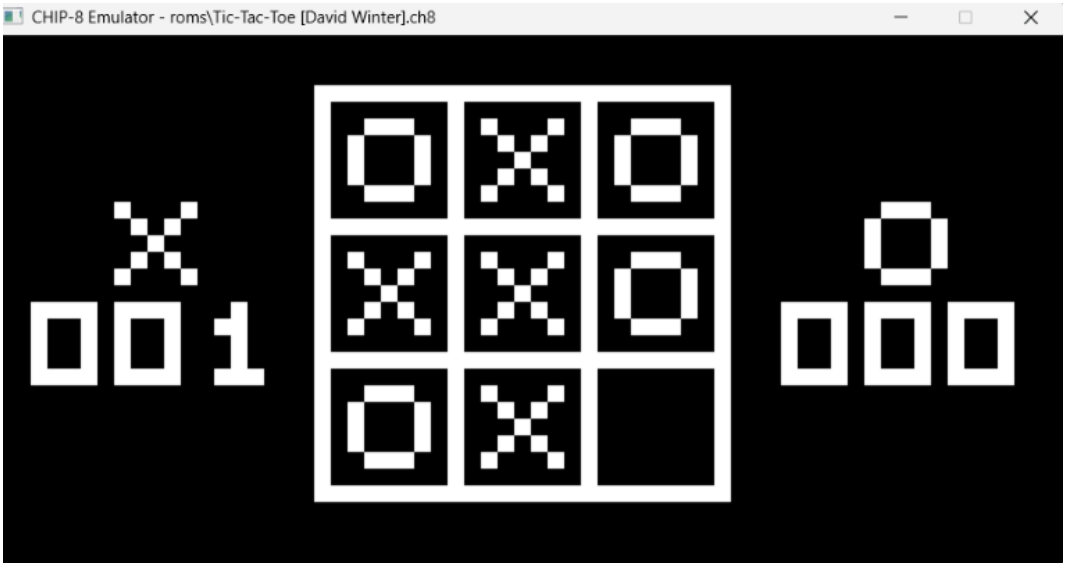
Hình 19 Kết quả chạy ROM Brix



Hình 20 Kết quả chạy ROM Tetris



Hình 21 Kết quả chạy ROM Submarine



Hình 22 Kết quả chạy ROM Tic-Tac-Toe

Bảng 27 Tổng hợp kết quả kiểm thử các ROM

STT	ROM kiểm thử	Kết quả
1	Dodge	Game hoạt động ổn định ở tần số 60Hz. Phát hiện va chạm chính xác giữa người chơi và chướng ngại vật , hệ thống tự động xóa màn hình và khởi động lại (restart) màn chơi mới ngay lập tức. Phím điều khiển nhạy, không giật lag.
2	Brix	Đồ họa và cơ chế va chạm XOR hoạt động chuẩn xác. Quả bóng nảy lại đúng hướng khi chạm vào thanh đỡ hoặc gạch. Gạch bị triệt tiêu và điểm số được cập nhật tức thì. Việc di chuyển thanh đỡ sát rìa màn hình không gây lỗi tràn biên đồ họa.
3	Tetris	Các khối gạch rơi và xếp chồng ổn định nhờ kiểm tra va chạm tốt. Thuật toán dịch bit hỗ trợ xoay khối gạch trơn tru. Bộ giải mã BCD hoạt động chính xác giúp hiển thị điểm số và số hàng đã xóa rõ ràng, không bị lỗi ký tự.
4	Submarine	Tàu chiến thả bom chìm bằng phím W nhạy bén. Nhận diện va chạm chuẩn giữa bom chìm và tàu ngầm di chuyển ở các tầng sâu để tính điểm và trừ số lượng bom chính xác.
5	Tic-Tac-Toe	Cơ chế dừng CPU đợi người chơi nhấn phím hoạt động hoàn hảo. Logic va chạm ngăn chặn thành công việc đi đè quân cờ lên ô đã có sẵn. Hệ thống tính toán so sánh rẽ nhánh phát hiện đúng điều kiện thắng/thua/hòa của trận đấu.

Nhìn chung việc chạy thử nghiệm thực tế các ROM game có mức độ phức tạp khác nhau, hệ thống giả lập CHIP-8 đã chứng minh được tính đúng đắn và độ tin cậy trong thiết kế phần cứng ảo. Tuy nhiên, các ROM kiểm thử còn phụ thuộc nhiều vào

các thiết lập đặc trưng (quirks) của từng hệ máy cũ, dẫn đến việc một số tựa game có thể hoạt động chưa đồng nhất trên các môi trường giả lập khác nhau.

CHƯƠNG 5 KẾT LUẬN

5.1. Kết luận

Sau quá trình tìm hiểu, phân tích và xây dựng chương trình giả lập CHIP-8, nhóm đã hoàn thành được các mục tiêu cơ bản của đề tài. Chương trình có khả năng nạp và thực thi các file ROM CHIP-8, xử lý các lệnh opcode, hiển thị màn hình game và nhận thao tác điều khiển từ bàn phím người dùng. Thông qua đề tài này, nhóm đã hiểu rõ hơn về nguyên lý hoạt động của một trình giả lập, đặc biệt là chu trình Fetch - Decode - Execute. Bên cạnh đó, nhóm cũng nắm được kiến trúc cơ bản của hệ thống CHIP-8 như bộ nhớ, thanh ghi, stack, bộ đếm chương trình, bộ định thời, màn hình và bàn phím.

Về mặt lập trình, chương trình được xây dựng bằng ngôn ngữ C++ theo hướng đối tượng. Các chức năng được tách thành những lớp riêng như Chip8 và Platform, trong đó lớp Chip8 đảm nhiệm phần xử lý logic giả lập, còn lớp Platform đảm nhiệm phần hiển thị và nhập liệu thông qua thư viện SDL2. Cách tổ chức này giúp mã nguồn rõ ràng, dễ theo dõi và thuận tiện cho việc bảo trì. Kết quả thực hiện cho thấy chương trình có thể chạy được nhiều ROM/game CHIP-8 khác nhau, hiển thị hình ảnh đúng cơ chế màn hình 64 x 32 pixel và phản hồi được thao tác bàn phím của người dùng. Điều này chứng minh chương trình đã mô phỏng được các chức năng cơ bản của hệ thống CHIP-8.

Tuy nhiên, chương trình vẫn còn một số hạn chế như chưa hỗ trợ đầy đủ các tính năng nâng cao, giao diện còn đơn giản và khả năng kiểm thử vẫn chủ yếu dựa trên việc chạy thử các ROM cụ thể. Đây cũng là cơ sở để nhóm tiếp tục cải tiến và phát triển chương trình trong tương lai.

5.2. Hướng phát triển

Trong thời gian tới, chương trình giả lập CHIP-8 có thể được tiếp tục cải tiến và mở rộng để hoạt động hoàn thiện hơn.

Trước hết, chương trình có thể được bổ sung giao diện chọn ROM trực quan hơn. Thay vì nhập tên ROM hoặc đường dẫn ROM bằng dòng lệnh, người dùng có thể chọn game thông qua menu hoặc cửa sổ giao diện, giúp việc sử dụng dễ dàng hơn.

Thứ hai, nhóm có thể cải thiện khả năng kiểm thử và độ chính xác của trình giả lập. Chương trình có thể được kiểm tra thêm với nhiều ROM CHIP-8 khác nhau, đặc

biệt là các ROM dùng để test opcode, nhằm đảm bảo các lệnh được xử lý đúng theo đặc tả của CHIP-8.

Thứ ba, chương trình có thể bổ sung tính năng âm thanh. CHIP-8 có bộ định thời âm thanh, vì vậy việc thêm âm thanh khi sound timer hoạt động sẽ giúp chương trình mô phỏng đầy đủ hơn so với hệ thống gốc.

Ngoài ra, chương trình có thể được cải thiện về giao diện hiển thị, chẳng hạn như cho phép thay đổi màu sắc màn hình, phóng to thu nhỏ linh hoạt hoặc hiển thị thông tin ROM đang chạy. Những cải tiến này giúp chương trình thân thiện hơn với người dùng. Một hướng phát triển khác là bổ sung chức năng tạm dừng, tiếp tục chạy, reset game hoặc lưu trạng thái chương trình. Các chức năng này sẽ giúp người dùng thuận tiện hơn trong quá trình chơi và kiểm thử game.

Cuối cùng, nhóm có thể tối ưu lại cấu trúc mã nguồn và bổ sung tài liệu hướng dẫn sử dụng chi tiết hơn. Điều này giúp chương trình dễ bảo trì, dễ mở rộng và thuận lợi hơn nếu tiếp tục phát triển trong các phiên bản sau.

TÀI LIỆU THAM KHẢO

[1] Tony Gaddis, *Starting Out with C++: From Control Structures through Objects*, 9th Edition, Pearson Education, 2018.

[2] Bjarne Stroustrup, *Programming: Principles and Practice Using C++*, 2nd Edition, Addison-Wesley Professional, 2014.

[3] Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.

[4] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.

[5] David Winter, *CHIP-8 Technical Reference*, 1997. Truy cập tại: http://devernay.free.fr/hacks/chip8/C8TECH10.HTM

[6] Tobias V. Langhoff, *Guide to making a CHIP-8 emulator*. Truy cập tại: https://tobiasvl.github.io/blog/write-a-chip-8-emulator/ ngày 20/6/2026.

[7] SDL Development Team, *Simple DirectMedia Layer (SDL2) Documentation*. Truy cập tại: https://wiki.libsdl.org 20/6/2026.

[8] Cplusplus.com, *The C++ Resources Network*. Truy cập tại: https://cplusplus.com ngày 20/6/2026.

[9] ISO/IEC, *Programming Languages — C++ (ISO/IEC 14882:2020)*, International Organization for Standardization, 2020.

[10] John Earnest, *Octo – CHIP-8 Development Environment*. Truy cập tại: https://johnearnest.github.io/Octo/ ngày 20/6/2026.